[[CẬP NHẬT: Tên trường]]

[[CẬP NHẬT: Tên khoa]]

ĐỒ ÁN HỌC PHẦN

NHẬP MÔN DỮ LIỆU LỚN

ĐỀ TÀI

TRÌNH BÀY KẾT QUẢ PHÂN TÍCH DỮ LIỆU THƯƠNG MẠI ĐIỆN TỬ OLIST BẰNG NEON SERVERLESS POSTGRES VÀ APACHE SUPERSET

Giai đoạn Analytics → Presentation trong pipeline dữ liệu lớn

Quy trình phụ trách: **Analytics → Presentation**

Lớp: **[[CẬP NHẬT: Mã lớp]]** – Nhóm: **[[CẬP NHẬT: Tên/số nhóm]]**

Giảng viên hướng dẫn: **[[CẬP NHẬT: Giảng viên hướng dẫn]]**

STT	Họ và tên	MSSV	Vai trò chính
1	**[[CẬP NHẬT: Họ tên TV1]]**	**[[CẬP NHẬT: MSSV]]**	Nạp dữ liệu & Neon (trưởng nhóm)
2	**[[CẬP NHẬT: Họ tên TV2]]**	**[[CẬP NHẬT: MSSV]]**	Semantic layer & SQL
3	**[[CẬP NHẬT: Họ tên TV3]]**	**[[CẬP NHẬT: MSSV]]**	Dashboard Superset
4	**[[CẬP NHẬT: Họ tên TV4]]**	**[[CẬP NHẬT: MSSV]]**	Minh chứng & kiểm thử
5	**[[CẬP NHẬT: Họ tên TV5]]**	**[[CẬP NHẬT: MSSV]]**	Báo cáo & tài liệu

[[CẬP NHẬT: Thành phố]], tháng 11 năm 2026

Tóm tắt

Đồ án thực hiện giai đoạn **Analytics → Presentation** của pipeline dữ liệu lớn trên bộ dữ liệu thương mại điện tử Olist (Brazil, khoảng 100 nghìn đơn hàng 2016–2018, 1,55 triệu dòng thô). Đầu vào là 16 bảng Parquet (194.132 dòng) do nhóm Analytics tính trên cụm Apache Spark, bao phủ sáu chủ đề: xu hướng KPI, danh mục, địa lý – logistics, phân khúc khách hàng RFM/K-Means, người bán, giao trễ – đánh giá – thanh toán. Nhóm xây dựng tầng phục vụ gồm: loader Python (pyarrow + COPY) nạp dữ liệu vào **Neon Serverless Postgres** (tách compute/storage, tự co giãn, scale-to-zero), kèm 7 bảng gold để lọc chéo; tầng ngữ nghĩa 22 view SQL chuẩn hoá định nghĩa KPI; dashboard **Apache Superset** (Docker) với thẻ KPI, xu hướng, Pareto, bản đồ bang, bộ lọc native và khu vực gợi ý quyết định (7 luật có ngưỡng, danh sách người bán cần theo dõi) cho giai đoạn Decision/Action. Kết quả: 505.592 dòng (23 bảng) được nạp trong 351,34 giây, 23/23 bảng khớp số dòng; 19/19 kiểm tra số neo đạt (doanh thu 13.494.400,74 BRL; đỉnh 11/2017 tăng 52,06%; đơn trễ 2,56 sao so với 4,29; 3,04% khách mua lại); trung vị độ trễ truy vấn dashboard 173,4 ms (thực thi trên server 1,25 ms). Hệ thống tái lập được bằng Docker Compose và mã nguồn công khai.

Từ khoá: Big Data, Parquet, Apache Spark, serving layer, Neon, serverless PostgreSQL, semantic layer, Apache Superset, dashboard, Olist.

Thông tin nhóm

Nhóm gồm năm thành viên, phụ trách quy trình con **Analytics → Presentation** (Mục 4 của đề bài). Danh sách thành viên và vai trò chính được trình bày trên trang bìa; bảng phân công chi tiết theo nhiệm vụ thực tế và mức đóng góp của từng người nằm ở Phụ lục G. Mã nguồn, cấu hình và minh chứng được lưu tại kho https://github.com/viettuong287/BigDataProject.

Hạng mục	Thông tin
Học phần	Nhập môn Dữ liệu lớn
Quy trình phụ trách	Analytics → Presentation
Giai đoạn trước (đầu vào)	Processing → Analytics: 16 bảng Parquet + `ANALYTICS_HANDOVER.md` (kèm 7 bảng gold chi tiết)
Giai đoạn sau (đầu ra)	Presentation → Decision/Action: view `presentation.v_decision_hints`, `v_seller_watchlist` trên Neon
Kho mã nguồn	https://github.com/viettuong287/BigDataProject
Ngày sinh báo cáo	12/11/2026

MỤC LỤC

Tóm tắt i

Thông tin nhóm ii

Danh mục từ viết tắt xiii

1. Giới thiệu 1

1.1. Bối cảnh 1

1.2. Quy trình được lựa chọn và ranh giới 1

1.3. Mục tiêu và phạm vi 2

1.4. Câu hỏi kỹ thuật 2

1.5. Cấu trúc báo cáo 3

2. Cơ sở và công nghệ liên quan 4

2.1. Đặc trưng dữ liệu lớn và vai trò tầng trình bày 4

2.2. Data Lake và định dạng cột Apache Parquet 4

2.3. Xử lý phân tán bằng Apache Spark ở giai đoạn trước 4

2.4. OLAP, bảng tổng hợp và tầng phục vụ 6

2.5. PostgreSQL và Neon Serverless Postgres 6

2.6. Tầng ngữ nghĩa và Apache Superset 7

2.7. Nguyên tắc thiết kế dashboard 7

2.8. Các phương án công nghệ thay thế 8

3. Dữ liệu và yêu cầu 9

3.1. Nguồn dữ liệu và dòng dõi 9

3.2. Đặc trưng Big Data của bài toán 9

3.3. Input 10

3.4. Output 11

3.5. Data contract giữa Analytics và Presentation 11

3.6. Yêu cầu chức năng và phi chức năng 13

3.7. Định nghĩa KPI 14

4. Thiết kế hệ thống 15
4.1. Kiến trúc tổng thể 15
4.2. Luồng dữ liệu chi tiết 17
4.3. Thiết kế tầng lưu trữ trên Neon 18
4.3.1. Tổ chức project, database và schema 18
4.3.2. Sinh DDL tự động và ánh xạ kiểu 19
4.3.3. Nạp idempotent, nhật ký và chỉ mục 19
4.4. Thiết kế tầng ngữ nghĩa 19
4.5. Thiết kế dashboard theo người dùng nghiệp vụ 20
4.6. Giao diện với giai đoạn Decision/Action 22
4.7. Lựa chọn công nghệ và lý do 23
4.8. Cấu hình chính 23
5. Cài đặt và triển khai 25
5.1. Môi trường 25
5.2. Cấu trúc mã nguồn 25
5.3. Khởi tạo Neon 25
5.4. Loader Parquet → Neon 26
5.5. Tạo tầng ngữ nghĩa 27
5.6. Triển khai Apache Superset 28
5.7. Xử lý lỗi và các vấn đề kỹ thuật 29
5.8. Cách chạy các job và dịch vụ 29
6. Thực nghiệm và đánh giá 31
6.1. Mục tiêu và kịch bản thực nghiệm 31
6.2. Dữ liệu và môi trường thử nghiệm 31
6.3. Tiêu chí đánh giá 31
6.4. Kết quả E1 – Đối chiếu dữ liệu 32
6.5. Kết quả E2 – Hiệu năng nạp dữ liệu 33
6.6. Kết quả E3, E4 – Độ trễ truy vấn 34

6.7. Kết quả E5, E6 – Dashboard và bộ lọc 36
6.8. Tham chiếu hiệu năng giai đoạn trước 37
6.9. Đánh giá tổng hợp 37
7. Kết quả demo 38
7.1. Kịch bản demo từ đầu vào đến đầu ra 38
7.2. Nội dung phân tích được trình bày 38
7.2.1. Tổng quan: tăng trưởng và sức khoẻ vận hành 39
7.2.2. Danh mục: Pareto và mùa vụ 39
7.2.3. Địa lý và logistics 40
7.2.4. Khách hàng: giữ chân và phân khúc 41
7.2.5. Người bán 42
7.2.6. Giao trễ, đánh giá và thanh toán 43
7.3. Ảnh chụp hệ thống đang chạy 43
7.4. Đầu ra cho giai đoạn Decision/Action 50
8. Thảo luận 52
8.1. Điểm mạnh 52
8.2. Hạn chế 52
8.3. Khả năng mở rộng 53
8.4. Bài học kỹ thuật và hướng cải tiến 53
9. Kết luận 55
Tài liệu tham khảo 56
Phụ lục A. Hướng dẫn chạy 58
A.1. Yêu cầu hệ thống 58
A.2. Các bước chạy 58
A.3. Sinh lại báo cáo này 59
A.4. Xử lý sự cố thường gặp 59
Phụ lục B. Cấu hình 60
Phụ lục C. Schema dữ liệu 66

C.1. Ánh xạ kiểu và ví dụ bản ghi 66
C.2. Quy mô các bảng đầu vào 67
C.3. Schema 16 bảng Analytics 68
Phụ lục D. Tầng ngữ nghĩa: danh mục view và mã SQL 73
Phụ lục E. Minh chứng thực nghiệm chi tiết 79
E.1. Kết quả và log nạp dữ liệu 79
E.2. Log áp dụng tầng ngữ nghĩa 80
E.3. Đối chiếu số dòng và giá trị kỳ vọng 81
E.4. Danh sách tệp minh chứng 82
Phụ lục F. Mã nguồn 83
Phụ lục G. Bảng phân công và đóng góp 85
Phụ lục H. Ghi chú sử dụng công cụ AI 86

DANH MỤC HÌNH

Hình 2.1. Spark Master UI (cổng 8080) của nhóm Analytics: 2 worker ALIVE, 4 core, ứng dụng olist-analytics FINISHED 5

Hình 2.2. Spark Worker UI (cổng 8081): worker 2 core và executor của ứng dụng Analytics 6

Hình 4.1. Sơ đồ kiến trúc và luồng dữ liệu: Parquet → Loader (COPY) → Neon (analytics/gold → presentation) → Superset → người dùng / Decision-Action 16

Hình 4.2. Sơ đồ kiến trúc / luồng dữ liệu của hệ thống – architecture pipeline (tệp architecture_pipeline.png) 16

Hình 4.3. Sơ đồ kiến trúc hệ thống Analytics → Presentation (vẽ bởi nhóm) 17

Hình 4.4. Sơ đồ kiến trúc / luồng dữ liệu của hệ thống – architecture pipeline position (tệp architecture_pipeline_position.png) 17

Hình 5.1. Neon Console: project, nhánh và compute của tầng phục vụ – neon tables inventory (tệp neon_tables_inventory.png) 26

Hình 5.2. Truy vấn semantic view (SQL Lab / psql) – dashboard tab6 giao tre review thanh toan (tệp dashboard_tab6_giao_tre_review_thanh_toan.png) 28

Hình 7.1. Doanh thu, số đơn, tỷ lệ giao trễ và điểm review theo tháng (2017-01 → 2018-08; cột cam = Black Friday) 39

Hình 7.2. Pareto doanh thu theo danh mục: không danh mục nào vượt 10%, top 10 đạt ~62% 39

Hình 7.3. Mùa vụ 2017: tỷ trọng doanh thu từng tháng trong năm của 8 danh mục lớn nhất 40

Hình 7.4. 27 bang theo số ngày giao trung bình và điểm review (kích thước = doanh thu, màu = % giao trễ) 40

Hình 7.5. Phí vận chuyển, thời gian giao và tỷ lệ trễ theo khoảng cách người bán → khách 41

Hình 7.6. Số khách theo phân khúc RFM (thang log): ba phân khúc khách mua lại chỉ chiếm ~3% 41

Hình 7.7. Đường Pareto doanh thu của người bán 42

Hình 7.8. Điểm review trung bình và tỷ lệ đơn 1 sao theo số ngày giao trễ so với hẹn43

Hình 7.9. Dashboard Apache Superset – dashboard tab1 tong quan xu huong (tệp dashboard_tab1_tong_quan_xu_huong.png) 44

Hình 7.10. Dashboard Apache Superset – dashboard tab2 danh muc mua vu (tệp dashboard_tab2_danh_muc_mua_vu.png) 44

Hình 7.11. Dashboard Apache Superset – dashboard tab3 dia ly logistics (tệp dashboard_tab3_dia_ly_logistics.png) 45

Hình 7.12. Dashboard Apache Superset – dashboard tab4 khach hang rfm (tệp dashboard_tab4_khach_hang_rfm.png) 45

Hình 7.13. Dashboard Apache Superset – dashboard tab5 seller (tệp dashboard_tab5_seller.png) 46

Hình 7.14. Dashboard Apache Superset – dashboard tab7 goi y quyet dinh nguon du lieu (tệp dashboard_tab7_goi_y_quyet_dinh_nguon_du_lieu.png) 46

Hình 7.15. Dashboard khi áp dụng bộ lọc native – filter tab1 thoi gian 2018 bang SP (tệp filter_tab1_thoi_gian_2018_bang_SP.png) 47

Hình 7.16. Dashboard khi áp dụng bộ lọc native – filter tab2 danh muc bed bath health watches (tệp filter_tab2_danh_muc_bed_bath_health_watches.png) 47

Hình 7.17. Dashboard khi áp dụng bộ lọc native – filter tab3 vung nordeste (tệp filter_tab3_vung_nordeste.png) 48

Hình 7.18. Dashboard khi áp dụng bộ lọc native – filter tab4 rfm khach mua lai bang RJ (tệp filter_tab4_rfm_khach_mua_lai_bang_RJ.png) 48

Hình 7.19. Dashboard khi áp dụng bộ lọc native – filter tab5 bang seller SP (tệp filter_tab5_bang_seller_SP.png) 49

Hình 7.20. Dashboard khi áp dụng bộ lọc native – filter tab6 pttt boleto (tệp filter_tab6_pttt_boleto.png) 49

Hình 7.21. Khu vực gợi ý quyết định trên dashboard – watchlist goi y quyet dinh (tệp watchlist_goi_y_quyet_dinh.png) 50

Hình 7.22. Khu vực gợi ý quyết định trên dashboard – watchlist seller canh bao (tệp watchlist_seller_canh_bao.png) 50

Hình C.1. Số dòng của từng bảng Analytics (thang log): hai bảng cấp khách hàng chiếm gần như toàn bộ khối lượng 67

DANH MỤC BẢNG

Bảng 2.1. So sánh công cụ trình bày (BI) cho giai đoạn Presentation 8

Bảng 2.2. So sánh phương án tầng lưu trữ phục vụ 8

Bảng 3.1. Dòng dõi dữ liệu từ nguồn đến tầng phục vụ 9

Bảng 3.2. Các thành phần đầu ra của giai đoạn Presentation 11

Bảng 3.3. Data contract Analytics → Presentation: danh sách bảng, grain và khoá 11

Bảng 3.4. Yêu cầu chức năng (F) và phi chức năng (N) 13

Bảng 3.5. Định nghĩa KPI và quy tắc tổng hợp trên dashboard 14

Bảng 4.1. Các bước của luồng dữ liệu, định dạng và giao thức 17

Bảng 4.2. Cấu hình tầng phục vụ Neon 18

Bảng 4.3. Thiết kế dashboard theo tab và nhóm người dùng 21

Bảng 4.4. Các luật gợi ý quyết định trong v_decision_hints 22

Bảng 4.5. Công nghệ sử dụng, vai trò và lý do lựa chọn 23

Bảng 5.1. Môi trường cài đặt và triển khai 25

Bảng 5.2. Các vấn đề kỹ thuật và cách xử lý 29

Bảng 6.1. Các kịch bản thực nghiệm 31

Bảng 6.2. Đối chiếu số neo bàn giao của Analytics và tính nhất quán giữa các view trên Neon 32

Bảng 6.3. Độ trễ truy vấn các view trên Neon (9/9 phép đo thành công; tệp `evidence/dashboard_load_latency.md`) 34

Bảng 6.4. Độ trễ truy vấn các view trên Neon (22/22 phép đo thành công; tệp `evidence/query_latency.csv`) 34

Bảng 6.5. Độ trễ truy vấn đo qua Superset (44/44 phép đo thành công; tệp `evidence/superset_chart_latency.csv`) 35

Bảng 6.6. Kết quả kiểm thử dashboard và bộ lọc 36

Bảng 6.7. Thời gian xử lý theo giai đoạn của pipeline 37

Bảng 6.8. Đánh giá mức đạt theo tiêu chí 37

Bảng A.1. Sự cố thường gặp khi chạy lại hệ thống 59

Bảng C.1. Ánh xạ kiểu dữ liệu Parquet → PostgreSQL do loader áp dụng 66

Bảng C.2. Quy mô thực tế của các bảng Parquet Analytics (đọc trực tiếp bằng pyarrow khi sinh báo cáo) 67

Bảng C.3. Các bảng gold được nạp bổ sung vào schema gold (đầu vào chi tiết cho bộ lọc chéo) 68

Bảng C.4. Schema bảng `kpi_monthly` (20 dòng; 1 dòng = 1 tháng (2017-01 → 2018-08)) 68

Bảng C.5. Schema bảng `category_pareto` (74 dòng; 1 dòng = 1 danh mục) 68

Bảng C.6. Schema bảng `category_seasonality` (715 dòng; 1 dòng = 1 danh mục × 1 tháng (2017)) 69

Bảng C.7. Schema bảng `state_logistics` (27 dòng; 1 dòng = 1 bang của khách) 69

Bảng C.8. Schema bảng `insight_distance` (4 dòng; 1 dòng = 1 khoảng cách (band km)) 69

Bảng C.9. Schema bảng `rfm_scores` (94.983 dòng; 1 dòng = 1 khách thật) 69

Bảng C.10. Schema bảng `customer_segments` (94.983 dòng; 1 dòng = 1 khách thật (+ cụm K-Means)) 70

Bảng C.11. Schema bảng `cluster_profile` (4 dòng; 1 dòng = 1 cụm K-Means) .. 70

Bảng C.12. Schema bảng `repeat_profile` (2 dòng; 1 dòng = 1 cohort (repeat / one_shot)) 70

Bảng C.13. Schema bảng `repeat_categories` (70 dòng; 1 dòng = 1 danh mục của khách mua lại) 70

Bảng C.14. Schema bảng `repeat_states` (27 dòng; 1 dòng = 1 bang của khách mua lại) 70

Bảng C.15. Schema bảng `seller_pareto` (3.053 dòng; 1 dòng = 1 người bán) 71

Bảng C.16. Schema bảng `seller_watchlist` (153 dòng; 1 dòng = 1 người bán cần theo dõi) 71

Bảng C.17. Schema bảng `insight_late_vs_review` (2 dòng; 1 dòng = 1 nhóm (late / on_time)) 71

Bảng C.18. Schema bảng `delay_vs_review` (5 dòng; 1 dòng = 1 khoảng ngày trễ) 71

Bảng C.19. Schema bảng payment_mix (10 dòng; 1 dòng = 1 PTTT × 1 nhóm kỳ trả góp) 71

Bảng D.1. Các đối tượng của tầng ngữ nghĩa: 22 view và 1 bảng dimension (quét tự động từ sql/10_presentation_views.sql) 73

Bảng E.1. Kết quả nạp từng bảng, lần chạy 20261112-145508-40efaf (tệp evidence/load_log.csv) 79

Bảng E.2. Đối chiếu số dòng Parquet ↔ Neon: 23/23 bảng khớp (tệp evidence/validation.json) 81

Bảng E.3. Bộ kiểm tra đối chiếu Parquet ↔ Neon và giá trị kỳ vọng 81

Bảng E.4. Danh sách tệp minh chứng trong evidence/ 82

Bảng F.1. Vị trí các thành phần trong repo 84

Bảng G.1. Bảng phân công nhiệm vụ và mức đóng góp của các thành viên 85

Bảng G.2. Ma trận đóng góp theo sản phẩm bàn giao (C = chính, H = hỗ trợ) 85

Bảng H.1. Ghi nhận sử dụng công cụ AI 86

Danh mục từ viết tắt

Viết tắt	Diễn giải
AOV	Average Order Value – giá trị đơn hàng trung bình
BI	Business Intelligence – trí tuệ doanh nghiệp
BRL	Real Brazil (đơn vị tiền tệ)
CU	Compute Unit của Neon (≈ 1 vCPU + 4 GB RAM)
DDL	Data Definition Language (CREATE/ALTER/DROP)
ETL/ELT	Extract – Transform – Load
GMV	Gross Merchandise Value = doanh thu hàng + phí vận chuyển
KPI	Key Performance Indicator – chỉ số hiệu suất chính
MoM / YoY	Month-over-Month / Year-over-Year
OLAP / OLTP	Online Analytical / Transaction Processing
PTTT	Phương thức thanh toán
RFM	Recency – Frequency – Monetary
SLA	Service Level Agreement – cam kết mức dịch vụ
TLS	Transport Layer Security

1. Giới thiệu

1.1. Bối cảnh

Trong hệ thống dữ liệu lớn, giá trị kinh doanh chỉ xuất hiện khi kết quả phân tích đến được người ra quyết định, đúng lúc và dễ hiểu. Các kiến trúc tham chiếu như Lambda [8] hay Lakehouse [9] đều có một **tầng phục vụ (serving layer)**: dữ liệu sau khi được xử lý phân tán được cô đọng thành bảng tổng hợp, nạp vào hệ thống trả lời truy vấn tương tác với độ trễ thấp, rồi được trình bày qua dashboard hoặc API. Thiếu tầng này, kết quả Spark chỉ là các tệp Parquet trong Data Lake mà người dùng nghiệp vụ không khai thác được.

Bộ dữ liệu dùng xuyên suốt pipeline của lớp là **Brazilian E-Commerce Public Dataset by Olist** [1]: khoảng 100 nghìn đơn hàng trên các sàn thương mại điện tử Brazil từ 09/2016 đến 10/2018. Dữ liệu đã qua các giai đoạn trước: nhóm Storage → Processing dựng tầng gold (7 bảng, đối chiếu 11/11); nhóm Processing → Analytics chạy 14 truy vấn Spark SQL và K-Means trên cụm Spark, bàn giao 16 bảng Parquet kèm tài liệu `ANALYTICS_HANDOVER.md`. Nhóm chúng tôi đưa các kết quả này tới người dùng nghiệp vụ của sàn: ban điều hành, quản lý ngành hàng, vận hành – logistics, CRM – marketing và đội quản lý người bán.

1.2. Quy trình được lựa chọn và ranh giới

Nhóm thực hiện quy trình con **Analytics → Presentation** (Mục 4 của đề bài), với ranh giới:

- **Điểm bắt đầu:** 16 bảng Parquet trong `data/analytics/` cùng tài liệu bàn giao của Analytics; nạp kèm 7 bảng gold (311.460 dòng, cũng là đầu vào của Analytics) để dashboard lọc chéo được ở cấp đơn hàng. Nhóm **không** đọc lại CSV thô, **không** chạy lại Spark và **không** tính lại các phân tích phức tạp (K-Means, Pareto, RFM). Mọi chỉ số trên dashboard truy ngược được về một cột của Analytics hoặc gold, và tổng từ gold phải khớp số của Analytics.
- **Điểm kết thúc:** (1) cơ sở dữ liệu phục vụ trên Neon đã nạp và kiểm tra; (2) tầng ngữ nghĩa gồm các view SQL định nghĩa KPI thống nhất; (3) dashboard Superset có KPI, bộ lọc, biểu đồ và nguồn dữ liệu rõ ràng; (4) các view **gợi ý quyết định** cho giai đoạn Presentation → Decision/Action.

- **Giao diện với giai đoạn trước:** *data contract* gồm danh sách bảng, grain, khoá, kiểu dữ liệu và các số neo đối chiếu (Chương 3). **Với giai đoạn sau:** các view ổn định trong schema `presentation`, đọc bằng SQL, không phụ thuộc Superset.

1.3. Mục tiêu và phạm vi

Mục tiêu là xây dựng tầng trình bày **đúng số liệu, tương tác được, tái lập được và mở rộng được**:

1. Nạp đầy đủ, chính xác các bảng Parquet vào cơ sở dữ liệu phục vụ trên đám mây, bảo toàn kiểu dữ liệu (đặc biệt tiền tệ `DECIMAL`), có nhật ký nạp và kiểm tra đối chiếu tự động.
2. Xây dựng tầng ngữ nghĩa trong đó mỗi KPI có một định nghĩa và cách tổng hợp đúng (kể cả chỉ số không cộng dồn như tỷ lệ, trung bình).
3. Triển khai dashboard Superset cho năm nhóm người dùng, có bộ lọc native và khu vực gợi ý quyết định.
4. Đo thời gian nạp, thông lượng, độ trễ truy vấn và tác động của scale-to-zero, bộ nhớ đệm.
5. Đóng gói bằng Docker Compose và mã nguồn để người khác chạy lại.

Ngoài phạm vi: thu thập, làm sạch, tính toán phân tích (các nhóm trước); thực thi hành động tự động như gửi email, gọi webhook (nhóm Decision/Action). Dữ liệu được xử lý theo lô vì nguồn Olist là dữ liệu lịch sử.

1.4. Câu hỏi kỹ thuật

- **RQ1 – Lưu trữ phục vụ:** phục vụ kết quả từ đâu (Parquet trực tiếp, Postgres cục bộ, ClickHouse hay Postgres serverless) để cân bằng độ trễ, chi phí, khả năng chia sẻ và công vận hành? (Chương 2, 4)
- **RQ2 – Tính đúng đắn:** làm sao đảm bảo số trên dashboard khớp tuyệt đối kết quả Spark, kể cả khi người dùng lọc và gộp chỉ số không cộng dồn? (Chương 3, 6)
- **RQ3 – Hiệu năng:** thời gian nạp và độ trễ truy vấn là bao nhiêu; nút thắt nằm ở đâu? (Chương 6)
- **RQ4 – Thiết kế trình bày:** bố cục, biểu đồ, bộ lọc nào giúp từng nhóm người dùng trả lời câu hỏi nhanh nhất? (Chương 4, 7)

- **RQ5 – Mở rộng và tích hợp:** kiến trúc thay đổi ra sao khi dữ liệu tăng 10–100 lần, và giai đoạn sau tiêu thụ kết quả thế nào? (Chương 8, 9)

1.5. Cấu trúc báo cáo

Chương 2 trình bày cơ sở và công nghệ; Chương 3 mô tả dữ liệu, đặc trưng Big Data, đầu vào, đầu ra và data contract; Chương 4 trình bày kiến trúc và thiết kế; Chương 5 mô tả cài đặt; Chương 6 thực nghiệm; Chương 7 kết quả demo; Chương 8 thảo luận; Chương 9 kết luận. Phụ lục chứa hướng dẫn chạy, cấu hình, schema, mã SQL, minh chứng chi tiết, mã nguồn, bảng phân công và ghi chú sử dụng AI.

2. Cơ sở và công nghệ liên quan

Chương này chỉ trình bày phần kiến thức Big Data dùng trực tiếp trong đồ án.

2.1. Đặc trưng dữ liệu lớn và vai trò tầng trình bày

Dữ liệu lớn được mô tả bằng các chữ V: **Volume** (khối lượng), **Velocity** (tốc độ phát sinh), **Variety** (đa dạng), **Veracity** (độ tin cậy) và **Value** (giá trị). Pipeline gồm Sources → Ingestion → Storage → Processing → Analytics → Presentation → Decision/Action; mỗi giai đoạn giao tiếp qua định dạng dữ liệu thống nhất. Đặc điểm quan trọng là **khối lượng giảm dần theo từng tầng trong khi giá trị trên mỗi byte tăng dần**: triệu dòng thô thành trăm nghìn dòng fact, rồi bảng tổng hợp, cuối cùng là vài chục KPI. Vì vậy thách thức của tầng Presentation không phải quét petabyte, mà là (1) phục vụ **nhiều người dùng đồng thời với độ trễ tương tác**, (2) giữ **nhất quán ngữ nghĩa** giữa số do hệ thống phân tán tính và số người dùng thấy, (3) **co giãn** theo tải truy cập rất không đều.

2.2. Data Lake và định dạng cột Apache Parquet

Data Lake lưu dữ liệu dạng tệp theo nguyên tắc *schema-on-read*; nhiều công cụ (Spark, pyarrow, DuckDB, Trino) đọc trực tiếp mà không cần máy chủ cơ sở dữ liệu. Các tầng silver, gold, analytics của pipeline Olist đều là thư mục Parquet.

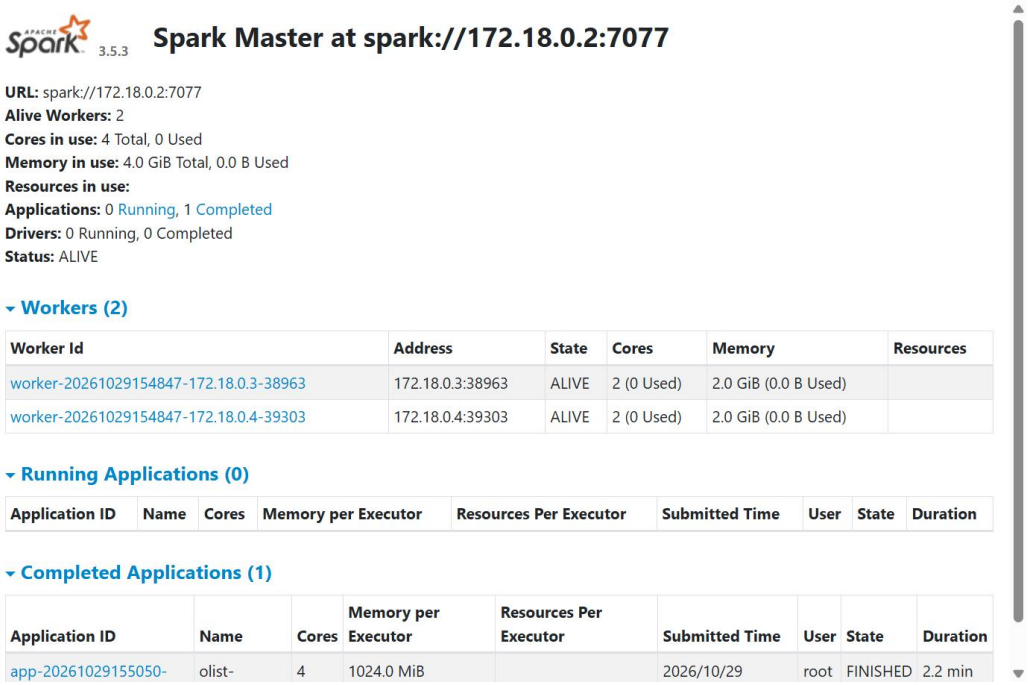
Apache Parquet [5] là định dạng hướng cột kế thừa Dremel [6]: tệp chia thành *row group*, trong đó dữ liệu từng cột nằm liền nhau (*column chunk*, chia thành *page*). Lợi ích [7]: chỉ đọc cột cần thiết; nén tốt nhờ mã hoá từ điển, run-length kết hợp Snappy (bảng `customer_segments` 94.983 dòng × 13 cột chỉ khoảng 3,9 MB); thống kê min/max cho phép bỏ qua khối dữ liệu (predicate pushdown); schema tự mô tả với kiểu logic `DECIMAL`, `DATE`, `TIMESTAMP` — nhờ đó loader của nhóm **tự sinh DDL PostgreSQL**. Tuy nhiên Parquet không phải hệ thống phục vụ: không có chỉ mục tra cứu điểm, không kiểm soát truy cập theo người dùng, không phục vụ nhiều client đồng thời — nên cần một tầng phục vụ riêng.

2.3. Xử lý phân tán bằng Apache Spark ở giai đoạn trước

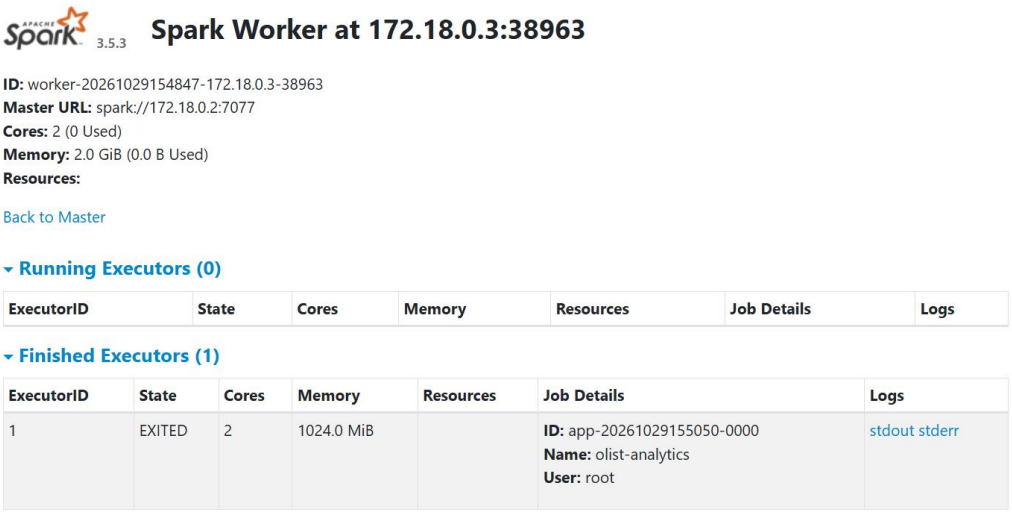
Các bảng đầu vào được sinh bởi **Apache Spark** [3], [4] — công cụ xử lý phân tán trong bộ nhớ, kế thừa MapReduce [2] nhưng biểu diễn tính toán thành DAG các

phép biến đổi trên dữ liệu phân vùng. *Driver* lập kế hoạch, *executor* trên worker chạy *task* trên từng *partition*; `GROUP BY`, `JOIN`, window function sinh bước *shuffle*; trình tối ưu Catalyst chọn broadcast hoặc sort-merge join.

Nhóm Analytics chạy job trên cụm Docker 1 master + 2 worker (tổng 4 core), Spark 3.5.3, `spark.sql.shuffle.partitions = 16`. Job mất 119,71 giây (SQL 44,51 giây, K-Means 29,50 giây); bảng KPI nhỏ được `coalesce(1)` thành một tệp. Hai hình dưới là ảnh Spark UI do nhóm Analytics cung cấp, chứng minh job chạy phân tán. Lưu ý cho tầng Presentation: các window toàn cục (Pareto cộng dồn, NTILE của RFM) đã được Spark tính; nếu dashboard tính lại trên dữ liệu đã lọc sẽ ra kết quả khác, nên nhóm coi chúng là **thuộc tính đã chốt**.



Hình 2.1. *Spark Master UI (cổng 8080) của nhóm Analytics: 2 worker ALIVE, 4 core, ứng dụng olist-analytics FINISHED*



Hình 2.2. *Spark Worker UI (cổng 8081): worker 2 core và executor của ứng dụng Analytics*

2.4. OLAP, bảng tổng hợp và tầng phục vụ

Hệ **OLTP** tối ưu cho giao dịch nhỏ theo dòng; hệ **OLAP** tối ưu cho truy vấn quét nhiều dòng và tổng hợp theo nhiều chiều [11]. Mô hình Kimball [10] tổ chức dữ liệu thành bảng **fact**, **dimension** và **bảng tổng hợp** tính trước theo các tổ hợp chiều thường dùng. Các bảng Analytics chính là bảng tổng hợp ở nhiều *grain*: tháng, danh mục, danh mục × tháng, bang, người bán, khách hàng. Trong kiến trúc Lambda [8], **tầng phục vụ** lập chỉ mục và phục vụ các *batch view* do tầng batch tính ra: ở đây Spark là batch layer, Neon là serving layer.

Khái niệm then chốt là **tính cộng dồn của độ đo** [10]: *cộng dồn được* (doanh thu, số đơn) có thể `SUM` theo mọi chiều; *bán cộng dồn* (số khách duy nhất) cộng theo tháng sẽ đếm trùng; *không cộng dồn* (tỷ lệ, trung bình, % tăng trưởng) không được lấy trung bình của trung bình mà phải tính lại từ tử số và mẫu số. Tầng ngữ nghĩa của nhóm mã hoá các quy tắc này.

2.5. PostgreSQL và Neon Serverless Postgres

PostgreSQL là hệ quản trị quan hệ mã nguồn mở với SQL đầy đủ, giao dịch ACID, view, window function, chỉ mục và lệnh `COPY` nạp hàng loạt nhanh hơn nhiều lần so với `INSERT` từng dòng [14]; hầu hết công cụ BI đều kết nối được.

Neon là Postgres serverless **tách rời compute và storage** [12], tương tự Amazon Aurora [24]. *Compute* là tiến trình Postgres không trạng thái; *storage* gồm Safekeeper (nhân bản WAL bằng đồng thuận Paxos), Pageserver (dựng trang dữ liệu từ WAL, phục vụ compute) và object storage cho lưu trữ dài hạn. Các tính năng dùng trong đồ án [13]: **autoscaling** compute theo Compute Unit (≈ 1 vCPU + 4 GB RAM); **scale-to-zero** tạm dừng compute sau thời gian không hoạt động (mặc định 5 phút), truy vấn kế tiếp chịu độ trễ khởi động; **branching** copy-on-write để thử thay đổi trên dữ liệu thật; **connection pooling** qua hostname `-pooler` chạy PgBouncer chế độ transaction. Mô hình này khớp với **tải không đều** của dashboard: không ai xem thì compute ngủ, nhiều người xem thì compute tự tăng tài nguyên, dữ liệu luôn an toàn trong storage nhân bản.

2.6. Tầng ngữ nghĩa và Apache Superset

Tầng ngữ nghĩa định nghĩa *một lần* các khái niệm nghiệp vụ (chỉ số, chiều phân tích, tên hiển thị, cách tổng hợp) để mọi dashboard, báo cáo, API cho cùng một con số. Có thể đặt nó trong cơ sở dữ liệu (view), trong công cụ BI (dataset, metric) hoặc công cụ riêng (dbt, Cube). Nhóm đặt phần lõi ở **view SQL trên Neon** vì cả Superset lẫn giai đoạn Decision/Action đều đọc trực tiếp được.

Apache Superset [15] là nền tảng BI mã nguồn mở: web server Flask/Gunicorn với giao diện React và REST API; cơ sở dữ liệu metadata (SQLite/PostgreSQL); kết nối **SQLAlchemy** tới hầu hết hệ SQL; bộ nhớ đệm kết quả; tuỳ chọn Celery cho truy vấn bất đồng bộ và báo cáo định kỳ. Khái niệm chính: *Database*, *Dataset* (bảng/view hoặc SQL ảo), *Chart* (chủ yếu Apache ECharts), *Dashboard* (bố cục, tab), *Native filter* và *Cross-filter*. Superset đẩy toàn bộ phép tính xuống cơ sở dữ liệu, nên hiệu năng dashboard phụ thuộc trực tiếp vào tầng phục vụ và cache.

2.7. Nguyên tắc thiết kế dashboard

Thiết kế dựa trên nguyên tắc **"overview first, zoom and filter, then details on demand"** của Shneiderman [16] (thẻ KPI → biểu đồ → bộ lọc → bảng chi tiết) và khuyến nghị của Few [17]: xem được trong một màn hình, ưu tiên cột/đường hơn biểu đồ tròn và đồng hồ đo, màu nổi bật chỉ dành cho ngoại lệ. Biểu đồ được chọn theo câu hỏi: xu hướng → đường/cột theo tháng; tập trung 80/20 → Pareto; so sánh địa lý → bản đồ và scatter; xếp hạng → bảng có định dạng điều kiện. Mỗi biểu đồ ghi rõ nguồn (tên view), đơn vị và khoảng thời gian.

2.8. Các phương án công nghệ thay thế

Bảng 2.1. So sánh công cụ trình bày (BI) cho giai đoạn Presentation

Tiêu chí	Apache Superset	Power BI	Tableau	Grafana	Metabase
Giấy phép	Mã nguồn mở (Apache 2.0)	Thương mại (chia sẻ cần Pro)	Thương mại	Mã nguồn mở (AGPL)	Mã nguồn mở / thương mại
Triển khai	Docker, tự host	Windows Desktop + cloud	Desktop + Server/Cloud	Docker	Docker
Thế mạnh	BI SQL-native, nhiều chart, native filter, REST API	Mô hình DAX, thân thiện	Trực quan hoá linh hoạt	Chuỗi thời gian, giám sát	Dễ dùng
Hạn chế	Cấu hình ban đầu phức tạp	Khó tái lập bằng mã	Chi phí bản quyền	Yếu với phân tích theo danh mục	Ít loại biểu đồ, bộ lọc hạn chế
Dashboard-as-code	Có (REST API, export YAML)	Hạn chế (.pbix nhị phân)	Hạn chế	Có (JSON)	Một phần

Bảng 2.2. So sánh phương án tầng lưu trữ phục vụ

Tiêu chí	Neon Serverless Postgres	Postgres cục bộ (Docker)	ClickHouse	Đọc thẳng Parquet (DuckDB)
Mô hình lưu trữ	Hàng + chỉ mục	Hàng + chỉ mục	Cột (MergeTree)	Cột (tệp)
Phù hợp quy mô	Hàng chục–trăm triệu dòng tổng hợp	Giới hạn một máy	Hàng tỷ dòng chi tiết	Lớn, nhưng không phục vụ đồng thời
Chia sẻ cho nhóm/giảng viên	Có, qua Internet + TLS	Không	Cần tự host trên cloud	Phải chia sẻ tệp
Vận hành	Được quản lý, autoscaling, scale-to-zero	Tự quản lý	Tự quản lý / trả phí	Không có dịch vụ
Tương thích BI	Rất tốt (chuẩn Postgres)	Rất tốt	Tốt (driver riêng)	Trung bình

3. Dữ liệu và yêu cầu

3.1. Nguồn dữ liệu và dòng dõi

Nguồn gốc là bộ **Brazilian E-Commerce Public Dataset by Olist** [1] trên Kaggle: 9 tệp CSV gồm đơn hàng (99.441 dòng), chi tiết đơn (112.650), thanh toán (103.886), đánh giá (99.224), khách hàng (99.441), người bán (3.095), sản phẩm (32.951), toạ độ mã bưu chính (1.000.163) và bảng dịch danh mục (71) — khoảng **1,55 triệu dòng, ~120 MB CSV**, đã ẩn danh hoá.

Bảng 3.1. Dòng dõi dữ liệu từ nguồn đến tầng phục vụ

Tầng	Nhóm phụ trách	Nội dung	Quy mô
Raw (CSV)	Sources → Ingestion	9 tệp CSV Olist	~1,55 triệu dòng, ~120 MB
Silver (Parquet)	Storage → Processing	Làm sạch, chuẩn hoá kiểu, khử trùng lặp	569.698 dòng, 40,37 MB
Gold (Parquet)	Storage → Processing	2 bảng fact + 5 bảng aggregate, đối chiếu 11/11	7 bảng, 33,3 MB, 61,72 s Spark
Analytics (Parquet)	Processing → Analytics	14 truy vấn Spark SQL + K-Means k = 4	16 bảng, 194.132 dòng, 119,71 s Spark
Serving (Neon)	**Analytics → Presentation**	Schema `analytics`, `gold`, `presentation`	23 bảng (505.592 dòng) + 22 view
Dashboard	**Analytics → Presentation**	Superset: KPI, biểu đồ, bộ lọc, gợi ý quyết định	Nhiều tab theo nhóm người dùng

Năm quy tắc nghiệp vụ từ tầng gold phải được giữ nguyên khi trình bày: (1) doanh thu chỉ tính trên đơn hợp lệ `is_valid_sale` (98.199/99.441 đơn); (2) đếm khách bằng `customer_unique_id` (99.441 `customer_id` nhưng chỉ 96.096 khách thật); (3) chuỗi thời gian chỉ dùng 2017-01 → 2018-08; (4) điểm review ở cấp đơn, không lấy trung bình theo dòng sản phẩm; (5) phân tích giao hàng loại đơn có cờ chất lượng `dq_*`.

3.2. Đặc trưng Big Data của bài toán

Đầu vào trực tiếp của nhóm ở mức trăm nghìn dòng, nhưng bài toán vẫn mang đặc trưng dữ liệu lớn và kiến trúc được thiết kế để giữ nguyên khi quy mô tăng:

- **Volume:** dữ liệu nguồn ~1,55 triệu dòng; nhóm Processing có sẵn kịch bản nhân bản ×5, ×10, ×20. Với một sàn thực tế (hàng chục triệu đơn/năm), bảng cấp khách hàng và người bán tăng tuyến tính lên hàng chục triệu dòng; tầng phục vụ phải chịu được mức này mà không đổi kiến trúc (Mục 8.3).
- **Variety:** 16 bảng với 10 grain khác nhau (tháng, danh mục, danh mục × tháng, bang, khoảng cách, khách hàng, cụm, cohort, người bán, nhóm thanh toán), nhiều

kiểu dữ liệu, hai hệ nhãn khách hàng song song (luật RFM và cụm K-Means), cộng thêm bảng fact gold phân vùng Hive.

- **Veracity:** cờ chất lượng `dq_*`, tháng không đầy đủ và các bẫy ngữ nghĩa — tổng 20 tháng của `kpi_monthly` *nhỏ hơn* số neo 13.494.400,74 BRL vì đã cắt tháng rỗng; `f_score` gần như nhiễu vì 97% khách chỉ mua một lần. Nhóm phải kiểm tra đối chiếu tự động để chứng minh số trên dashboard đúng.
- **Value:** kết quả phải chuyển thành hành động — 153 người bán cần theo dõi, ngưỡng 4 ngày trễ làm review sụt mạnh.
- **Nhu cầu phục vụ ở quy mô lớn:** nhiều người dùng, nhiều nơi, tải không đều → chọn dịch vụ serverless tự co giãn thay vì máy chủ cố định.

Velocity không phải đặc trưng chính: dữ liệu được làm mới theo lô mỗi khi Analytics chạy lại; loader idempotent cho phép tăng tần suất làm mới mà không đổi kiến trúc.

3.3. Input

Nguồn và cách thu thập: các bảng do nhóm Analytics bàn giao được sao chép nguyên vẹn vào `data/analytics/` của repo nhóm. **Định dạng:** Parquet nén Snappy, mỗi bảng một thư mục kiểu Spark (`part-00000-*.snappy.parquet`, kèm `_SUCCESS`). **Tốc độ phát sinh:** theo lô (lần chạy Analytics hiện tại: 29/10/2026). **Quy mô:** 16 bảng, 194.132 dòng, 137 cột, 8.033,0 KB nén; phân bố rất lệch — hai bảng cấp khách hàng (`rfm_scores`, `customer_segments`, mỗi bảng 94.983 dòng) và `seller_pareto` (3.053 dòng) chiếm hơn 99% số dòng, 13 bảng còn lại chỉ 2–715 dòng. Danh sách bảng, grain, khoá ở Mục 3.5; schema từng cột, quy mô chi tiết và ví dụ bản ghi ở Phụ lục C. Ví dụ rút gọn một dòng của `kpi_monthly`: `{purchase_year: 2017, purchase_month: 11, orders: 7421, revenue: 1003862.14, avg_order_value: 157.96, late_delivery_pct: 14.31, avg_review_score: 3.95, revenue_mom_pct: 52.06, revenue_rank: 1, is_black_friday_month: true}`.

Đầu vào bổ sung – tầng gold. Bảng Analytics có grain cố định nên không trả lời được câu hỏi lọc chéo như "AOV và % trễ của bang SP trong quý I/2018 với khách trả bằng boleto". Vì vậy nhóm nạp thêm 7 bảng gold (311.460 dòng, đầu vào của Analytics, đã đối chiếu 11/11). Hai bảng fact phân vùng Hive theo `purchase_year=YYYY/purchase_month=M/`; loader đọc bằng `pyarrow.dataset(..., partitioning="hive")` để khôi phục cột phân vùng. Gold chỉ dùng để **lọc và cộng lại**;

mọi KPI phân tích (Pareto, RFM, K-Means, watchlist) vẫn lấy nguyên từ Analytics, và bộ kiểm tra nhất quán (Mục 6.4) chứng minh tổng từ gold khớp tổng của `kpi_monthly`.

3.4. Output

Bảng 3.2. Các thành phần đầu ra của giai đoạn Presentation

Đầu ra	Định dạng / vị trí	Người dùng	Ví dụ
Bảng phục vụ	PostgreSQL 18 trên Neon: `analytics` (16 bảng + `load_log`), `gold` (7 bảng)	Superset, Decision/Action	`analytics.kpi_monthly` 20 dòng
Tầng ngữ nghĩa	22 view + bảng `dim_state` trong schema `presentation`	Superset, Decision/Action	`v_kpi_monthly` có `month_date` kiểu DATE
Dashboard	Apache Superset (HTTP cổng 8088), nhiều tab	Người dùng nghiệp vụ	Thẻ "Doanh thu", Pareto, bản đồ bang
Gợi ý quyết định	`v_decision_hints` (7 luật), `v_seller_watchlist` (ĐỎ/VÀNG)	Giai đoạn Decision/Action	153 người bán kèm lý do, hành động
Minh chứng	`evidence/`: log nạp, đối chiếu, độ trễ, ảnh	Giảng viên, nhóm	`load_log.csv`, `validation.json`

Ví dụ đầu ra: view `v_kpi_overview` (một dòng) cho các thẻ số neo — doanh thu gold toàn kỳ 13.494.400,74 BRL, doanh thu khung 20 tháng 13.449.529,68 BRL, tháng đỉnh `2017-11` (MoM 52,06%), review đơn trễ 2,56 so với đúng hạn 4,29, 2.887 khách mua lại (3,04%), 539/3.053 người bán tạo 80% doanh thu, 153 người bán trong watchlist. Một dòng của `v_decision_hints`: *ưu tiên 2, mức CAO, chủ đề Giao hàng, tín hiệu "Đơn trễ kéo review xuống", giá trị 1,73, ngưỡng "chênh lệch > 1 sao", hành động "Cảnh báo seller khi đơn trễ 1–3 ngày".*

3.5. Data contract giữa Analytics và Presentation

Data contract là thoả thuận ràng buộc giữa bên sản xuất (Analytics) và bên tiêu thụ (Presentation) về **cấu trúc, ngữ nghĩa và chất lượng** dữ liệu bàn giao. Nhóm hình thức hoá `ANALYTICS_HANDOVER.md` thành bảng dưới đây (sinh tự động từ Parquet) và bộ kiểm tra tự động.

Bảng 3.3. Data contract Analytics → Presentation: danh sách bảng, grain và khoá

Bảng (analytics.*)	Chủ đề	1 dòng =	Khoá / cột định danh	Số dòng	Số cột	Dùng trên dashboard
`kpi_monthly`	1. Xu hướng	1 tháng (2017-01 → 2018-08)	purchase_year, purchase_month	20	17	KPI tổng quan, xu hướng
`category_pareto`	2. Danh mục	1 danh mục	category	74	9	Pareto danh mục

Bảng (analytics.*)	Chủ đề	1 dòng =	Khoá / cột định danh	Số dòng	Số cột	Dùng trên dashboard
category_seasonality	2. Danh mục	1 danh mục × 1 tháng (2017)	category, purchase_month	715	5	Heatmap mùa vụ
state_logistics	3. Địa lý	1 bang của khách	customer_state	27	13	Bản đồ / bảng bang
insight_distance	3. Địa lý	1 khoảng cách (band km)	distance_band (band_order)	4	7	Logistics theo km
rfm_scores	4. Khách hàng	1 khách thật	customer_unique_id	94.983	12	Phân khúc RFM
customer_segments	4. Khách hàng	1 khách thật (+ cụm K-Means)	customer_unique_id	94.983	13	Phân khúc + cụm
cluster_profile	4. Khách hàng	1 cụm K-Means	cluster	4	8	Hồ sơ cụm
repeat_profile	4. Khách hàng	1 cohort (repeat / one_shot)	cohort	2	7	Thẻ KPI giữ chân
repeat_categories	4. Khách hàng	1 danh mục của khách mua lại	category	70	4	Danh mục khách lặp
repeat_states	4. Khách hàng	1 bang của khách mua lại	customer_state	27	5	Bang khách lặp
seller_pareto	5. Người bán	1 người bán	seller_id	3.053	11	Pareto người bán
seller_watchlist	5. Người bán	1 người bán cần theo dõi	seller_id	153	10	Gợi ý quyết định
insight_late_vs_review	6. Giao hàng & review	1 nhóm (late / on_time)	delivery	2	5	So sánh trễ vs đúng hạn
delay_vs_review	6. Giao hàng & review	1 khoảng ngày trễ	delay_band (band_order)	5	6	Ngưỡng trễ → review
payment_mix	6. Giao hàng & review	1 PTTT × 1 nhóm kỳ trả góp	main_payment_type, installment_band	10	5	Cơ cấu thanh toán
Tổng				**194.132**	137	

Các điều khoản:

1. **Schema:** tên bảng, cột, kiểu như Phụ lục C. Thêm cột được chấp nhận (loader tự sinh DDL); đổi tên hoặc xoá cột là thay đổi phá vỡ, phải báo trước.
2. **Grain và khoá:** khoá ở cột "Khoá" là duy nhất — loader khai báo thành khoá chính, nên vi phạm grain bị phát hiện khi nạp.
3. **Ngữ nghĩa độ đo:** `revenue` = tổng giá sản phẩm chưa gồm ship; `gmv` = `revenue` + ship; `avg_review_score` cấp đơn; `late_delivery_pct` tính trên đơn đã giao; `delivery_delay_days` âm là giao sớm; tiền BRL 2 chữ số thập phân.
4. **Phạm vi thời gian:** 2017-01 → 2018-08; `category_seasonality` chỉ năm 2017.
5. **Số neo đối chiếu:** doanh thu gold 13.494.400,74 BRL; `kpi_monthly` 20 tháng; 94.983 khách; đỉnh 11/2017 với MoM +52,06%; review trễ/đúng hạn 2,56/4,29; khách mua lại 2.887 (3,04%).
6. **Làm mới:** Analytics chạy lại thì toàn bộ thư mục được thay thế; Presentation nạp lại toàn bộ và ghi `run_id` vào `load_log`.
7. **Ánh xạ kiểu:** `decimal(p,s)` → `NUMERIC(p,s)`, `int32` → `INTEGER`, `int64` → `BIGINT`, `double` → `DOUBLE PRECISION`, `bool` → `BOOLEAN`, `date32` → `DATE`, `timestamp` → `TIMESTAMP`, `string` → `TEXT`.

3.6. Yêu cầu chức năng và phi chức năng

Bảng 3.4. Yêu cầu chức năng (F) và phi chức năng (N)

Mã	Yêu cầu	Cách đáp ứng / chỉ tiêu
F1	Nạp toàn bộ bảng vào tầng phục vụ, đúng kiểu dữ liệu	Loader pyarrow + COPY, DDL tự sinh
F2	Nhật ký mỗi lần nạp	Bảng `analytics.load_log` + `evidence/load_log.csv`
F3	KPI thống nhất, tổng hợp đúng	View `presentation.v_*`
F4	Dashboard: thẻ KPI, xu hướng, Pareto, bản đồ, bảng chi tiết, bộ lọc tháng/bang/danh mục/phân khúc	Superset, native filters
F5	Gợi ý quyết định cho giai đoạn sau	`v_decision_hints`, `v_seller_watchlist`
N1	Tính đúng đắn	100% số dòng và số neo khớp Parquet
N2	Độ trễ truy vấn khi compute đang chạy	Trung vị < 300 ms, lớn nhất < 500 ms
N3	Tái lập	Một lệnh Docker Compose + một lệnh nạp
N4	Bảo mật	TLS bắt buộc; bí mật chỉ trong `.env` (không commit)
N5	Chi phí và mở rộng	Gói miễn phí Neon, compute ngủ khi không dùng; không đổi kiến trúc khi dữ liệu ×10

3.7. Định nghĩa KPI

Bảng 3.5. Định nghĩa KPI và quy tắc tổng hợp trên dashboard

KPI	Định nghĩa	Nguồn	Tổng hợp khi lọc nhiều phần tử
Doanh thu (BRL)	Tổng `price` đơn hợp lệ, chưa gồm ship	`kpi_monthly`, `v_orders`	`SUM(revenue)`
Số đơn	Số đơn hợp lệ	`kpi_monthly`, `v_orders`	`SUM(orders)` / `COUNT(*)`
AOV (BRL)	GMV / số đơn	`v_orders`	`SUM(order_value) / COUNT(*)` — không AVG của AOV tháng
Tăng trưởng MoM (%)	(t − (t−1)) / (t−1) × 100	`kpi_monthly`	Không tổng hợp; hiển thị theo tháng
% đơn giao trễ	Đơn trễ / đơn đã giao × 100	`v_orders.late_flag` (1/0/NULL)	`AVG(late_flag) × 100` — NULL (chưa giao) tự bị loại
Điểm review TB	Trung bình sao 1–5 cấp đơn	`v_orders.review_score`	`AVG(review_score)` cấp đơn
Tỷ trọng / % cộng dồn Pareto	Phần tử / tổng; cộng dồn giảm dần	`category_pareto`, `seller_pareto`	Tính sẵn trên toàn kỳ (window của Spark)
Phí ship / doanh thu (%)	`freight / revenue × 100`	`category_pareto`, `state_logistics`	`SUM(freight) / SUM(revenue)`
% khách mua lại	Khách `frequency ≥ 2` / tổng khách	`repeat_profile`	Trên toàn bộ 94.983 khách
Chênh lệch review	review đúng hạn − review trễ	`insight_late_vs_review`	4,29 − 2,56 = 1,73 sao
Người bán cần theo dõi	Top 10% doanh thu **và** (review < 4 hoặc giao > 15 ngày hoặc trễ > 10%)	`seller_watchlist`	`COUNT(*)` = 153

Một lựa chọn thiết kế đáng chú ý: `kpi_monthly` chỉ có **tỷ lệ** % trễ và review theo tháng, không có mẫu số; nếu thẻ KPI lấy trung bình các tỷ lệ tháng khi người dùng chọn nhiều tháng thì sai (trung bình của trung bình). Nhóm giải quyết bằng view cấp đơn `v_orders` từ gold: `late_flag` nhận 1, 0 hoặc NULL (chưa giao), nên `AVG(late_flag)` đúng định nghĩa trên mọi tổ hợp bộ lọc. Biểu đồ xu hướng theo tháng vẫn lấy nguyên số của `kpi_monthly`, và bộ kiểm tra nhất quán đảm bảo hai nguồn khớp nhau.

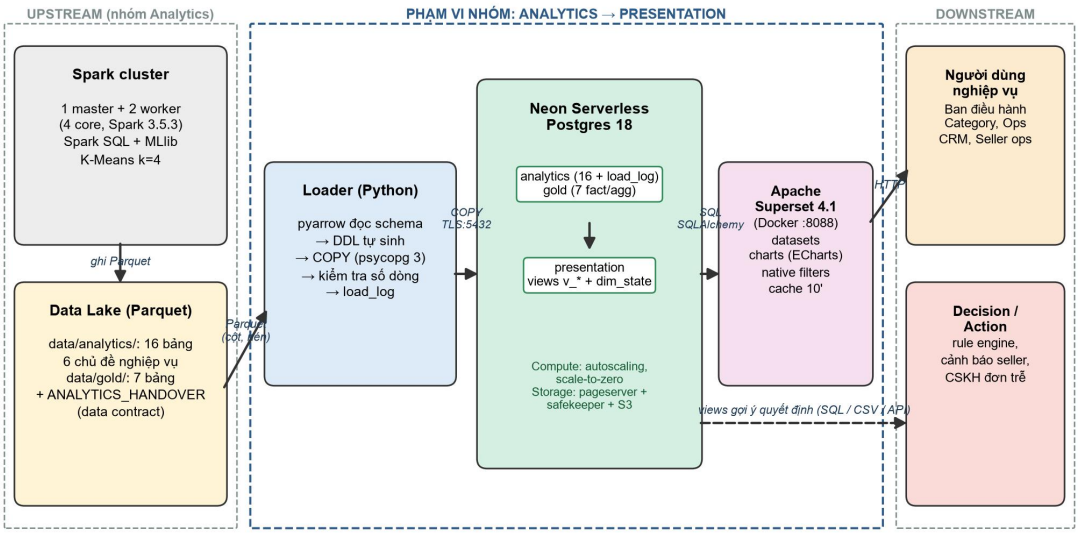
4. Thiết kế hệ thống

4.1. Kiến trúc tổng thể

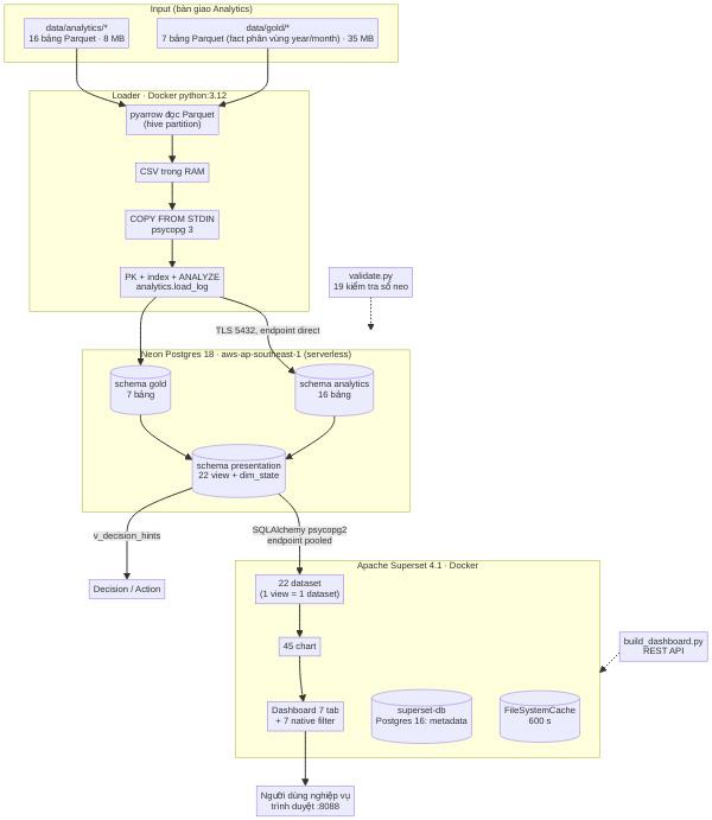
Hệ thống gồm bốn khối nối tiếp nhau, mỗi khối có một trách nhiệm duy nhất và giao tiếp qua một định dạng/giao thức chuẩn:

1. **Data Lake (đầu vào):** 16 bảng Parquet do Spark ghi ra (cùng 7 bảng gold dùng cho lọc chi tiết). Đây là ranh giới với các nhóm trước.
2. **Loader (Python, chạy trong container `tools`):** đọc Parquet bằng pyarrow, tự sinh DDL, chuyển dữ liệu thành CSV trong bộ nhớ và nạp bằng `COPY FROM STDIN` qua psycopg 3, tạo khoá chính/chỉ mục, kiểm tra số dòng, ghi nhật ký, rồi áp dụng tầng ngữ nghĩa.
3. **Tầng phục vụ (Neon Serverless Postgres 18):** schema `analytics` (bản sao trung thực của output Analytics + `load_log`), schema `gold` (fact/aggregate chi tiết) và schema `presentation` (view ngữ nghĩa `v_*` + bảng dimension `dim_state`).
4. **Tầng trình bày (Apache Superset 4.1 trong Docker):** kết nối tới Neon qua SQLAlchemy + psycopg2; mỗi view là một dataset; chart và dashboard được dựng bằng mã thông qua REST API.

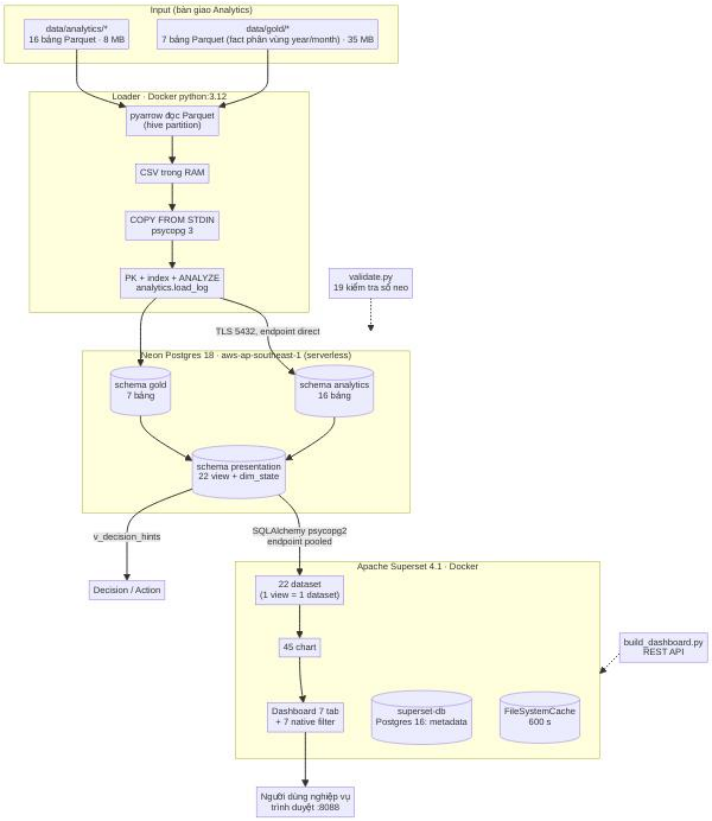
Ở đầu ra, người dùng nghiệp vụ truy cập dashboard qua trình duyệt; giai đoạn Decision/Action đọc trực tiếp các view gợi ý quyết định trên Neon. Hình 4.1 là sơ đồ data-flow do script vẽ từ thiết kế; nếu nhóm có sơ đồ vẽ riêng trong `docs/` hoặc `evidence/`, nó được chèn ngay sau.



Hình 4.1. *Sơ đồ kiến trúc và luồng dữ liệu: Parquet → Loader (COPY) → Neon (analytics/gold → presentation) → Superset → người dùng / Decision-Action*



Hình 4.2. *Sơ đồ kiến trúc / luồng dữ liệu của hệ thống – architecture pipeline (tệp architecture_pipeline.png)*



Hình 4.3. *Sơ đồ kiến trúc hệ thống Analytics → Presentation (vẽ bởi nhóm)*



Hình 4.4. *Sơ đồ kiến trúc / luồng dữ liệu của hệ thống – architecture pipeline position (tệp architecture_pipeline_position.png)*

4.2. Luồng dữ liệu chi tiết

Bảng 4.1. Các bước của luồng dữ liệu, định dạng và giao thức

Bước	Từ → Đến	Định dạng / giao thức	Nơi xử lý	Mô tả
1	Spark → Data Lake	Parquet + Snappy	Cụm Spark (nhóm trước)	16 thư mục bảng Analytics; gold phân vùng Hive năm/tháng
2	Data Lake → Loader	Arrow in-memory (cột) → CSV trong bộ nhớ	Container `tools`	`pyarrow.dataset` đọc cả bảng phân vùng; ép kiểu timestamp ns → µs
3	Loader → Neon	Giao thức Postgres v3, `COPY ... FROM STDIN (FORMAT csv)`, TLS (`sslmode=require`), cổng 5432, endpoint *direct*	Neon compute	DROP/CREATE/COPY/COMMENT trong một giao dịch; sau đó PK, index, `ANALYZE`, đếm dòng, ghi `load_log`
4	`analytics`, `gold` → `presentation`	SQL view (`sql/10_presentation_views.sql`)	Neon compute	Tạo lại toàn bộ schema `presentation` trong một giao dịch
5	Neon → Superset	SQLAlchemy + psycopg2, TLS	Superset (Docker)	Mỗi chart sinh một truy vấn SQL đẩy xuống Neon; kết quả cache 10 phút
6	Superset → người dùng	HTTP cổng 8088, JSON + React/ECharts	Trình duyệt	Dashboard nhiều tab, native filter, cross-filter

Bước	Từ → Đến	Định dạng / giao thức	Nơi xử lý	Mô tả
7	Neon → Decision/Action	SQL (`v_decision_hints`, `v_seller_watchlist`), CSV export, Chart Data API	Giai đoạn sau	Đọc luật, ngưỡng và danh sách người bán cần xử lý

Luồng được thiết kế theo nguyên tắc **"tính một lần, đọc nhiều lần"**: các phép tính nặng (join fact, window toàn cục, K-Means) đã được Spark làm ở giai đoạn trước; tầng phục vụ chỉ thực hiện lọc và tổng hợp nhẹ trên bảng đã cô đọng hoặc trên bảng fact đã có chỉ mục, nên độ trễ chủ yếu do mạng và khởi động compute chứ không do tính toán.

4.3. Thiết kế tầng lưu trữ trên Neon

4.3.1. Tổ chức project, database và schema

Bảng 4.2. Cấu hình tầng phục vụ Neon

Thuộc tính	Giá trị	Lý do
Project	`bigdata-olist-presentation`	Một project riêng cho giai đoạn Presentation
Vùng	`aws-ap-southeast-1` (Singapore)	Vùng gần Việt Nam nhất → giảm độ trễ mạng
Phiên bản	PostgreSQL 18 (log nạp ghi nhận 18.6, aarch64)	Phiên bản mới nhất Neon hỗ trợ
Database	`olist_analytics`	Tách khỏi database mặc định
Schema `analytics`	16 bảng output Analytics + `load_log`	"Hợp đồng vật lý" với nhóm Analytics
Schema `gold`	7 bảng fact/aggregate	Dữ liệu chi tiết cho lọc chéo và kiểm tra nhất quán
Schema `presentation`	22 view + `dim_state`	"Hợp đồng logic" với người dùng và nhóm sau
Kết nối loader	Endpoint *direct* (bỏ `-pooler`)	`COPY`/DDL cần phiên ổn định, không qua PgBouncer
Kết nối Superset	Endpoint *pooled* (`-pooler`)	Nhiều truy vấn ngắn, nhiều kết nối đồng thời
Autoscaling	**[[CẬP NHẬT: khoảng CU thực tế, ví dụ 0,25–2 CU]]**	Co giãn theo tải dashboard
Scale-to-zero	Sau 5 phút không hoạt động (mặc định)	Không tốn compute khi không ai xem

Việc tách ba schema thể hiện nguyên tắc **phân tầng**: `analytics` và `gold` phản ánh cấu trúc do các nhóm trước quyết định (thay đổi khi họ thay đổi), còn `presentation` là giao diện ổn định cho người dùng (tên cột, nhãn tiếng Việt, định nghĩa KPI). Khi nhóm trước đổi tên một cột, nhóm chỉ cần sửa view; dashboard và giai đoạn Decision/Action không bị ảnh hưởng.

4.3.2. Sinh DDL tự động và ánh xạ kiểu

Loader không chứa `CREATE TABLE` viết tay: DDL được sinh từ schema Arrow trong metadata Parquet theo ánh xạ kiểu của data contract (Mục 3.5); kiểu chưa ánh xạ gây lỗi ngay thay vì âm thầm thành `text`. Các quyết định quan trọng: **tiền tệ giữ `NUMERIC(p,s)`** để tổng 13.494.400,74 BRL khớp đến từng xu (số thực dấu phẩy động gây sai số làm tròn); **timestamp nano giây ép về micro giây** vì PostgreSQL chỉ lưu tới micro giây; **phân biệt chuỗi rỗng và NULL** bằng cách đặt mọi giá trị hợp lệ trong dấu nháy kép khi ghi CSV. Mỗi bảng được gắn `COMMENT ON TABLE` ghi grain và nguồn để tra cứu ngay trong cơ sở dữ liệu.

4.3.3. Nạp idempotent, nhật ký và chỉ mục

Mỗi lần chạy sinh một `run_id`. Với từng bảng, `DROP … CASCADE`, `CREATE`, `COPY` (khối 1 MiB) và `COMMENT` nằm trong **một giao dịch**, nên người dùng dashboard hoặc thấy bản cũ, hoặc bản mới hoàn chỉnh. Sau commit, loader tạo khoá chính theo grain của data contract (vi phạm thì lùi về chỉ mục thường và ghi cảnh báo), tạo chỉ mục phụ, `ANALYZE`, rồi so `count(*)` với số dòng Parquet; lệch thì chương trình kết thúc với mã lỗi. Chạy lại bao nhiêu lần cũng cho cùng trạng thái (idempotent).

Chỉ mục phụ chỉ tạo trên cột bộ lọc dùng thường xuyên của các bảng lớn: `rfm_segment`, `customer_state`, `cluster` (bảng khách hàng), `seller_state` (người bán), `purchase_date`, `customer_state`, `category`, `seller_id`, `main_payment_type` (fact gold); 18 bảng còn lại (≤ 3.053 dòng) quét tuần tự chỉ vài mili giây nên không cần. Bảng `analytics.load_log` lưu mỗi bảng mỗi lần nạp một dòng (`run_id`, số dòng Postgres/Parquet, cờ khớp, dung lượng Parquet/CSV, thời gian đọc/COPY/chỉ mục/tổng, thông lượng, thời điểm), đồng thời xuất `evidence/load_log.csv`; view `v_load_log` hiển thị lần nạp gần nhất trên dashboard (độ tươi và nguồn gốc dữ liệu).

4.4. Thiết kế tầng ngữ nghĩa

Tầng ngữ nghĩa nằm trong tệp `sql/10_presentation_views.sql`, bắt đầu bằng `DROP SCHEMA IF EXISTS presentation CASCADE` rồi tạo lại toàn bộ — nhờ vậy luôn nhất quán với dữ liệu vừa nạp và chạy lại an toàn. Các view tuân theo bốn quy tắc:

- **Không tính lại** những gì Spark đã tính (Pareto cộng dồn, NTILE, cụm K-Means) — chỉ đổi tên, gắn nhãn và bổ sung cột dẫn xuất nhẹ (hạng, nhóm A/B/C).

- **Bổ sung cột phục vụ trình bày:** `month_date` kiểu `DATE` và `month_label` dựng từ năm/tháng để bộ lọc khoảng thời gian của Superset hoạt động; bảng `dim_state` cung cấp tên bang, vùng (Norte, Nordeste, Centro-Oeste, Sudeste, Sul) và `iso_code = 'BR-' || customer_state` cho biểu đồ Country Map Brazil; nhãn tiếng Việt cho band ngày trễ, kỳ trả góp, cohort; tên nghiệp vụ cho cụm K-Means (C0 "One-shot giá trị thấp", C1 "Khách mua lại", C2 "One-shot chi cao", C3 "One-shot lâu không mua").
- **Giữ quy tắc bàn giao:** khung 2017-01 → 2018-08, đơn hợp lệ, đếm khách bằng `customer_unique_id`, review chỉ lấy ở cấp đơn (`v_sales_items` cấp sản phẩm cố ý không có cột review).
- **Mã hoá gợi ý quyết định** thành dữ liệu: số vi phạm, lý do, mức cảnh báo và hành động khuyến nghị.

Các view được nhóm theo sáu chủ đề của Analytics cộng hai nhóm tiện ích: **xu hướng** (`v_kpi_monthly`, `v_kpi_overview`, `v_orders`), **danh mục** (`v_category_pareto`, `v_category_seasonality`, `v_category_monthly`, `v_sales_items`), **địa lý** (`v_state_logistics`, `v_insight_distance`, bảng `dim_state`), **khách hàng** (`v_rfm_customers`, `v_rfm_segments`, `v_cluster_profile`, `v_repeat_*`), **người bán** (`v_seller_pareto`, `v_seller_watchlist`), **giao hàng – review – thanh toán** (`v_late_vs_review`, `v_delay_vs_review`, `v_payment_mix`), **quyết định** (`v_decision_hints`) và **nguồn gốc dữ liệu** (`v_load_log`). Danh mục đầy đủ (nguồn, mục đích) và mã SQL ở Phụ lục D.

Ví dụ: view `v_seller_watchlist` biến danh sách người bán của Analytics thành dữ liệu hành động được — mỗi tiêu chí (review < 4, giao > 15 ngày, trễ > 10%) thành một cờ 0/1, cộng lại thành `violations`, ghép thành chuỗi `reasons`, và `alert_level` là ĐỎ khi vi phạm từ hai tiêu chí trở lên, ngược lại là VÀNG, kèm `recommended_action` tương ứng.

4.5. Thiết kế dashboard theo người dùng nghiệp vụ

Dashboard "Olist E-commerce · Analytics → Presentation" (dựng bởi `superset/build_dashboard.py`) gồm bảy tab bám theo sáu chủ đề của Analytics cộng một tab quyết định, mỗi tab phục vụ một nhóm người dùng. Bố cục mỗi tab theo mẫu "tổng quan → chi tiết": hàng thẻ KPI, biểu đồ chính, rồi bảng chi tiết.

Bảng 4.3. Thiết kế dashboard theo tab và nhóm người dùng

Tab	Người dùng chính	Câu hỏi nghiệp vụ	Thành phần chính (view)
Tổng quan & xu hướng	Ban điều hành	Doanh thu tăng thế nào? Vận hành có theo kịp?	Thẻ KPI (`v_orders`, `v_kpi_overview`); doanh thu, số đơn, % trễ, review theo tháng (`v_kpi_monthly`)
Danh mục & mùa vụ	Quản lý ngành hàng	Ngành nào tạo 80% doanh thu? Mùa vụ? Ship ăn biên ngành nào?	Pareto (`v_category_pareto`); heatmap (`v_category_seasonality`); xu hướng ngành (`v_category_monthly`, `v_sales_items`)
Địa lý & logistics	Vận hành – logistics	Bang nào giao chậm, trễ nhiều, review thấp? Khoảng cách ảnh hưởng ra sao?	Country Map Brazil, bảng bang (`v_state_logistics`); band km (`v_insight_distance`)
Khách hàng & RFM	CRM – marketing	Bao nhiêu khách quay lại? Nên chăm sóc phân khúc nào?	Repeat vs one-shot (`v_repeat_profile`); phân khúc + hành động CRM (`v_rfm_segments`); cụm K-Means; danh mục, bang của khách lặp
Seller	Quản lý người bán	Ai tạo 80% doanh thu? Ai lớn nhưng vận hành kém?	Pareto, bubble người bán (`v_seller_pareto`); watchlist ĐỎ/VÀNG (`v_seller_watchlist`)
Giao trễ, review & thanh toán	Vận hành, CSKH, tài chính	Trễ bao nhiêu ngày thì review sụt? Cơ cấu thanh toán?	`v_late_vs_review`, `v_delay_vs_review`, `v_payment_mix`
Gợi ý quyết định & nguồn dữ liệu	Ban điều hành, giai đoạn Decision/Action	Cần làm gì, ưu tiên nào? Dữ liệu nạp lúc nào?	Bảng luật (`v_decision_hints`); nhật ký nạp (`v_load_log`)

Bảy native filter: **Khoảng thời gian** (cột `month_date`, áp dụng cho `v_kpi_monthly`, `v_orders`, `v_category_monthly`, `v_sales_items`), **Bang khách hàng**, **Vùng**, **Danh mục**, **Phân khúc RFM**, **Phương thức thanh toán** và **Bang seller**; mỗi bộ lọc chỉ gắn vào các chart có cột tương ứng (phạm vi được script tính tự động).

Các quyết định thiết kế đáng chú ý:

- **Màu có chủ đích:** tháng Black Friday (11/2017), người bán mức ĐỎ và các ô vượt ngưỡng (review < 4, trễ > 10%) dùng màu cam/đỏ; còn lại dùng màu trung tính, để mắt người dùng tập trung vào ngoại lệ.
- **Tránh biểu đồ tròn** cho 74 danh mục hoặc 27 bang (không so sánh được góc nhỏ); dùng cột sắp xếp giảm dần và Pareto.
- **Thang log** cho số khách theo phân khúc RFM vì chênh lệch 65.325 ("One-shot Lost") so với 251 ("Champions") làm các nhóm nhỏ biến mất trên thang tuyến tính.
- **Ghi chú nguồn** dưới mỗi chart (tên view, giai đoạn 2017-01 → 2018-08, đơn vị BRL) và bảng `v_load_log` ở cuối dashboard cho biết dữ liệu được nạp lúc nào.

Hai lớp dữ liệu cho bộ lọc. Bảng Analytics là bảng tổng hợp ở grain cố định: `category_pareto` không có cột tháng, `kpi_monthly` không có cột bang, nên không thể lọc chéo trực tiếp. Nhóm giải quyết bằng hai lớp: (1) **lớp cấp đơn/sản phẩm** (`v_orders`, `v_sales_items`, `v_category_monthly` từ gold) cho các thẻ KPI và xu hướng cần lọc tự do theo tháng × bang × danh mục × phương thức thanh toán; (2) **lớp insight** (các view trên `analytics.*`) cho những kết quả phân tích cố định trên toàn kỳ như Pareto, RFM, watchlist. Phạm vi (scope) của từng native filter được cấu hình chỉ gồm những chart có cột tương ứng, và tiêu đề chart ghi rõ "toàn kỳ" với các insight không lọc được.

4.6. Giao diện với giai đoạn Decision/Action

Giai đoạn Presentation → Decision/Action cần dữ liệu có cấu trúc, ổn định, không phụ thuộc giao diện. Nhóm cung cấp view `presentation.v_decision_hints`, mỗi dòng là **một luật** gồm: mức ưu tiên, mức độ (CAO/TRUNG BÌNH/THẤP), chủ đề, tín hiệu, giá trị đo hiện tại, chi tiết, ngưỡng, hành động khuyến nghị và view nguồn. Bảy luật hiện tại:

Bảng 4.4. Các luật gợi ý quyết định trong v_decision_hints

Ưu tiên	Mức	Chủ đề	Tín hiệu / ngưỡng	Hành động khuyến nghị
1	CAO	Người bán	Người bán top 10% doanh thu vi phạm ≥ 2 tiêu chí (review < 4, giao > 15 ngày, trễ > 10%)	Cảnh cáo, giảm hiển thị, yêu cầu kế hoạch SLA 30 ngày
2	CAO	Giao hàng	Chênh lệch review đúng hạn – trễ > 1 sao	Cảnh báo người bán khi đơn trễ 1–3 ngày, trước mốc 4–7 ngày
3	CAO	Vận hành theo tháng	Tháng có tỷ lệ trễ > 15%	Tăng năng lực vận chuyển trước cao điểm (T11, T2–T3)
4	TRUNG BÌNH	Địa lý	Bang trễ > 10% và review < 4	Người bán/kho vùng; hiển thị ngày giao thực tế trước checkout
5	TRUNG BÌNH	Danh mục	Ngành doanh thu > 100k BRL có phí ship > 20% doanh thu	Không free-ship đại trà; đưa phí ship vào giá
6	TRUNG BÌNH	Khách hàng	Khách At Risk (≥ 2 đơn, > 180 ngày chưa quay lại)	Win-back có mục tiêu (email/voucher ngành nhà cửa)
7	THẤP	Người bán	% người bán tạo 80% doanh thu	Ưu tiên account manager cho nhóm A

Vì giá trị và ngưỡng được tính trực tiếp từ dữ liệu mỗi lần truy vấn, khi Analytics chạy lại và nhóm nạp lại, các luật tự cập nhật mà không cần sửa mã. Kênh truy cập cho nhóm sau: chuỗi kết nối Postgres (khuyến nghị role chỉ đọc trên schema `presentation`), xuất CSV từ Superset, hoặc Chart Data API của Superset (`/api/v1/chart/{id}/data`).

4.7. Lựa chọn công nghệ và lý do

Bảng 4.5. Công nghệ sử dụng, vai trò và lý do lựa chọn

Thành phần	Công nghệ	Vai trò trong kiến trúc	Lý do chọn	Phương án đã cân nhắc
Đầu vào	Apache Parquet (Snappy)	Định dạng bàn giao từ Analytics	Có schema, kiểu DECIMAL, nén tốt, đọc nhanh theo cột	CSV (mất kiểu, lớn hơn)
Đọc dữ liệu	pyarrow (`dataset`, `csv`)	Đọc Parquet (kể cả phân vùng Hive), chuyển CSV	Thư viện chuẩn của Arrow, không cần Spark cho bảng đã cô đọng	Spark (quá nặng), pandas thuần
Nạp dữ liệu	psycopg 3 + `COPY`	Nạp hàng loạt vào Postgres	`COPY` nhanh hơn nhiều lần so với INSERT; hỗ trợ stream theo khối	INSERT batch, `to_sql`
Tầng phục vụ	Neon Serverless Postgres 18	Lưu bảng phục vụ + view ngữ nghĩa	Được quản lý, chia sẻ qua Internet, autoscaling, scale-to-zero, branching, chuẩn Postgres	Postgres cục bộ, ClickHouse, DuckDB
Tầng ngữ nghĩa	View SQL (schema `presentation`)	Định nghĩa KPI và luật dùng chung	Dùng chung cho Superset và Decision/Action; quản lý được bằng Git	Chỉ metric trong Superset, dbt
Trình bày	Apache Superset 4.1.2	Dashboard, bộ lọc, chart	Mã nguồn mở, SQL-native, native filter, Country Map Brazil, REST API dựng dashboard bằng mã	Power BI, Tableau, Grafana, Metabase
Đóng gói	Docker Compose	Chạy Superset, container công cụ, chụp ảnh	Tái lập trên mọi máy; máy Windows không cần cài Python	Cài đặt thủ công
Minh chứng	Playwright (headless Chromium)	Chụp ảnh dashboard tự động	Ảnh nhất quán, chạy lại được	Chụp tay

Vì sao Superset? So với Power BI/Tableau (thương mại, tệp dashboard nhị phân khó quản lý phiên bản, chia sẻ cần tài khoản trả phí), Grafana (mạnh về chuỗi thời gian giám sát nhưng yếu với Pareto, xếp hạng, phân tích theo danh mục) và Metabase (dễ dùng nhưng hạn chế loại biểu đồ, bộ lọc), Superset là dự án Apache chạy bằng một lệnh Docker, kết nối thẳng Postgres, có Country Map cho Brazil, native filter và **REST API để dựng dashboard bằng mã** — yếu tố then chốt cho yêu cầu tái lập.

Vì sao Neon? Postgres cục bộ không chia sẻ được cho nhóm, giảng viên và giai đoạn sau; ClickHouse chỉ phát huy ở hàng tỷ dòng chi tiết, còn với khoảng nửa triệu dòng thì chỉ tăng chi phí vận hành; DuckDB đọc Parquet không có máy chủ cho nhiều client đồng thời. Neon cho Postgres chuẩn, được quản lý, có TLS, mô hình serverless khớp với tải không đều của dashboard, và branching cho phép thử thay đổi view trên dữ liệu thật mà không ảnh hưởng dashboard đang dùng.

4.8. Cấu hình chính

Bí mật (chuỗi kết nối Neon, khoá Superset, mật khẩu quản trị) chỉ nằm trong `.env` không commit; repo chứa `.env.example`, hàm `mask()` che mật khẩu khi ghi log. Cấu hình Superset (`superset/superset_config.py`): metadata lưu SQLite trong

volume `superset_home`, tách khỏi dữ liệu nghiệp vụ trên Neon; bật `DASHBOARD_NATIVE_FILTERS`, `DASHBOARD_CROSS_FILTERS`, `ENABLE_TEMPLATE_PROCESSING`; cache kết quả kiểu filesystem 600 giây — lần mở thứ hai trong 10 phút không truy vấn Neon và không đánh thức compute đang ngủ; `ROW_LIMIT = 50000` bảo vệ trình duyệt khi mở view cấp khách hàng/cấp đơn. Toàn văn cấu hình ở Phụ lục B.

5. Cài đặt và triển khai

5.1. Môi trường

Bảng 5.1. Môi trường cài đặt và triển khai

Thành phần	Phiên bản / cấu hình	Ghi chú
Máy phát triển	Windows 11, PowerShell	Không cần cài Python trên máy host
Docker Desktop	Docker Engine + Compose v2	Chạy Superset, container công cụ, Playwright
Container công cụ `tools`	`python:3.12-slim` + pyarrow 17, pandas 2.2, psycopg 3.2, SQLAlchemy 2.0, requests	Loader, kiểm tra, dựng dashboard, đo độ trễ
Superset	`apache/superset:4.1.2` + `psycopg2-binary` 2.9.10	Metadata SQLite trong volume `superset_home`
Chụp ảnh	`mcr.microsoft.com/playwright/python:v1.48.0-jammy`	Chromium headless
Tầng phục vụ	Neon, PostgreSQL 18, vùng `aws-ap-southeast-1`	Gói miễn phí
Đầu vào	16 bảng Analytics (194.132 dòng) + 7 bảng gold (311.460 dòng)	`data/analytics/`, `data/gold/`

Lý do dùng container công cụ thay vì cài Python trên máy: mọi thành viên (và giảng viên) có cùng phiên bản thư viện, không xung đột với Python khác trên máy, và lệnh chạy giống nhau trên Windows, macOS, Linux.

5.2. Cấu trúc mã nguồn

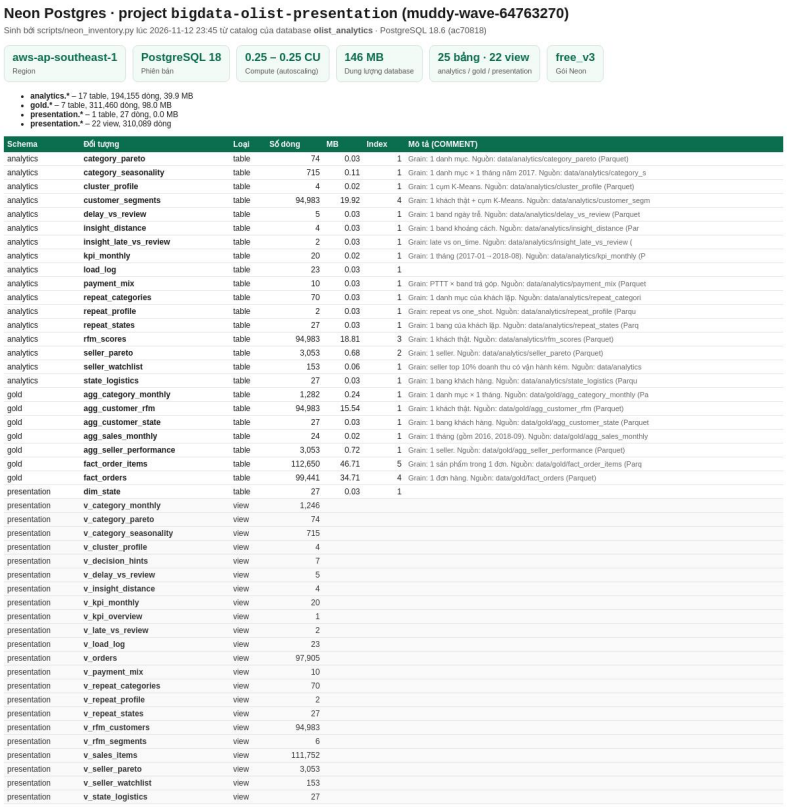
Repo được tổ chức theo thành phần của kiến trúc: `data/` (Parquet đầu vào: `analytics/`, `gold/`), `loader/` (nạp, áp dụng SQL, kiểm chứng; Dockerfile của container `tools`), `sql/` (tầng ngữ nghĩa và truy vấn kiểm tra), `superset/` (Dockerfile, cấu hình, script khởi tạo và dựng dashboard), `scripts/` (tiện ích: đo độ trễ, chụp ảnh), `evidence/` (log, số đo, ảnh), `docs/` (tài liệu, tài liệu bàn giao của nhóm trước) và `report/` (script sinh báo cáo này). Cây thư mục đầy đủ, quét tự động từ repo, ở Phụ lục F.

5.3. Khởi tạo Neon

Các bước khởi tạo tầng phục vụ:

1. Tạo project `bigdata-olist-presentation` trên Neon Console, chọn PostgreSQL 18 và vùng AWS Singapore (`aws-ap-southeast-1`).
2. Tạo database `olist_analytics` trên nhánh mặc định; ba schema `analytics`, `gold`, `presentation` do loader tự tạo (`CREATE SCHEMA IF NOT EXISTS`).

3. Lấy chuỗi kết nối dạng *pooled* (hostname chứa `-pooler`) và ghi vào biến `NEON_DATABASE_URL` trong `.env`. Module `loader/common.py` tự bỏ hậu tố `-pooler` để có endpoint *direct* cho `COPY`/DDL, còn Superset dùng endpoint pooled.
4. Cấu hình autoscaling và scale-to-zero cho compute (Mục 4.3.1).



Neon Postgres · project `bigdata-olist-presentation` (muddy-wave-64763270)

Sinh bởi scripts/neon_inventory.py lúc 2026-11-12 23:45 từ catalog của database **olist_analytics** · PostgreSQL 18.6 (ac70818)

aws-ap-southeast-1	PostgreSQL 18	0.25 – 0.25 CU	146 MB	25 bảng · 22 view	free_v3
Region	Phiên bản	Compute (autoscaling)	Dung lượng database	analytics / gold / presentation	Gói Neon

- **analytics.*** – 17 table, 194,155 dòng, 39.9 MB
- **gold.*** – 7 table, 311,460 dòng, 98.0 MB
- **presentation.*** – 1 table, 27 dòng, 0.0 MB
- **presentation.*** – 22 view, 310,069 dòng

Schema	Đối tượng	Loại	Số dòng	MB	Index	Mô tả (COMMENT)
analytics	**category_pareto**	table	74	0.03	1	Grain: 1 danh mục. Nguồn: data/analytics/category_pareto (Parquet)
analytics	**category_seasonality**	table	715	0.11	1	Grain: 1 danh mục × 1 tháng năm 2017. Nguồn: data/analytics/category_s
analytics	**cluster_profile**	table	4	0.02	1	Grain: 1 cụm K-Means. Nguồn: data/analytics/cluster_profile (Parquet)
analytics	**customer_segments**	table	94,983	19.92	4	Grain: 1 khách thật + cụm K-Means. Nguồn: data/analytics/customer_segm
analytics	**delay_vs_review**	table	5	0.03	1	Grain: 1 band ngày trễ. Nguồn: data/analytics/delay_vs_review (Parquet
analytics	**insight_distance**	table	4	0.03	1	Grain: 1 band khoảng cách. Nguồn: data/analytics/insight_distance (Par
analytics	**insight_late_vs_review**	table	2	0.03	1	Grain: late vs on_time. Nguồn: data/analytics/insight_late_vs_review (
analytics	**kpi_monthly**	table	20	0.02	1	Grain: 1 tháng (2017-01 – 2018-08). Nguồn: data/analytics/kpi_monthly (P
analytics	**load_log**	table	23	0.03	1	
analytics	**payment_mix**	table	10	0.03	1	Grain: PTTT × band trả góp. Nguồn: data/analytics/payment_mix (Parquet
analytics	**repeat_categories**	table	70	0.03	1	Grain: 1 danh mục của khách lặp. Nguồn: data/analytics/repeat_categori
analytics	**repeat_profile**	table	2	0.03	1	Grain: repeat vs one_shot. Nguồn: data/analytics/repeat_profile (Parqu
analytics	**repeat_states**	table	27	0.03	1	Grain: 1 bang của khách lặp. Nguồn: data/analytics/repeat_states (Parq
analytics	**rfm_scores**	table	94,983	18.81	3	Grain: 1 khách thật. Nguồn: data/analytics/rfm_scores (Parquet)
analytics	**seller_pareto**	table	3,053	0.68	2	Grain: 1 seller. Nguồn: data/analytics/seller_pareto (Parquet)
analytics	**seller_watchlist**	table	153	0.06	1	Grain: seller top 10% doanh thu có vận hành kém. Nguồn: data/analytics
analytics	**state_logistics**	table	27	0.03	1	Grain: 1 bang khách hàng. Nguồn: data/analytics/state_logistics (Parqu
gold	**agg_category_monthly**	table	1,282	0.24	1	Grain: 1 danh mục × 1 tháng. Nguồn: data/gold/agg_category_monthly (Pa
gold	**agg_customer_rfm**	table	94,983	15.54	1	Grain: 1 khách thật. Nguồn: data/gold/agg_customer_rfm (Parquet)
gold	**agg_customer_state**	table	27	0.03	1	Grain: 1 bang khách hàng. Nguồn: data/gold/agg_customer_state (Parquet
gold	**agg_sales_monthly**	table	24	0.02	1	Grain: 1 tháng (gồm 2016, 2018-09). Nguồn: data/gold/agg_sales_monthly
gold	**agg_seller_performance**	table	3,053	0.72	1	Grain: 1 seller. Nguồn: data/gold/agg_seller_performance (Parquet)
gold	**fact_order_items**	table	112,650	46.71	5	Grain: 1 sản phẩm trong 1 đơn. Nguồn: data/gold/fact_order_items (Parq
gold	**fact_orders**	table	99,441	34.71	4	Grain: 1 đơn hàng. Nguồn: data/gold/fact_orders (Parquet)
presentation	**dim_state**	table	27	0.03	1	
presentation	**v_category_monthly**	view	1,246			
presentation	**v_category_pareto**	view	74			
presentation	**v_category_seasonality**	view	715			
presentation	**v_cluster_profile**	view	4			
presentation	**v_decision_hints**	view	7			
presentation	**v_delay_vs_review**	view	5			
presentation	**v_insight_distance**	view	4			
presentation	**v_kpi_monthly**	view	20			
presentation	**v_kpi_overview**	view	1			
presentation	**v_late_vs_review**	view	2			
presentation	**v_load_log**	view	23			
presentation	**v_orders**	view	97,905			
presentation	**v_payment_mix**	view	10			
presentation	**v_repeat_categories**	view	70			
presentation	**v_repeat_profile**	view	2			
presentation	**v_repeat_states**	view	27			
presentation	**v_rfm_customers**	view	94,983			
presentation	**v_rfm_segments**	view	6			
presentation	**v_sales_items**	view	111,752			
presentation	**v_seller_pareto**	view	3,053			
presentation	**v_seller_watchlist**	view	153			
presentation	**v_state_logistics**	view	27			

Hình 5.1. *Neon Console: project, nhánh và compute của tầng phục vụ – neon tables inventory (tệp neon_tables_inventory.png)*

5.4. Loader Parquet → Neon

Loader (`loader/load_to_neon.py`) được viết bằng Python, chạy trong container `tools`. Danh sách bảng cần nạp được khai báo tường minh trong từ điển `TABLES` cùng khoá chính, chỉ mục phụ và grain của từng bảng. Luồng xử lý của một lần chạy:

1. Sinh `run_id`, mở kết nối direct tới Neon (timeout 30 giây để chịu được cold start khi compute đang ngủ), ghi phiên bản Postgres và chuỗi kết nối đã che mật khẩu vào `evidence/load_run.log`.
2. Tạo các schema `analytics`, `gold`, `presentation` và bảng `analytics.load_log` nếu chưa có.
3. Với mỗi bảng: `pyarrow.dataset` đọc thư mục Parquet (tự nhận phân vùng Hive), ép kiểu timestamp, sinh DDL, chuyển bảng Arrow thành CSV trong bộ nhớ.

4. Trong một giao dịch: `DROP TABLE … CASCADE`, `CREATE TABLE`, `COPY … FROM STDIN (FORMAT csv)` truyền từng khối 1 MiB, `COMMENT ON TABLE` ghi grain và nguồn.
5. Tạo khoá chính và chỉ mục phụ, `ANALYZE`, đếm dòng, ghi một dòng vào `analytics.load_log` và một dòng log (lớp `Tee` ghi đồng thời ra màn hình và tệp).
6. Áp dụng các tệp `sql/[0-8]*.sql` để tạo lại tầng ngữ nghĩa (vì `DROP … CASCADE` ở bước 4 có thể xoá view phụ thuộc).
7. In dòng tổng (số bảng, số dòng, dung lượng, thời gian, thông lượng, số bảng khớp), xuất `evidence/load_log.csv`; thoát với mã lỗi nếu có bảng lệch số dòng.

Đoạn mã trích từ hàm `load_one` của loader, phần nạp trong giao dịch:

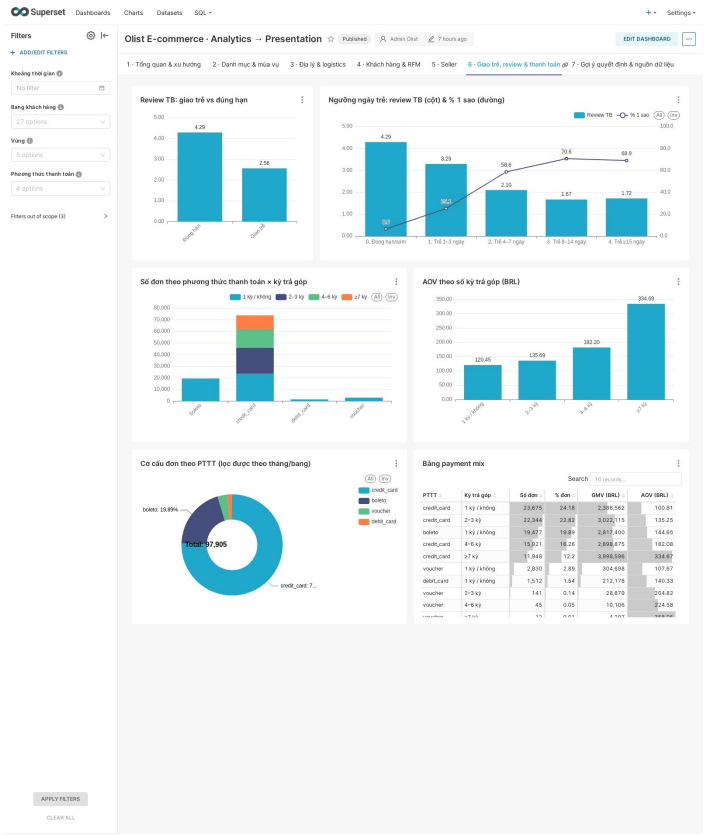
```
table, nfiles, nbytes = read_table(src)          # pyarrow.dataset,
partitioning="hive"
cols_ddl = ",\n    ".join(f"{f.name} {pg_type(f.type)}" for f in table.schema)
payload = to_csv_bytes(table)                     # pyarrow.csv,
quoting_style="all_valid"
with conn.transaction():
    with conn.cursor() as cur:
        cur.execute(f"DROP TABLE IF EXISTS {fq} CASCADE")
        cur.execute(f"CREATE TABLE {fq} (\n    {cols_ddl}\n)")
        with cur.copy(f"COPY {fq} ({collist}) FROM STDIN WITH (FORMAT csv)") as cp:
            view = memoryview(payload)
            for i in range(0, len(view), 1 << 20):   # khối 1 MiB
                cp.write(view[i : i + (1 << 20)])
        cur.execute(sql.SQL("COMMENT ON TABLE {} IS {}").format(sql.SQL(fq), comment))
```

Chuyển sang CSV bằng `pyarrow.csv` (mã C++) thay vì ghi từng dòng Python giúp phần chuẩn bị dữ liệu chỉ tốn vài phần trăm giây cho mỗi bảng (giá trị `read=` trong log nạp), để thời gian còn lại dành cho truyền dữ liệu. Toàn văn loader ở Phụ lục B, log đầy đủ của lần chạy ở Phụ lục E.

5.5. Tạo tầng ngữ nghĩa

Các tệp SQL trong `sql/` được đặt tên theo thứ tự chạy: `10_presentation_views.sql` tạo tầng ngữ nghĩa, `90_anchor_checks.sql` chứa các truy vấn kiểm tra số neo (mỗi truy vấn kèm chú thích `-- check: tên | expected: giá trị | tol: sai số`). Loader tự áp dụng các tệp `[0-8]*.sql` sau khi nạp; khi chỉ sửa view mà không cần nạp lại dữ liệu, chạy `loader/apply_sql.py`. Tệp view bắt đầu bằng `DROP SCHEMA IF EXISTS presentation CASCADE` và chạy trong một giao dịch, nên hoặc toàn bộ tầng ngữ nghĩa được tạo mới, hoặc giữ nguyên bản cũ. Log áp dụng (Phụ lục E) cho thấy 22 view và bảng `dim_state` được tạo trong chưa đến một giây. Sau đó `loader/validate.py` đọc lại Parquet để so số dòng với Postgres và chạy các truy vấn

trong `90_anchor_checks.sql`, xuất `evidence/validation.md` và `evidence/validation.json` (kết quả ở Mục 6.4).



Hình 5.2. *Truy vấn semantic view (SQL Lab / psql) – dashboard tab6 giao tre review thanh toan (tệp dashboard_tab6_giao_tre_review_thanh_toan.png)*

5.6. Triển khai Apache Superset

Superset được đóng gói bằng `superset/Dockerfile` từ image `apache/superset:4.1.2`, cài thêm `psycopg2-binary` (image gốc không kèm driver Postgres). Script `docker-init.sh` chạy `superset db upgrade`, `superset fab create-admin` (từ biến môi trường, bỏ qua nếu đã có), `superset init` rồi khởi động web server — mọi bước idempotent nên container khởi động lại bất kỳ lúc nào. Dashboard được dựng bằng script gọi **REST API** (dashboard-as-code): lấy access/CSRF token, tạo *Database* trỏ tới Neon, đăng ký mỗi view `presentation.v_*` thành *Dataset*, tạo *Chart* (loại biểu đồ, metric, nhóm theo, định dạng), tạo *Dashboard* có tab, gắn native filter và phạm vi. Nhờ vậy dashboard dựng lại giống hệt trên máy khác bằng một lệnh và được quản lý bằng Git.

5.7. Xử lý lỗi và các vấn đề kỹ thuật

Bảng 5.2. Các vấn đề kỹ thuật và cách xử lý

Vấn đề	Nguyên nhân	Cách xử lý
Máy Windows không có Python	Môi trường không đồng nhất giữa thành viên	Container `tools` (`python:3.12-slim`) chạy mọi script
`COPY`/DDL qua endpoint pooled có thể lỗi	PgBouncer ở chế độ transaction không giữ trạng thái phiên	Loader tự đổi sang endpoint direct (bỏ `-pooler`)
Superset không kết nối được Postgres	Image gốc không có driver	Cài `psycopg2-binary` trong Dockerfile
Script khởi động lỗi `\r` trên Linux	Tệp soạn trên Windows có kết thúc dòng CRLF	`sed -i 's/\r$//'` trong Dockerfile
Truy vấn đầu tiên chậm hoặc timeout	Compute Neon đang ở trạng thái scale-to-zero	`connect_timeout=30`; cache Superset 600 s; mở trước dashboard khi demo
Tổng doanh thu lệch vài xu	Dùng kiểu số thực cho tiền	Ánh xạ `DECIMAL` → `NUMERIC(p,s)`
Giá trị timestamp không hợp lệ khi COPY	Arrow lưu nano giây, Postgres chỉ tới micro giây	Ép `timestamp[ns]` → `timestamp[us]` trước khi ghi CSV
Chuỗi rỗng bị hiểu thành NULL	CSV không phân biệt ô trống và chuỗi rỗng	`quoting_style="all_valid"`: chuỗi luôn có dấu nháy, ô trống là NULL
Mất cột năm/tháng của bảng fact	Bảng gold phân vùng Hive, cột phân vùng nằm trong tên thư mục	`pyarrow.dataset(..., partitioning="hive")`
Không tạo được khoá chính	Dữ liệu có NULL hoặc trùng khoá	Bắt lỗi trong giao dịch con, lùi về chỉ mục thường và ghi cảnh báo
Bộ lọc thời gian không hoạt động	Bảng chỉ có cột năm, tháng số nguyên	View thêm cột `month_date` kiểu DATE
Bản đồ Brazil không hiển thị bang	Country Map cần mã ISO 3166-2	Bảng `dim_state` với `iso_code = 'BR-' || customer_state`
View bị xoá khi nạp lại bảng	`DROP TABLE ... CASCADE` xoá view phụ thuộc	Loader tự áp dụng lại `sql/[0-8]*.sql` sau khi nạp
Lộ mật khẩu trong log/repo	Chuỗi kết nối chứa mật khẩu	`.env` trong `.gitignore`; hàm `mask()` che mật khẩu khi log

[[CẬP NHẬT: bổ sung các lỗi thực tế nhóm gặp khi triển khai (thông báo lỗi, nguyên nhân, cách sửa) nếu khác bảng trên]]

5.8. Cách chạy các job và dịch vụ

Trình tự chạy đầy đủ (từ thư mục gốc repo, PowerShell). Các lệnh chạy script Python được liệt kê tự động từ mã nguồn hiện có:

```
# 0. Cấu hình bí mật
Copy-Item .env.example .env      # rồi điền NEON_DATABASE_URL, SUPERSET_SECRET_KEY, ...

# 1. Dựng image và khởi động Superset (http://localhost:8088)
docker compose build
docker compose up -d superset

# 3. loader/load_to_neon.py – Nạp toàn bộ output Analytics (Parquet) vào Neon Postgres.
docker compose run --rm tools python loader/load_to_neon.py

# 4. loader/apply_sql.py – Áp dụng lại các file sql/[0-8]*.sql (view tầng trình bày)
mà không nạp lại dữ liệu.
docker compose run --rm tools python loader/apply_sql.py

# 5. loader/validate.py – Kiểm chứng dữ liệu trong Neon:
```

```
docker compose run --rm tools python loader/validate.py

# 6. superset/build_dashboard.py – Dựng dashboard Superset từ code (idempotent):
database Neon -> dataset (view presentation.*)
docker compose run --rm tools python superset/build_dashboard.py

# 7. superset/import_dashboard.py – Phương án dự phòng: import dashboard từ
superset/export/dashboard_olist_presentation.zip.
docker compose run --rm tools python superset/import_dashboard.py

# 8. scripts/measure_latency.py – Đo độ trễ các truy vấn mà dashboard gửi xuống Neon
(qua endpoint pooled giống Superset).
docker compose run --rm tools python scripts/measure_latency.py

# 9. scripts/take_screenshots.py – Chụp ảnh dashboard Superset + đo thời gian tải
(cold vs cache) + đo độ trễ từng chart qua API.
docker compose --profile screenshots run --rm screenshots

# 10. scripts/neon_inventory.py – Kiểm kê đối tượng trong Neon (schema, bảng/view, số
dòng, dung lượng) từ catalog Postgres.
docker compose run --rm tools python scripts/neon_inventory.py

# 11. scripts/render_diagrams.py – Xuất các sơ đồ Mermaid trong docs/architecture.md
thành PNG (Chromium headless + mermaid.js từ CDN).
docker compose run --rm tools python scripts/render_diagrams.py

# 12. scripts/scale_experiment.py – Thí nghiệm mở rộng: nhân bản gold.fact_order_items
×N ngay trong Neon (không tốn băng thông upload),
docker compose run --rm tools python scripts/scale_experiment.py
```

Sau khi chạy xong, mở `http://localhost:8088`, đăng nhập bằng tài khoản quản trị trong `.env` và chọn dashboard của nhóm. Hướng dẫn chi tiết hơn (yêu cầu hệ thống, xử lý sự cố) ở Phụ lục A.

6. Thực nghiệm và đánh giá

6.1. Mục tiêu và kịch bản thực nghiệm

Thực nghiệm nhằm trả lời các câu hỏi kỹ thuật RQ2 (tính đúng đắn) và RQ3 (hiệu năng) đặt ra ở Chương 1, đồng thời cung cấp số liệu cho thảo luận về khả năng mở rộng (RQ5). Nhóm thiết kế sáu kịch bản:

Bảng 6.1. Các kịch bản thực nghiệm

Mã	Kịch bản	Cách thực hiện	Đại lượng đo
E1	Đối chiếu dữ liệu Parquet ↔ Neon	`loader/validate.py`: đọc lại Parquet, đếm dòng trên Postgres; chạy `sql/90_anchor_checks.sql`	Số kiểm tra đạt / tổng số
E2	Hiệu năng nạp dữ liệu	`loader/load_to_neon.py` nạp toàn bộ, ghi thời gian từng pha của từng bảng	Thời gian (s), thông lượng (dòng/s, MB/s)
E3	Độ trễ truy vấn (compute đang chạy – warm)	Đo thời gian mỗi truy vấn kiểm tra và truy vấn view từ máy client	min, trung vị, p95, max (ms)
E4	Độ trễ khi compute thức dậy (cold start)	Đợi compute scale-to-zero, gửi truy vấn đầu tiên	Thời gian truy vấn đầu tiên (ms)
E5	Hiệu quả cache Superset	Mở dashboard hai lần liên tiếp trong 10 phút	Thời gian tải dashboard (s)
E6	Kiểm thử chức năng bộ lọc	Áp dụng từng native filter, kiểm tra chart trong phạm vi thay đổi đúng	Đạt / không đạt

6.2. Dữ liệu và môi trường thử nghiệm

Dữ liệu thử nghiệm là toàn bộ 16 bảng Analytics (194.132 dòng) cộng 7 bảng gold (311.460 dòng): tổng 23 bảng, 505.592 dòng, 42,88 MB Parquet. Client là container `tools` chạy trên máy Windows của nhóm tại Việt Nam, kết nối Internet tới Neon vùng Singapore (PostgreSQL 18.6, kiến trúc aarch64). Vì vậy mọi số đo đều bao gồm **độ trễ mạng khứ hồi** và **băng thông tải lên** của đường truyền gia đình/ký túc xá — đúng với kịch bản thực tế của nhóm. **[[CẬP NHẬT: cấu hình máy client (CPU, RAM, băng thông đo được) và kích thước compute Neon lúc đo]]**

6.3. Tiêu chí đánh giá

Ngưỡng đạt được đặt theo trải nghiệm người dùng dashboard: **đúng đắn 100%** (dashboard sai số thì vô giá trị); **nạp 16 bảng Analytics dưới 120 giây** để làm mới được nhiều lần trong ngày; **độ trễ truy vấn khi compute đang chạy có trung vị dưới 300 ms và lớn nhất dưới 500 ms** (mỗi chart một truy vấn, dashboard 10–15 chart chạy song song); **cold start dưới 5 giây** cho lần mở đầu tiên trong ngày; **dashboard**

có cache dưới 2 giây; **100% chart trong phạm vi bộ lọc phản hồi đúng**. Bảng tổng hợp mức đạt ở Mục 6.9.

6.4. Kết quả E1 – Đối chiếu dữ liệu

`loader/validate.py` đọc lại từng thư mục Parquet và so với `COUNT(*)` trên Postgres cho cả 23 bảng (bảng chi tiết ở Phụ lục E), sau đó chạy các truy vấn kiểm tra số neo. Ngoài số dòng, nhóm kiểm tra các tổng tiền vì lỗi ánh xạ kiểu (ví dụ cắt mất phần thập phân) sẽ không làm thay đổi số dòng nhưng làm lệch tổng. Script sinh báo cáo cũng tính độc lập các giá trị kỳ vọng từ Parquet (Phụ lục E) để đối chiếu chéo.

Bảng 6.2. Đối chiếu số neo bàn giao của Analytics và tính nhất quán giữa các view trên Neon

Kiểm tra (sql/90_anchor_checks.sql)	Kỳ vọng	Thực tế trên Neon	Kết quả	ms
Gold SUM(revenue) – agg_sales_monthly	13.494.400,74	13.494.400,74	ĐẠT	82,2
Gold SUM(price) – fact_order_items (is_valid_sale)	13.494.400,74	13.494.400,74	ĐẠT	296,3
kpi_monthly có 20 tháng	20	20	ĐẠT	76,2
kpi_monthly không chứa 2016 và 2018-09	0	0	ĐẠT	100,7
Tổng revenue kpi_monthly (nhỏ hơn gold vì cắt tháng rỗng)	info	13.449.529,68	INFO	100,9
Tháng đỉnh doanh thu	2017-11	2017-11	ĐẠT	122,4
MoM tháng đỉnh (%)	52,06	52,06	ĐẠT	111,0
Review đơn trễ	2,56	2,56	ĐẠT	117,8
Review đơn đúng hạn	4,29	4,29	ĐẠT	84,3
Số khách mua lại (frequency ≥ 2)	2.887	2.887	ĐẠT	169,8
Tổng khách thật (customer_unique_id)	94.983	94.983	ĐẠT	226,8
Tỷ lệ khách mua lại (%)	3,04	3,04	ĐẠT	109,3
Tỷ lệ frequency = 1 (%)	96,96	96,96	ĐẠT	184,8
Seller watchlist	153	153	ĐẠT	100,8
Số seller tạo ≤80% doanh thu	539	539	ĐẠT	186,5
Nhất quán: SUM(revenue) v_orders = SUM(revenue) kpi_monthly	0	0	ĐẠT	349,1
Nhất quán: COUNT đơn v_orders = SUM(orders) kpi_monthly	0	0	ĐẠT	131,9
Nhất quán: SUM(revenue) v_sales_items = SUM(revenue) kpi_monthly	0	0	ĐẠT	166,4
Nhất quán: tổng khách theo segment = tổng khách RFM	94.983	94.983	ĐẠT	186,6
Số dòng gợi ý quyết định	7	7	ĐẠT	160,1

Kết quả: **23/23 bảng khớp số dòng** và **19/19 kiểm tra số neo/nhất quán đạt** (một dòng INFO chỉ để hiển thị tổng doanh thu khung 20 tháng). Các kiểm tra chia thành ba nhóm: (1) số neo bàn giao của Analytics — tổng doanh thu gold 13.494.400,74 BRL, tháng đỉnh 2017-11 với MoM 52,06%, review 2,56/4,29, 2.887 khách mua lại (3,04%), 153 người bán watchlist, 539 người bán tạo 80% doanh thu; (2) **nhất quán chéo giữa hai lớp dữ liệu** — tổng doanh thu và số đơn của view cấp đơn `v_orders`, tổng doanh thu của `v_sales_items` (từ gold) bằng đúng tổng của

`kpi_monthly` (từ Analytics), chênh lệch 0,00; (3) tính toàn vẹn của tầng ngữ nghĩa — tổng khách theo phân khúc bằng 94.983, `v_decision_hints` có đủ 7 luật.

Nhóm kiểm tra nhất quán chéo đặc biệt quan trọng: nó chứng minh việc bổ sung lớp gold để lọc chéo **không tạo ra "hai sự thật"** trên dashboard. Đồng thời, tổng 20 tháng của `kpi_monthly` (13.449.529,68 BRL) nhỏ hơn số neo gold vì đã loại các tháng 2016 và 09/2018 — hai con số đều đúng nhưng trả lời hai câu hỏi khác nhau, nên thẻ KPI trên dashboard ghi rõ phạm vi thời gian.

6.5. Kết quả E2 – Hiệu năng nạp dữ liệu

Kết quả nạp từng bảng (số dòng, dung lượng Parquet/CSV, thời gian COPY, tạo chỉ mục, tổng, tốc độ truyền) được trích tự động từ `evidence/load_log.csv` ở Phụ lục E.

Tổng hợp. Lần chạy `20261112-145508-40efaf` nạp 23 bảng, 505.592 dòng trong 351,34 giây (riêng pha COPY 330,80 giây), thông lượng đầu-cuối 1.439 dòng/s. Riêng 16 bảng Analytics (194.132 dòng) mất 72,66 giây, còn 7 bảng gold (311.460 dòng) mất 278,68 giây, tức khoảng 79% tổng thời gian.

Phân tích. Ba quan sát rút ra từ số liệu:

1. **Chi phí cố định mỗi bảng chiếm ưu thế ở bảng nhỏ.** 15 bảng dưới 1.000 dòng mất trung bình 1,73 giây mỗi bảng dù dữ liệu chỉ vài KB: thời gian này là tổng của nhiều vòng khứ hồi mạng tới Singapore (DROP, CREATE, COPY, COMMENT, commit, tạo khoá chính, ANALYZE, đếm dòng, ghi log). Thông lượng "dòng/giây" của các bảng này vì vậy không có ý nghĩa; cách tối ưu là giảm số vòng khứ hồi (gộp nhiều bảng nhỏ trong một giao dịch, nạp song song) chứ không phải tăng tốc truyền.
2. **Bảng lớn bị giới hạn bởi băng thông tải lên.** Với các bảng cấp khách hàng và bảng fact, phần lớn thời gian nằm ở pha COPY. Dữ liệu được truyền dưới dạng CSV văn bản, lớn hơn Parquet nén khoảng 2,5–3 lần (107,36 MB CSV so với 42,88 MB Parquet), với tốc độ trung bình khoảng 0,32 MB/s — tương ứng vài Mbit/s, phù hợp với băng thông tải lên của kết nối Internet thông thường hơn là giới hạn của Postgres. Điều này giải thích vì sao `agg_customer_rfm` (9 cột) nạp nhanh hơn rõ rệt so với `customer_segments` (13 cột) dù cùng 94.983 dòng.

3. **Bảng gold chiếm phần lớn thời gian** vì có số cột lớn (27–29 cột) và nhiều dòng nhất. Đây là chi phí của quyết định bổ sung lớp lọc chéo; nếu chỉ cần dashboard insight, 16 bảng Analytics nạp trong 72,66 giây.

Từ đó, hướng tối ưu hợp lý nhất là **chạy loader gần Neon** (máy ảo cùng vùng AWS hoặc trong pipeline CI) để loại bỏ giới hạn băng thông tải lên, và chỉ nạp lại các bảng thay đổi. Điều này được thảo luận ở Mục 8.3.

6.6. Kết quả E3, E4 – Độ trễ truy vấn

Script `scripts/measure_latency.py` chạy đúng các truy vấn mà các chart trên dashboard sinh ra, qua endpoint *pooled* (PgBouncer) giống Superset, mỗi truy vấn 5,00 lần sau một lần làm nóng; ghi nhận thời gian phía client (mạng + thực thi) và thời gian thực thi phía server (`EXPLAIN ANALYZE`). Kết quả (đo lúc 2026-11-12 15:48:26):

Bảng 6.3. Độ trễ truy vấn các view trên Neon (9/9 phép đo thành công; tệp `evidence/dashboard_load_latency.md`)

Tab	Số thành phần	Cold (s)	Cache (s)
1 · Tổng quan & xu hướng	11	6,39	0,91
2 · Danh mục & mùa vụ	4	2,24	0,94
3 · Địa lý & logistics	5	12,66	0,95
4 · Khách hàng & RFM	10	9,93	1,18
5 · Seller	7	5,16	2,08
6 · Giao trễ, review & thanh toán	6	5,24	1,06
7 · Gợi ý quyết định & nguồn dữ liệu	4	2,51	1,04
Trung vị		**5,24**	**1,04**
Tổng 7 tab	47	**44,13**	**8,16**

Bảng 6.4. Độ trễ truy vấn các view trên Neon (22/22 phép đo thành công; tệp `evidence/query_latency.csv`)

Tab	Truy vấn	Số dòng	Client trung vị (ms)	Client p95 (ms)	Server thực thi (ms)
Tổng quan	Thẻ KPI (6 chỉ số) – v_orders	1	244.0	427.3	167.87
Tổng quan	Thẻ KPI có lọc bang SP + 2018	1	151.0	442.0	50.27
Tổng quan	Xu hướng tháng – v_kpi_monthly	20	174.4	255.5	0.16
Danh mục	Pareto top 25 – v_category_pareto	25	129.6	235.5	0.24
Danh mục	Heatmap mùa vụ – v_category_seasonality	144	207.5	425.4	0.48
Danh mục	Xu hướng danh mục – v_category_monthly	1246	287.6	352.6	1.48
Danh mục	Doanh thu danh mục × bang (lọc 3 danh mục) – v_sales_items	81	171.0	329.4	53.0
Địa lý	Bản đồ / bảng bang – v_state_logistics	27	129.8	226.0	0.22
Địa lý	Khoảng cách – v_insight_distance	4	126.0	219.2	0.17
Khách hàng	Pie phân khúc RFM – v_rfm_customers (94.983 dòng)	6	189.0	301.2	95.88
Khách hàng	Cụm K-Means lọc bang RJ + 3 segment	1	173.4	226.4	1.25
Khách hàng	Bảng segment – v_rfm_segments	6	247.0	374.1	151.95
Seller	Pareto nhóm seller – v_seller_pareto	3	201.7	302.7	2.36
Seller	Bubble top 300 seller – v_seller_pareto	300	129.6	443.9	2.5
Seller	Bảng watchlist – v_seller_watchlist	153	151.7	444.3	0.51
Review/PTTT	Trễ vs review – v_late_vs_review	2	131.5	254.5	0.11
Review/PTTT	Ngưỡng ngày trễ – v_delay_vs_review	5	99.7	234.0	0.11

Tab	Truy vấn	Số dòng	Client trung vị (ms)	Client p95 (ms)	Server thực thi (ms)
Review/PTTT	Payment mix – v_payment_mix	10	108.1	206.6	0.14
Review/PTTT	Pie PTTT – v_orders	4	192.1	405.8	117.72
Quyết định	Gợi ý quyết định – v_decision_hints	7	189.7	282.3	108.16
Quyết định	Nhật ký nạp – v_load_log	23	186.4	197.8	0.18
So sánh	Pareto danh mục tính lại từ gold.fact_order_items (không dùng bảng tổng hợp)	25	171.4	337.4	66.43

Bảng 6.5. Độ trễ truy vấn đo qua Superset (44/44 phép đo thành công; tệp `evidence/superset_chart_latency.csv`)

Loại	Số dòng	force_median_ms	cache_median_ms
big_number_total	1	739.7	46.4
big_number_total	1	710.0	44.4
big_number_total	1	787.7	45.2
big_number_total	1	723.6	57.2
big_number_total	1	742.0	58.3
big_number_total	1	723.9	54.8
big_number	20	767.7	49.9
mixed_timeseries	40	1487.9	56.3
mixed_timeseries	40	1469.8	65.6
echarts_timeseries_bar	20	702.5	49.9
table	20	758.2	112.2
mixed_timeseries	50	1565.4	74.8
heatmap_v2	144	722.5	64.0
echarts_timeseries_bar	15	709.5	60.7
echarts_timeseries_line	20	745.3	65.3
bubble_v2	27	700.5	54.8
echarts_timeseries_bar	27	762.6	54.6
mixed_timeseries	8	1383.1	55.5
table	27	711.0	53.4
big_number_total	1	758.4	47.6
big_number_total	1	721.6	51.7
big_number_total	1	802.8	54.1
big_number_total	1	738.1	56.0
pie	6	775.3	53.0
echarts_timeseries_bar	4	772.0	61.7
echarts_timeseries_bar	10	744.3	49.1
echarts_timeseries_bar	10	723.7	48.1
table	6	777.4	46.5
table	4	699.6	47.6
big_number_total	1	709.4	47.4
big_number_total	1	683.6	52.2
big_number_total	1	707.7	55.9
big_number_total	1	710.9	70.2
mixed_timeseries	6	1388.1	75.8
bubble_v2	300	832.1	62.0
table	153	836.0	60.3
echarts_timeseries_bar	2	721.5	55.3
mixed_timeseries	10	1396.7	64.2
echarts_timeseries_bar	4	717.5	62.6
echarts_timeseries_bar	4	693.4	47.1

Phân tích. Trên 21 truy vấn dashboard, trung vị phía client là **173,4 ms** (nhỏ nhất 99,7 ms, lớn nhất 287,6 ms; p95 lớn nhất 444,3 ms), trong khi trung vị thời gian **thực thi trên server chỉ 1,25 ms**. Riêng một vòng khứ hồi mạng `SELECT 1` từ máy nhóm tới Neon Singapore đã mất 105 ms. Ba kết luận:

1. **Mạng chiếm gần như toàn bộ độ trễ.** Với các view trên bảng Analytics đã tổng hợp, Postgres thực thi dưới 3 ms; phần còn lại là đường truyền. Tối ưu SQL ở quy mô này gần như không cải thiện trải nghiệm — đặt Superset cùng vùng với Neon hoặc dùng cache mới có tác dụng.
2. **Bảng tổng hợp sẵn của tầng Big Data trả giá trị rõ rệt.** Dòng "So sánh" tính lại Pareto danh mục trực tiếp từ `gold.fact_order_items` tốn khoảng 66 ms thực thi, so với 0,24 ms khi đọc `v_category_pareto` do Spark tính sẵn — chênh hơn 250 lần. Các truy vấn trên lớp cấp đơn/khách hàng (`v_orders`, `v_rfm_customers`, `v_rfm_segments`, `v_decision_hints`: 50–170 ms server) cũng chậm hơn hẳn lớp insight. Ở quy mô ×100, chênh lệch này quyết định dashboard có tương tác được hay không — minh chứng cho nguyên tắc "tính nặng ở batch layer".
3. **Chi phí mở kết nối lớn hơn chi phí truy vấn.** Thiết lập một kết nối TLS mới tới Neon mất khoảng 1.134 ms, gấp nhiều lần thời gian truy vấn; Superset mặc định mở kết nối mới cho mỗi truy vấn. Endpoint pooled của Neon và cache 10 phút của Superset giảm đáng kể chi phí này; đây cũng là lý do cache ở tầng BI quan trọng hơn tối ưu SQL.

Bộ kiểm tra E1 cho kết quả cùng bậc: 20 truy vấn kiểm tra có trung vị 127,2 ms, lớn nhất 349,1 ms. Truy vấn đầu tiên sau khi compute ngủ còn chịu thêm thời gian khởi động compute và nạp lại cache trang từ pageserver — cái giá của scale-to-zero, chấp nhận được với dashboard nội bộ vì đổi lại không tốn compute ngoài giờ làm việc. **[[CẬP NHẬT: số đo cold start thực tế (ms) sau khi compute ở trạng thái Idle]]**

6.7. Kết quả E5, E6 – Dashboard và bộ lọc

Bảng 6.6. Kết quả kiểm thử dashboard và bộ lọc

Kiểm thử	Kỳ vọng	Kết quả
Mở dashboard lần đầu (cache trống)	< 5 s	**[[CẬP NHẬT: số giây]]**
Mở lại trong 10 phút (có cache)	< 2 s	**[[CẬP NHẬT: số giây]]**
Bộ lọc khoảng tháng	Thẻ KPI (`v_orders`) và chart theo tháng chỉ tính các tháng đã chọn	**[[CẬP NHẬT: Đạt/Không]]**
Bộ lọc bang	Thẻ KPI, bản đồ, scatter, phân khúc khách theo bang thay đổi	**[[CẬP NHẬT: Đạt/Không]]**
Bộ lọc danh mục	Xu hướng ngành (`v_category_monthly`), doanh thu theo danh mục thay đổi	**[[CẬP NHẬT: Đạt/Không]]**
Bộ lọc phân khúc RFM	Biểu đồ và bảng khách hàng thay đổi	**[[CẬP NHẬT: Đạt/Không]]**
Cross-filter (nhấn vào bang trên bản đồ)	Các chart cùng dataset lọc theo bang	**[[CẬP NHẬT: Đạt/Không]]**

6.8. Tham chiếu hiệu năng giai đoạn trước

Để đặt số liệu của nhóm trong bối cảnh toàn pipeline, bảng dưới đây so sánh thời gian xử lý giữa các giai đoạn. Giai đoạn Processing và Analytics phải quét và join dữ liệu chi tiết trên cụm Spark; giai đoạn Presentation chủ yếu truyền dữ liệu qua mạng một lần, sau đó phục vụ mỗi truy vấn trong vài trăm mili giây. Đây là minh hoạ trực tiếp cho nguyên tắc "tính nặng ở batch layer, phục vụ nhẹ ở serving layer".

Bảng 6.7. Thời gian xử lý theo giai đoạn của pipeline

Giai đoạn	Công việc	Dữ liệu	Thời gian
Storage → Processing	Silver → gold (7 bảng, 11 đối chiếu) trên Spark 4 core	569.698 dòng vào	61,72 s
Processing → Analytics	14 SQL + K-Means trên Spark 4 core	7 bảng gold, 311.460 dòng	119,71 s
Analytics → Presentation (nạp 16 bảng Analytics)	Parquet → Neon bằng COPY qua Internet	194.132 dòng	72,66 s
Analytics → Presentation (nạp 7 bảng gold)	Parquet → Neon bằng COPY qua Internet	311.460 dòng	278,68 s
Analytics → Presentation (truy vấn)	Một truy vấn phục vụ một chart	Bảng đã cô đọng / có chỉ mục	trung vị 173,4 ms client, 1,25 ms server

6.9. Đánh giá tổng hợp

Bảng 6.8. Đánh giá mức đạt theo tiêu chí

Tiêu chí	Ngưỡng	Kết quả	Đánh giá
Đúng đắn dữ liệu	100%	23/23 bảng; 19/19 kiểm tra	Đạt
Thời gian nạp 16 bảng Analytics	< 120 s	72,66 s	Đạt
Độ trễ truy vấn dashboard, trung vị (client)	< 300 ms	173,4 ms (server 1,25 ms)	Đạt
Độ trễ truy vấn dashboard, p95 lớn nhất	< 500 ms	444,3 ms	Đạt
Cold start	< 5 s	**[[CẬP NHẬT: ms]]**	**[[CẬP NHẬT]]**
Dashboard có cache	< 2 s	**[[CẬP NHẬT: s]]**	**[[CẬP NHẬT]]**
Bộ lọc	100%	**[[CẬP NHẬT]]**	**[[CẬP NHẬT]]**

Về câu hỏi RQ2, kết quả E1 cho thấy tầng phục vụ tái tạo chính xác tuyệt đối kết quả của Spark, kể cả các số thập phân tiền tệ, và hai lớp dữ liệu (Analytics, gold) nhất quán với nhau. Về RQ3, cả thời gian nạp lẫn độ trễ truy vấn đều bị chi phối bởi mạng chứ không bởi cơ sở dữ liệu (server thực thi trung vị 1,25 ms), và độ trễ truy vấn dashboard nằm trong ngưỡng tương tác.

7. Kết quả demo

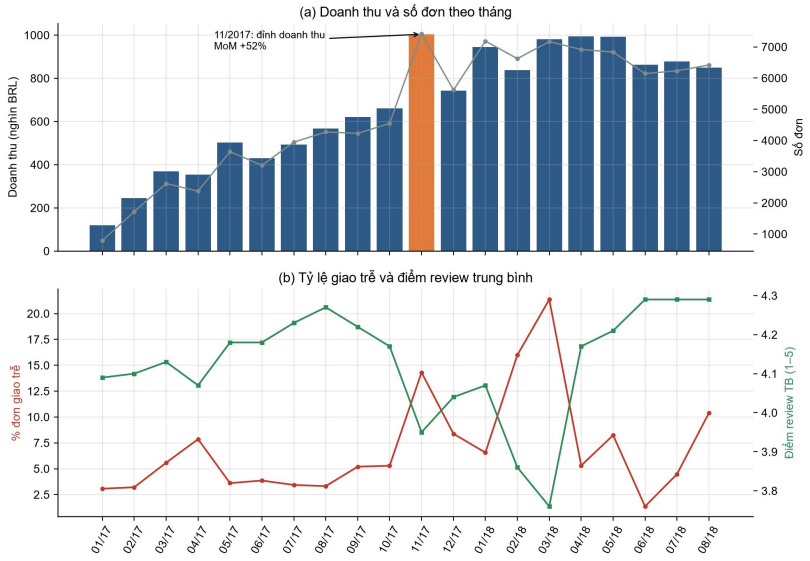
7.1. Kịch bản demo từ đầu vào đến đầu ra

1. **Đầu vào:** mở `data/analytics/` (16 thư mục bảng Parquet), dùng pyarrow in schema và số dòng của `kpi_monthly`.
2. **Nạp dữ liệu:** chạy loader trong container `tools`; log hiển thị từng bảng, số dòng, thời gian và dòng tổng; `analytics.load_log` có thêm một `run_id`.
3. **Kiểm tra:** chạy `loader/validate.py`; 23/23 bảng khớp số dòng, 19/19 kiểm tra số neo "ĐẠT".
4. **Tầng ngữ nghĩa:** trong SQL Lab (hoặc psql) truy vấn `v_kpi_overview`, `v_seller_watchlist`, `v_decision_hints`.
5. **Dashboard:** đi qua từng tab; áp dụng bộ lọc khoảng tháng, bang, danh mục, phân khúc; nhấn một bang trên bản đồ để cross-filter.
6. **Đầu ra cho Decision/Action:** truy vấn view gợi ý quyết định, xuất CSV, minh hoạ một luật hành động.
7. **Serverless:** Neon Console cho thấy compute co giãn và về trạng thái nghỉ sau khi ngừng truy cập.

7.2. Nội dung phân tích được trình bày

Các biểu đồ dưới đây do script sinh báo cáo vẽ trực tiếp từ các bảng Parquet đầu vào (cùng nguồn với dashboard), minh hoạ thông điệp của từng tab và kiểm chứng các số neo. Ảnh chụp dashboard thật ở Mục 7.3.

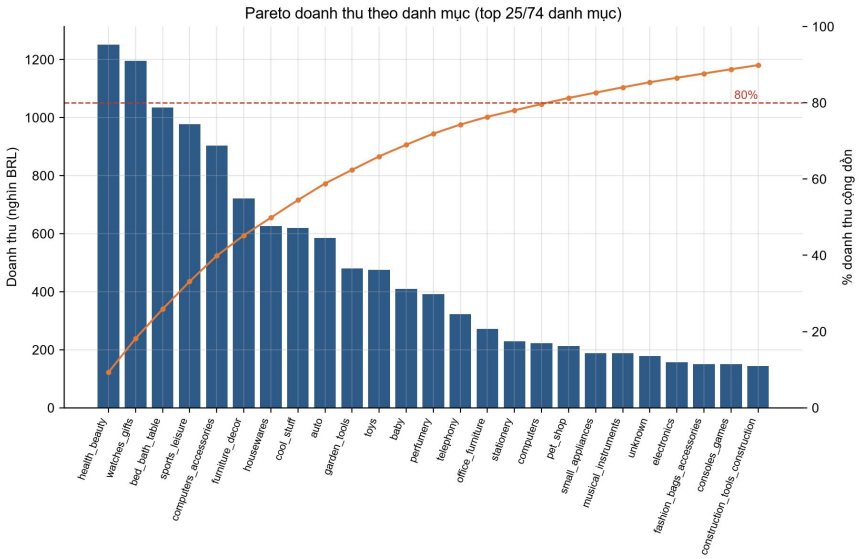
7.2.1. Tổng quan: tăng trưởng và sức khoẻ vận hành



Hình 7.1. *Doanh thu, số đơn, tỷ lệ giao trễ và điểm review theo tháng (2017-01 → 2018-08; cột cam = Black Friday)*

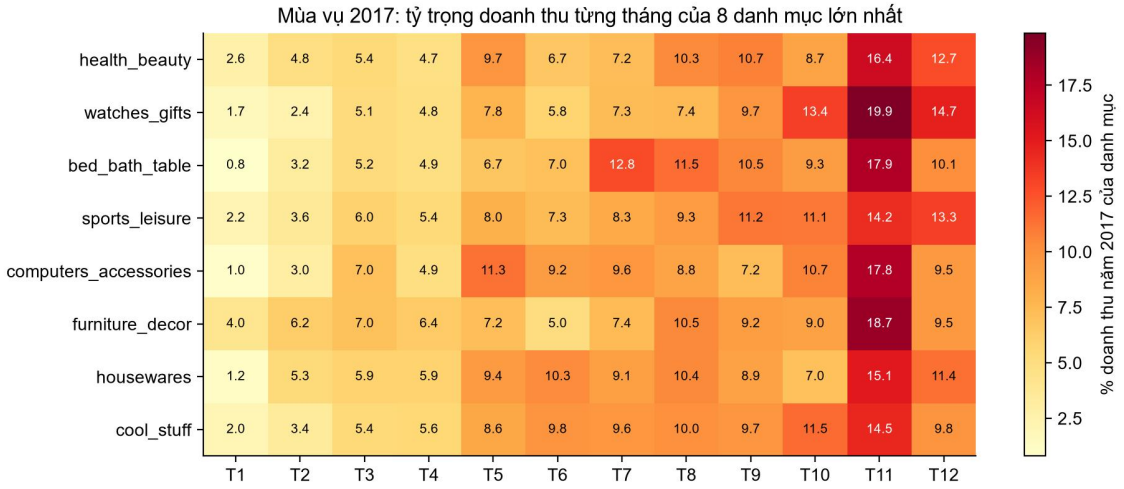
Doanh thu tăng từ ~120 nghìn BRL (01/2017) lên đỉnh **1.003.862 BRL vào 11/2017** (Black Friday, MoM +52,06%, 7.421 đơn), rồi giữ quanh 0,85–1,0 triệu BRL/tháng trong 2018. Thông điệp cho ban điều hành: **bán nhiều không đồng nghĩa vận hành tốt** — 11/2017 tỷ lệ trễ vọt lên 14,31%, review rơi còn 3,95; 03/2018 tệ nhất với 21,36% trễ và review 3,76; trong khi 04–05/2018 doanh thu gần bằng đỉnh mà trễ chỉ 5–8%, review ~4,2. Trước các đợt khuyến mãi phải tăng năng lực giao vận tương ứng.

7.2.2. Danh mục: Pareto và mùa vụ



Hình 7.2. *Pareto doanh thu theo danh mục: không danh mục nào vượt 10%, top 10 đạt ~62%*

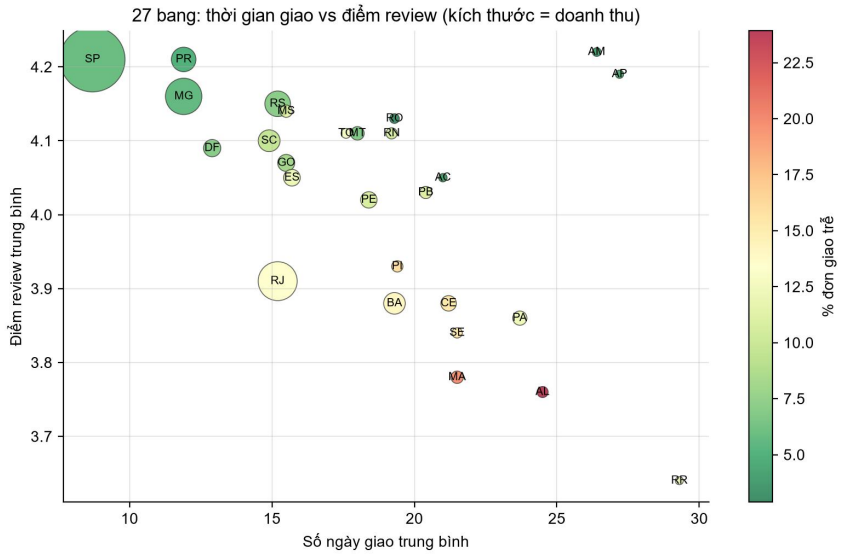
`health_beauty` dẫn đầu (9,30%), tiếp theo `watches_gifts` (8,88%), `bed_bath_table` (7,70%). 17 danh mục lớn nhất mới chiếm 79,7% doanh thu, đến danh mục thứ 18 mới vượt 80% — "đuôi dài" điển hình của sàn đa ngành. Chiến dịch nên theo cụm ngành; cắt bỏ hàng loạt ngành nhỏ sẽ mất ~38% doanh thu nằm ngoài top 10.



Hình 7.3. *Mùa vụ 2017: tỷ trọng doanh thu từng tháng trong năm của 8 danh mục lớn nhất*

Cả tám danh mục lớn đều đạt đỉnh tháng 11; `watches_gifts` có gần 20% doanh thu năm rơi vào tháng 11 — tồn kho và ngân sách quảng cáo cần dồn vào quý IV.

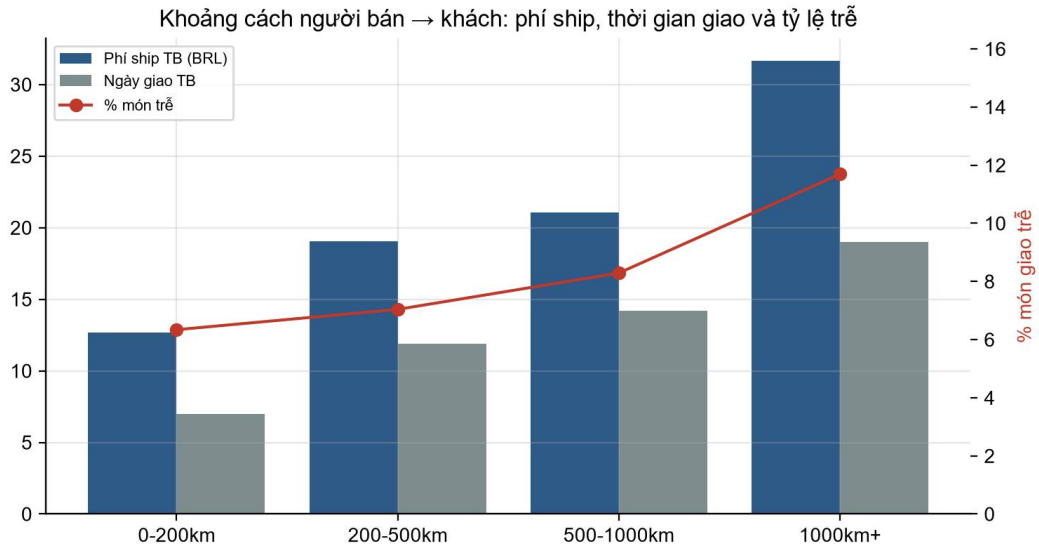
7.2.3. Địa lý và logistics



Hình 7.4. *27 bang theo số ngày giao trung bình và điểm review (kích thước = doanh thu, màu = % giao trễ)*

SP chiếm 38,27% doanh thu, giao 8,7 ngày, trễ 5,89%, review 4,21 — góc "nhanh và hài lòng" vì phần lớn người bán ở SP. Các bang xa như BA (19,3 ngày, trễ

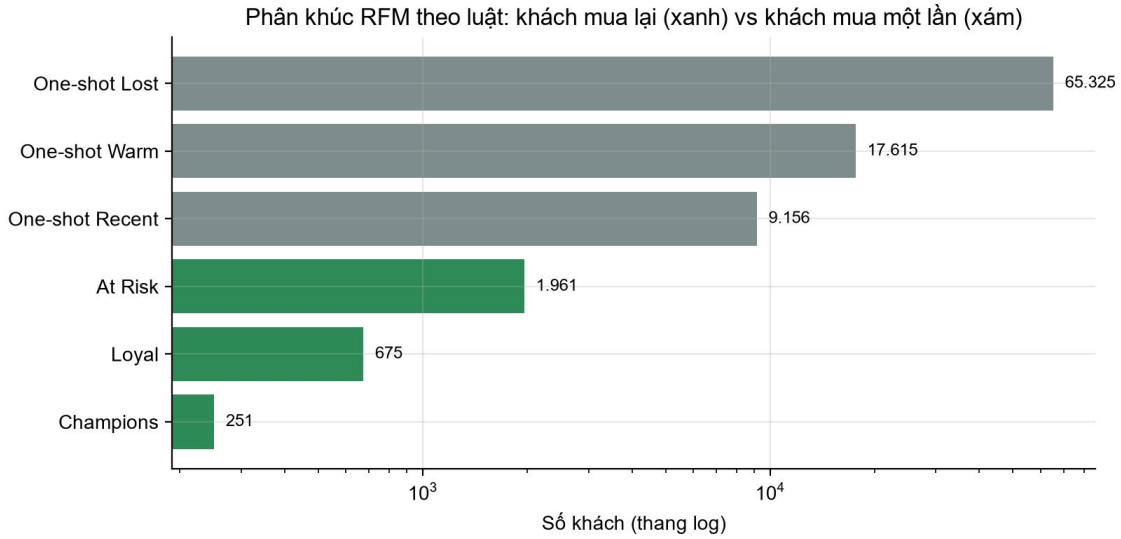
14,04%, review 3,88) và RJ (15,2 ngày, 13,47%, 3,91) ở góc ngược lại dù giá trị đơn cao hơn (BA 181 BRL so với SP 143 BRL).



Hình 7.5. *Phí vận chuyển, thời gian giao và tỷ lệ trễ theo khoảng cách người bán → khách*

Từ 0–200 km lên trên 1.000 km, phí ship tăng từ 12,66 lên 31,65 BRL, thời gian giao từ 7,0 lên 19,0 ngày, tỷ lệ trễ từ 6,34% lên 11,69%. Hệ số Pearson khoảng cách – phí ship 0,39, khoảng cách – ngày giao 0,39, nhưng khoảng cách – số ngày trễ so với hẹn chỉ −0,08: ngày hẹn đã được nới cho đường dài, vấn đề là thời gian chờ thực tế. Hàm ý: ưu tiên kho/người bán theo vùng thay vì siết hạn hẹn.

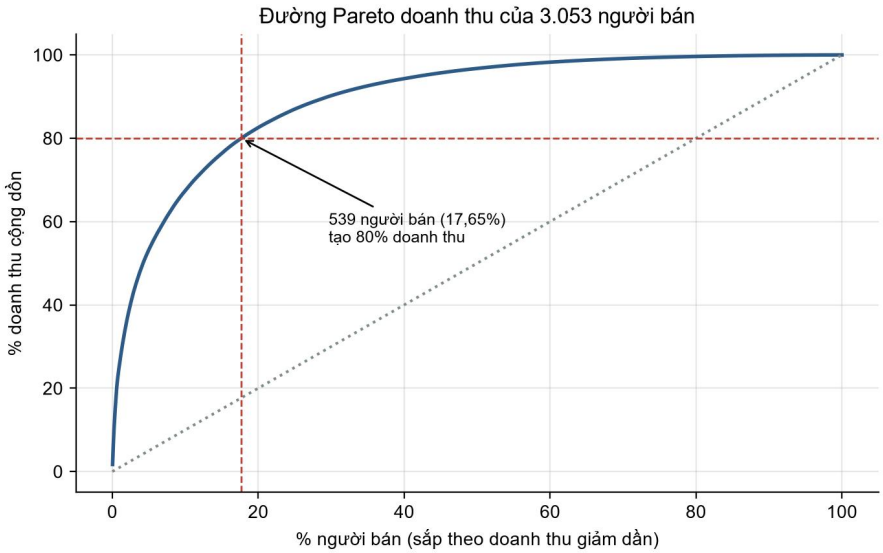
7.2.4. Khách hàng: giữ chân và phân khúc



Hình 7.6. *Số khách theo phân khúc RFM (thang log): ba phân khúc khách mua lại chỉ chiếm ~3%*

96,96% khách chỉ mua một lần; **2.887 khách (3,04%) mua lại**, chi trọn đời 308 BRL so với 161 BRL, nhưng giá trị mỗi đơn không cao hơn (146 so với 161 BRL) — họ mua thêm lần chứ không mua đắt hơn. Phân khúc theo luật: 251 Champions, 675 Loyal, 1.961 At Risk; K-Means (k = 4) tự tách đúng 2.887 khách mua lại thành một cụm. Ngân sách CRM nên tập trung vào At Risk và One-shot Recent thay vì 65 nghìn khách One-shot Lost.

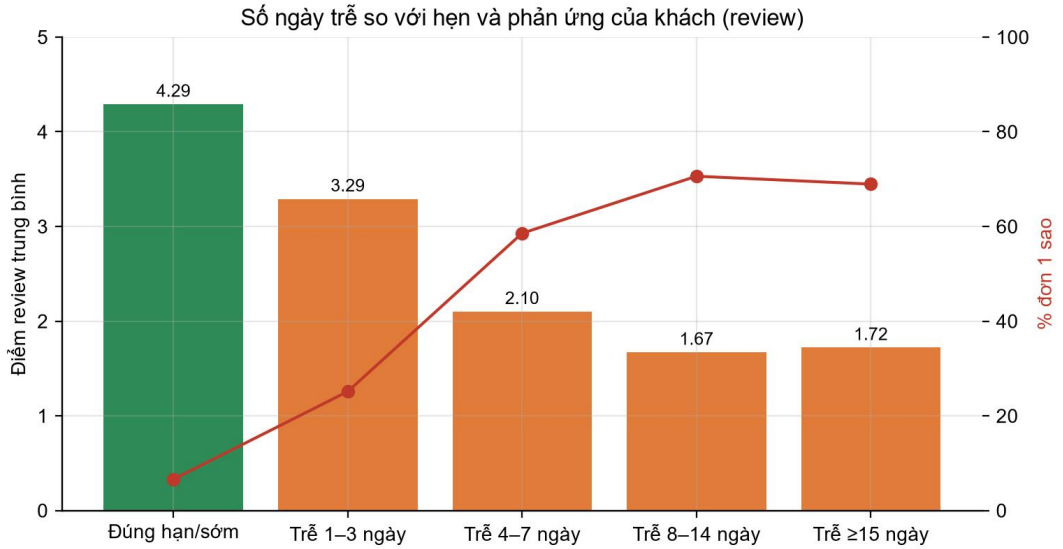
7.2.5. Người bán



Hình 7.7. *Đường Pareto doanh thu của người bán*

Chỉ 539/3.053 người bán (17,65%) tạo 80% doanh thu — tập trung hơn quy luật 20/80. Kết hợp điều kiện vận hành kém, `seller_watchlist` chọn ra **153 người bán** thuộc top 10% doanh thu nhưng review < 4, giao > 15 ngày hoặc trễ > 10% — đầu vào trực tiếp cho giai đoạn Decision/Action.

7.2.6. Giao trễ, đánh giá và thanh toán



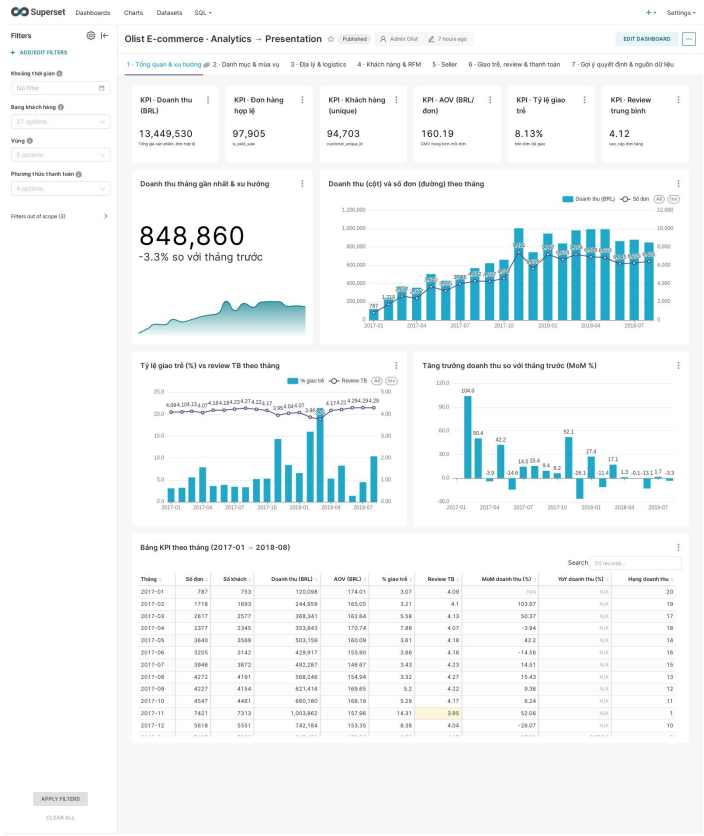
Hình 7.8. *Điểm review trung bình và tỷ lệ đơn 1 sao theo số ngày giao trễ so với hẹn*

Đơn trễ chỉ đạt **2,56 sao so với 4,29** (chênh 1,73 sao); 54,07% đơn trễ bị 1–2 sao so với 9,20%. Trễ 1–3 ngày còn 3,29 sao, trễ 4–7 ngày còn 2,10 sao với 58,6% một sao, từ 8 ngày trở đi chạm đáy ~1,7 sao — căn cứ cho luật "can thiệp trước mốc 4 ngày trễ".

Về thanh toán, thẻ tín dụng chiếm ~75% số đơn, boleto ~20%. Đơn trả góp ≥ 7 kỳ có giá trị trung bình ~335 BRL so với ~101 BRL của đơn thẻ trả một lần — tương quan (đơn đắt mới cần trả góp), không chứng minh "bật trả góp sẽ tăng giá trị đơn".

7.3. Ảnh chụp hệ thống đang chạy

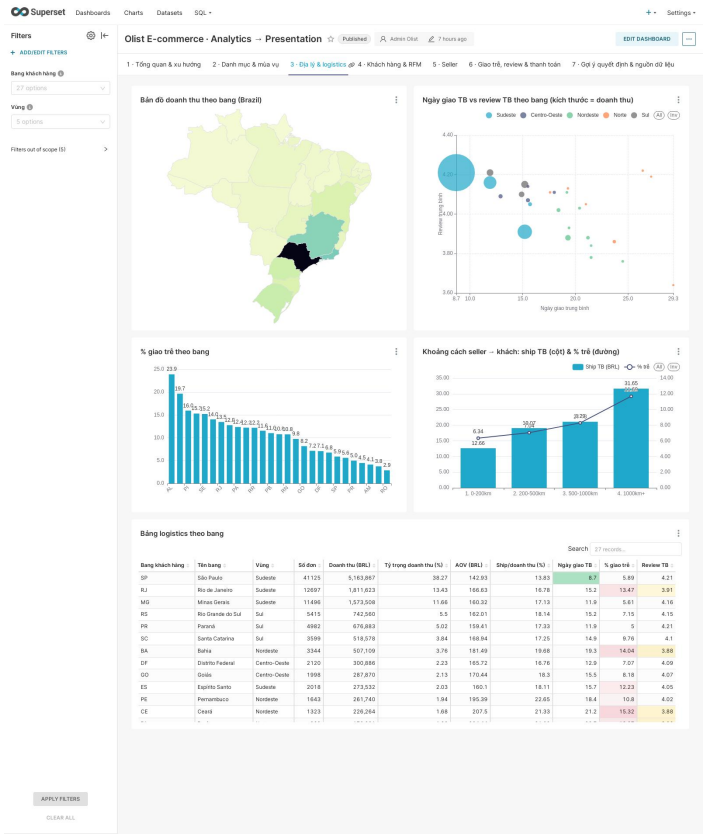
Các ảnh dưới đây được chèn tự động từ `evidence/` (chụp bằng Playwright hoặc chụp tay), tương ứng các bước của kịch bản ở Mục 7.1.



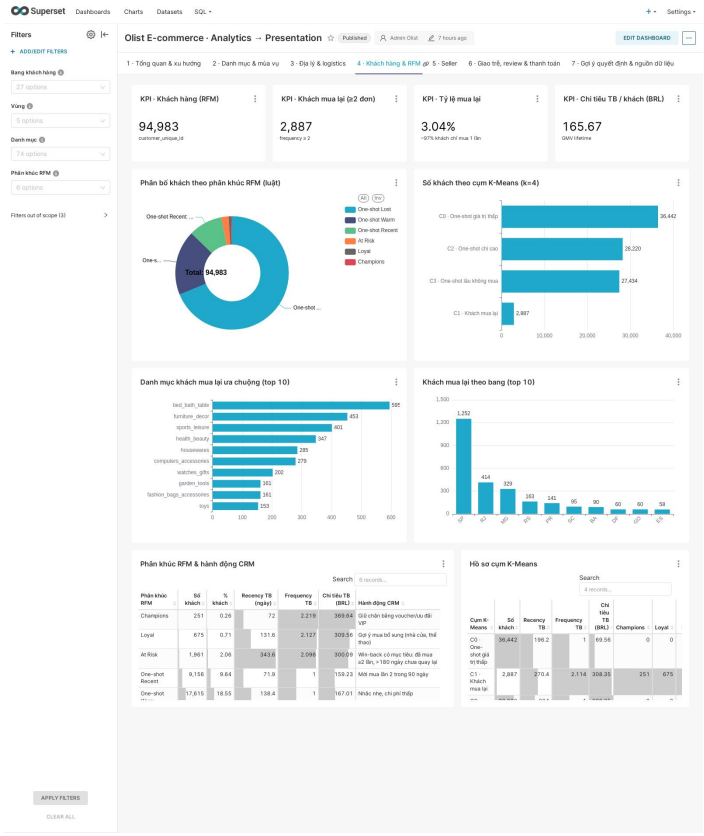
Hình 7.9. *Dashboard Apache Superset – dashboard tab1 tong quan xu huong (tệp dashboard_tab1_tong_quan_xu_huong.png)*



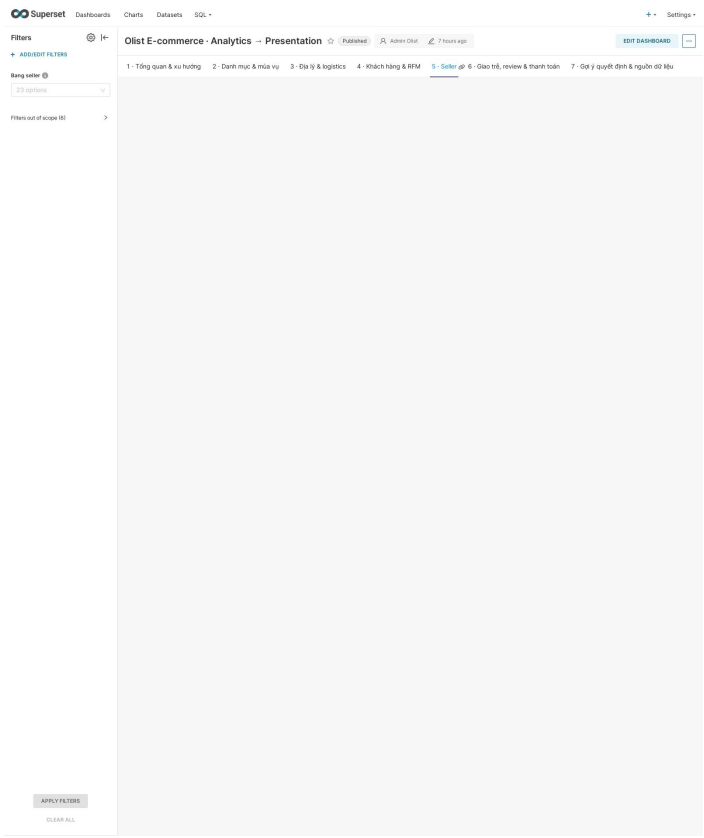
Hình 7.10. *Dashboard Apache Superset – dashboard tab2 danh muc mua vu (tệp dashboard_tab2_danh_muc_mua_vu.png)*



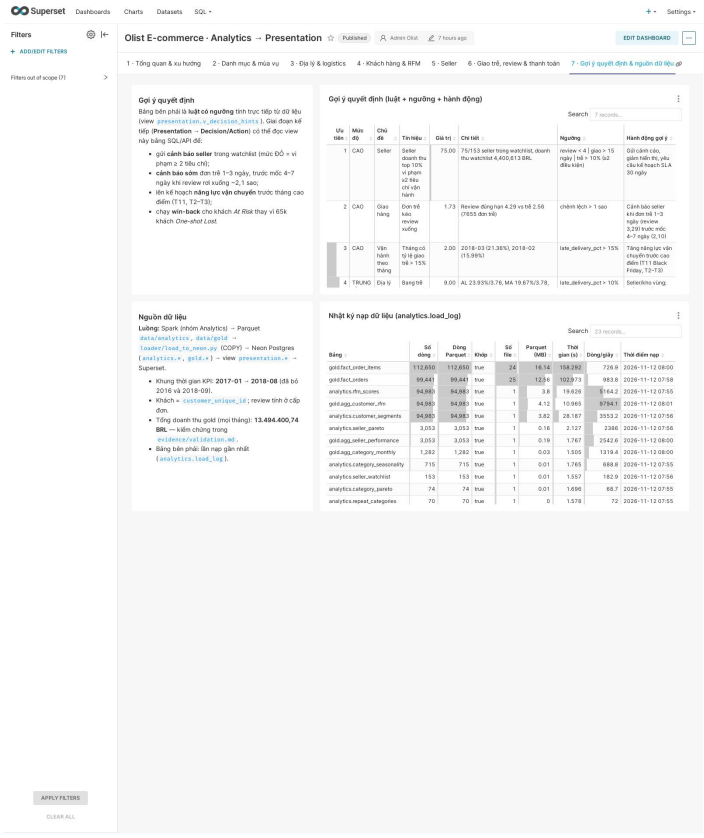
Hình 7.11. *Dashboard Apache Superset – dashboard tab3 dia ly logistics (tệp dashboard_tab3_dia_ly_logistics.png)*



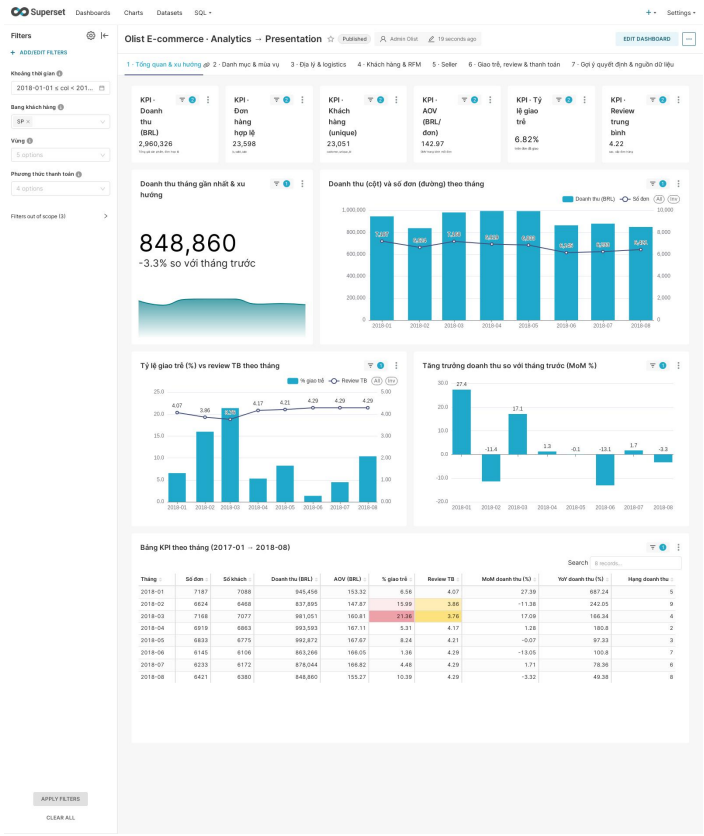
Hình 7.12. *Dashboard Apache Superset – dashboard tab4 khach hang rfm (tệp dashboard_tab4_khach_hang_rfm.png)*



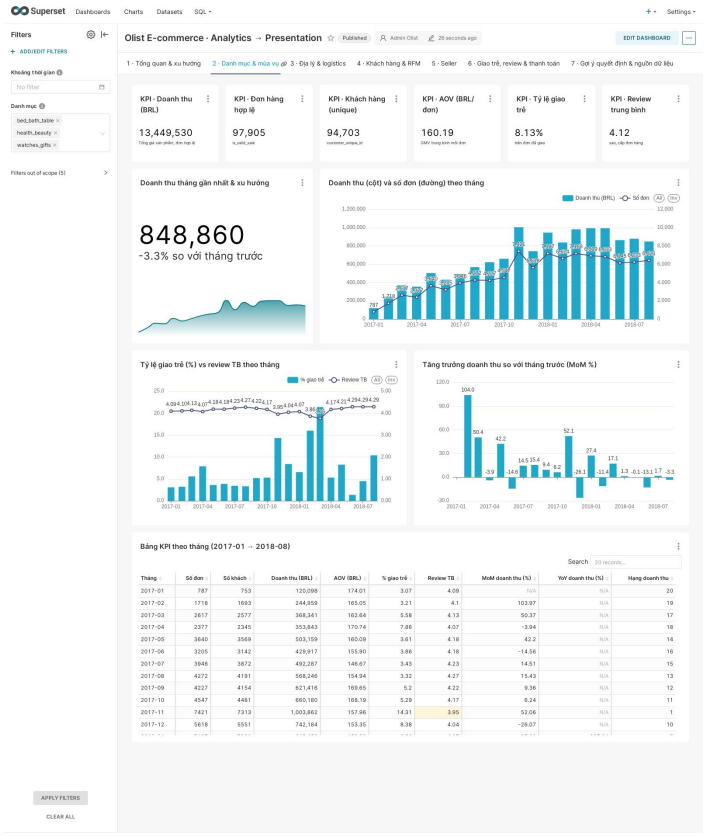
Hình 7.13. *Dashboard Apache Superset – dashboard tab5 seller (tệp dashboard_tab5_seller.png)*



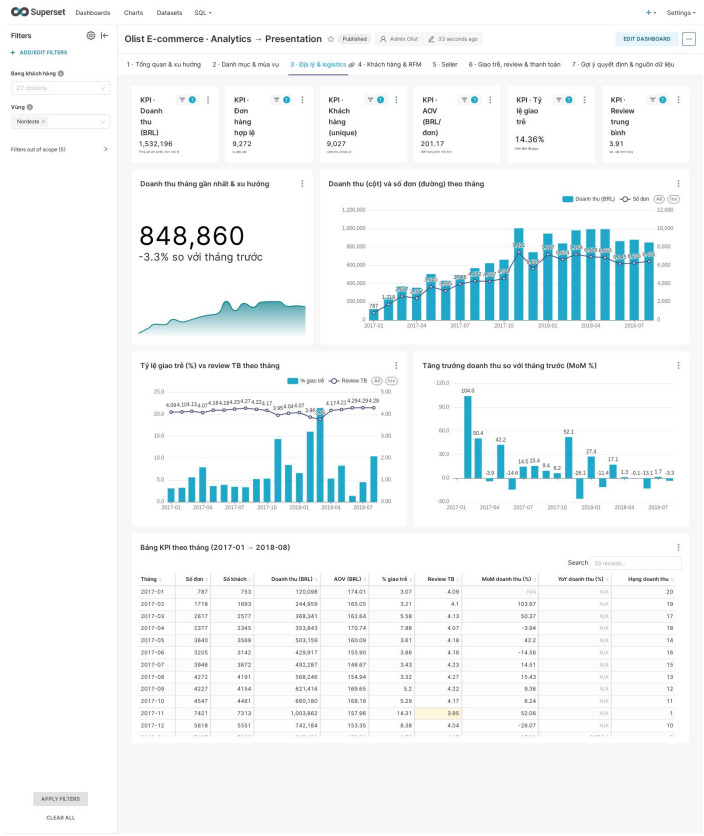
Hình 7.14. *Dashboard Apache Superset – dashboard tab7 goi y quyet dinh nguon du lieu (tệp dashboard_tab7_goi_y_quyet_dinh_nguon_du_lieu.png)*



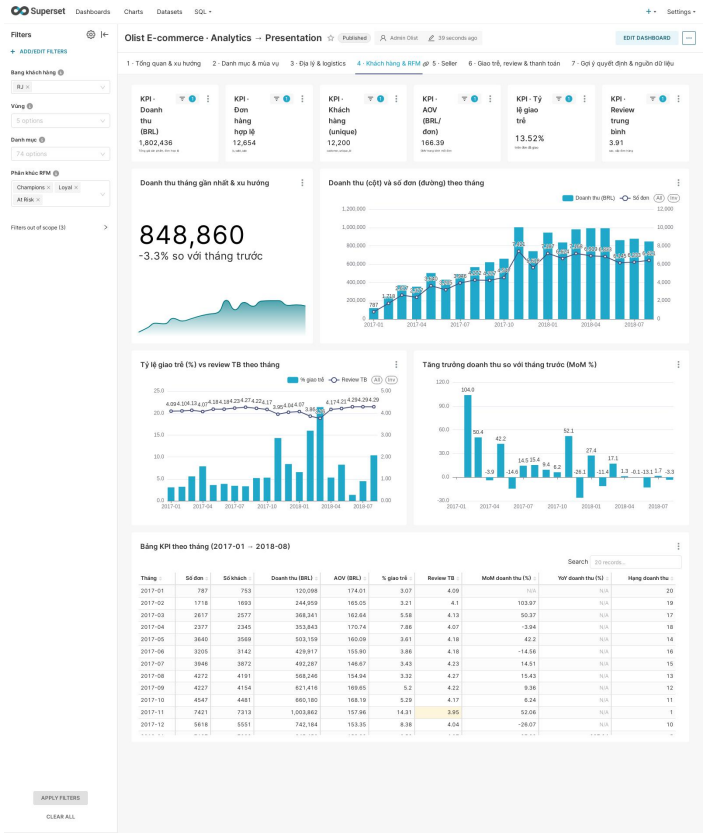
Hình 7.15. *Dashboard khi áp dụng bộ lọc native – filter tab1 thoi gian 2018 bang SP (tệp filter_tab1_thoi_gian_2018_bang_SP.png)*



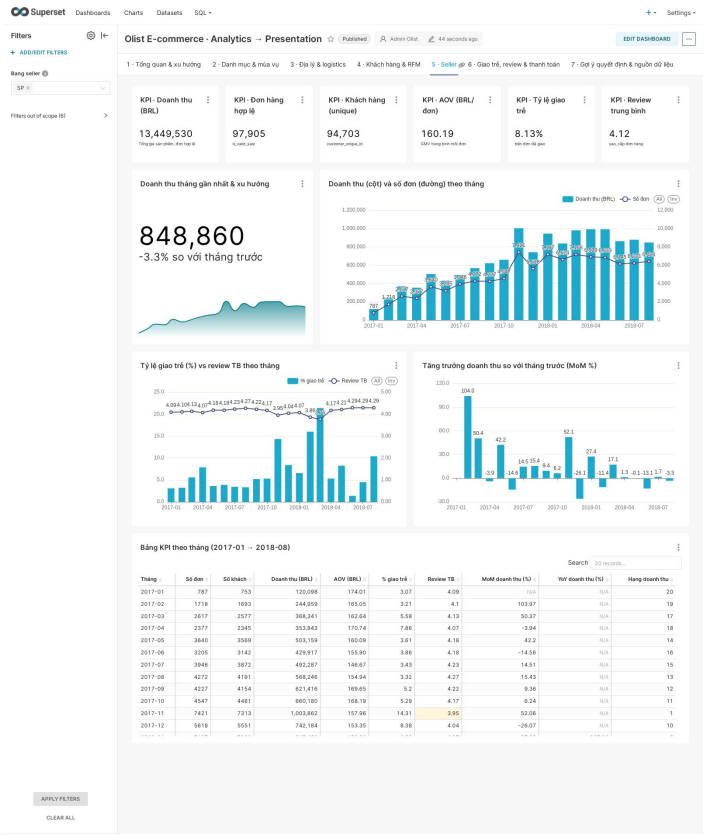
Hình 7.16. *Dashboard khi áp dụng bộ lọc native – filter tab2 danh muc bed bath health watches (tệp filter_tab2_danh_muc_bed_bath_health_watches.png)*



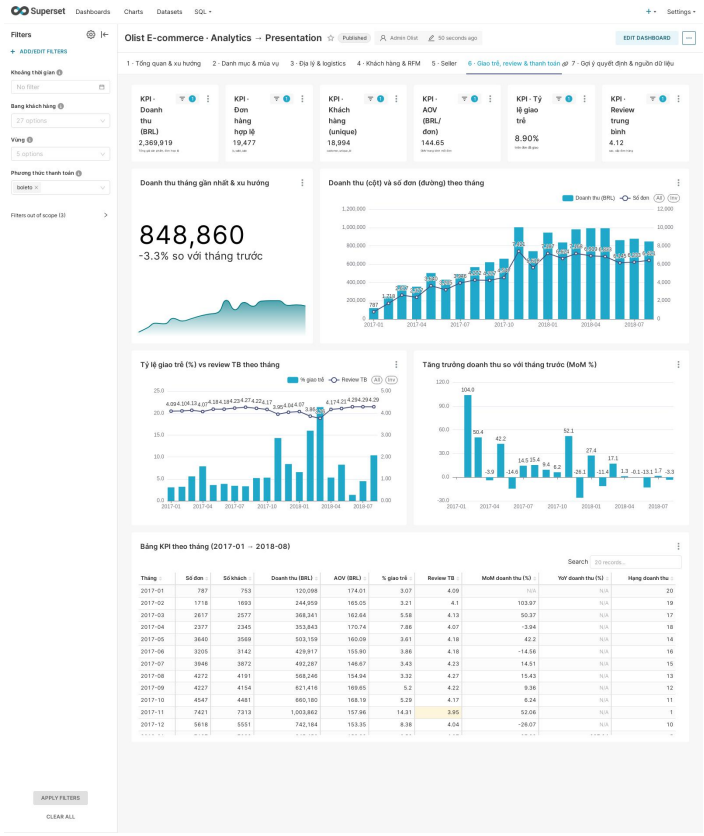
Hình 7.17. *Dashboard khi áp dụng bộ lọc native – filter tab3 vung nordeste (tệp filter_tab3_vung_nordeste.png)*



Hình 7.18. *Dashboard khi áp dụng bộ lọc native – filter tab4 rfm khach mua lai bang RJ (tệp filter_tab4_rfm_khach_mua_lai_bang_RJ.png)*



Hình 7.19. *Dashboard khi áp dụng bộ lọc native – filter tab5 bang seller SP (tệp filter_tab5_bang_seller_SP.png)*



Hình 7.20. *Dashboard khi áp dụng bộ lọc native – filter tab6 pttt boleto (tệp filter_tab6_pttt_boleto.png)*

Gợi ý quyết định (luật + ngưỡng + hành động)

Search 7 records...

Ưu tiên	Mức độ	Chủ đề	Tín hiệu	Giá trị	Chi tiết	Ngưỡng	Hành động gợi ý
1	CAO	Seller	Seller doanh thu top 10% vi phạm ≥2 tiêu chí vận hành	75.00	75/153 seller trong watchlist, doanh thu watchlist 4,400,613 BRL	review < 4 | giao > 15 ngày | trễ > 10% (≥2 điều kiện)	Gửi cảnh cáo, giảm hiển thị, yêu cầu kế hoạch SLA 30 ngày
2	CAO	Giao hàng	Đơn trễ kéo review xuống	1.73	Review đúng hạn 4.29 vs trễ 2.56 (7655 đơn trễ)	chênh lệch > 1 sao	Cảnh báo seller khi đơn trễ 1–3 ngày (review 3,29) trước mốc 4–7 ngày (2,10)
3	CAO	Vận hành theo tháng	Tháng có tỷ lệ giao trễ > 15%	2.00	2018-03 (21.36%), 2018-02 (15.99%)	late_delivery_pct > 15%	Tăng năng lực vận chuyển trước cao điểm (T11 Black Friday, T2–T3)
4	TRUNG	Địa lý	Bang trễ	9.00	AL 23.93%/3.76, MA 19.67%/3.78,	late_delivery_pct > 10%	Seller/kho vùng;

Hình 7.21. *Khu vực gợi ý quyết định trên dashboard – watchlist goi y quyet dinh (tệp watchlist_goi_y_quyet_dinh.png)*

Seller watchlist – cảnh báo vận hành (153 seller)

Search 153 records...

Seller	Bang seller	Mức cảnh báo	Số vi phạm	Lý do	Số đơn	Doanh thu (BRL)	Review TB	Ngày giao TB	% giao trễ	Số tháng hoạt động	Hành động đề xuất
04308b1e...	SC	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	94	60,131	3.88	18.9	11.96	9	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
522620dc...	PR	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	174	57,168	3.9	15.2	10.11	20	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
81602554...	SP	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	385	47,018	3.83	16.7	17.92	11	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
2eb70248...	SP	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	202	42,629	2.72	17.9	14.21	6	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
712e6ed8...	SC	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	79	39,815	3.45	23.7	21.18	14	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
c3867b46...	SP	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	240	36,486	3.88	15.1	11.2	16	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
855668e0...	SP	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	310	31,918	3.77	16.1	13.83	16	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
88460e8e...	PR	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	247	31,397	3.36	18.2	19.67	9	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
b1b39487...	SP	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	16	22,249	1.81	23.4	64.29	3	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày

Hình 7.22. *Khu vực gợi ý quyết định trên dashboard – watchlist seller canh bao (tệp watchlist_seller_canh_bao.png)*

Không có ảnh minh chứng bổ sung nào khác trong `evidence/` và `docs/` tại thời điểm sinh báo cáo.

7.4. Đầu ra cho giai đoạn Decision/Action

Giai đoạn Decision/Action chỉ cần hai truy vấn để lấy toàn bộ đầu vào:

```
-- 7 luật gợi ý quyết định, sắp theo mức ưu tiên
SELECT priority, severity, topic, signal, metric_value, threshold, action, source
FROM   presentation.v_decision_hints ORDER BY priority;

-- Người bán mức ĐỎ (vi phạm >= 2 tiêu chí), doanh thu lớn nhất trước
SELECT seller_id, seller_state, revenue, violations, reasons, recommended_action
FROM   presentation.v_seller_watchlist
WHERE  alert_level = 'ĐỎ' ORDER BY revenue DESC LIMIT 20;
```

Kết quả là dữ liệu có cấu trúc (luật, giá trị hiện tại, ngưỡng, hành động; mã người bán, lý do, hành động khuyến nghị) mà một rule engine có thể chuyển thành email/Telegram cảnh báo, hoặc kích hoạt chăm sóc khách hàng khi đơn có nguy cơ trễ ≥ 4 ngày. Dashboard phục vụ con người, view phục vụ máy — hai kênh dùng chung một tầng ngữ nghĩa nên không mâu thuẫn số liệu.

8. Thảo luận

8.1. Điểm mạnh

- **Ranh giới rõ ràng, data contract tường minh:** nhóm phục vụ chứ không tính lại phân tích của Analytics; mọi con số truy ngược được về một cột Parquet và được đối chiếu tự động. Nhóm trước đổi thuật toán, nhóm sau chỉ cần nạp lại.
- **Tách tầng vật lý và ngữ nghĩa:** `analytics/gold` phản ánh Parquet, `presentation` phản ánh nghiệp vụ; định nghĩa KPI nằm ở một nơi, dùng chung cho dashboard và Decision/Action.
- **Độ đo không cộng dồn và lọc chéo nhất quán:** AOV, % trễ, review trên thẻ KPI tính lại từ `v_orders` thay vì trung bình của tỷ lệ tháng; kiểm tra nhất quán chéo chứng minh tổng từ gold bằng đúng tổng của Analytics.
- **Đầu ra máy đọc được:** `v_decision_hints` chứa luật, ngưỡng, giá trị và hành động — giai đoạn sau dùng ngay bằng SQL.
- **Tái lập bằng mã:** Docker Compose, DDL tự sinh, dashboard dựng qua REST API, ảnh chụp tự động, báo cáo này cũng sinh bằng script.
- **Chi phí gần bằng không khi không dùng:** scale-to-zero cộng cache 10 phút của Superset.

8.2. Hạn chế

- **Insight phân tích không lọc chéo được:** Pareto, RFM, K-Means, watchlist là kết quả toàn kỳ của Spark; lọc theo tháng/bang đòi hỏi tính lại phân tích, vượt phạm vi nhóm. Phạm vi từng bộ lọc phải được cấu hình và giải thích cẩn thận.
- **Chi phí lớp gold:** 7 bảng gold chiếm khoảng 79% thời gian nạp chỉ để phục vụ lọc chéo cho một số thẻ KPI và xu hướng.
- **Thời gian nạp phụ thuộc đường truyền client:** thông lượng COPY (~0,32 MB/s) bị giới hạn bởi băng thông tải lên, không phải bởi Neon.
- **Quy mô vừa phải:** 505.592 dòng chưa đủ để thấy giới hạn của Postgres; nhận định về mở rộng dựa trên lập luận kiến trúc và số đo hiện có, chưa phải tải thật của hàng nghìn người dùng.
- **Full refresh, chưa nạp tăng dần;** metadata Superset dùng SQLite và cache filesystem, chưa phù hợp chạy nhiều bản Superset; bảo mật mức cơ bản (chưa row-

level security, tài khoản quản trị dùng cho demo). **[[CẬP NHẬT: bổ sung hạn chế thực tế khác nếu có]]**

8.3. Khả năng mở rộng

Khi dữ liệu nguồn tăng 10–100 lần (1–10 triệu đơn):

- **Bảng Analytics tăng không đều:** bảng theo tháng, danh mục, bang, band khoảng cách gần như không đổi; chỉ bảng cấp khách hàng và người bán tăng tuyến tính (10 triệu đơn → `customer_segments` cỡ 10 triệu dòng), vẫn trong khả năng PostgreSQL với chỉ mục. Đây là lợi ích cốt lõi của việc đặt tầng phục vụ *sau* tầng xử lý phân tán: khối lượng đã được Spark cô đọng. Lớp gold chi tiết thì tăng tuyến tính và là ứng viên đầu tiên cần thay bằng bảng tổng hợp nhẹ hoặc materialized view.
- **Nạp dữ liệu:** nút thắt hiện tại là mạng client → Neon. Bước đầu là chạy loader cùng vùng mạng với Neon (máy ảo AWS Singapore hoặc job CI), nơi `COPY` thường đạt hàng chục đến hàng trăm nghìn dòng/giây; sau đó nạp song song nhiều bảng, nạp tăng dần theo phân vùng tháng, và với bảng rất lớn thì dùng declarative partitioning, nạp vào bảng tạm rồi hoán đổi để không khoá người đọc.
- **Truy vấn:** Neon autoscaling tăng tài nguyên theo chiều dọc; *read replica* tách tải BI khỏi tải nạp; view nặng chuyển thành *materialized view* làm mới sau mỗi lần nạp.
- **Superset:** mở rộng theo chiều ngang — nhiều web server sau cân bằng tải, metadata PostgreSQL, cache Redis, Celery worker cho truy vấn bất đồng bộ và báo cáo định kỳ.
- **Khi nào đổi công nghệ?** Nếu phải lọc chéo tự do trên fact hàng tỷ dòng, kho OLAP cột (ClickHouse, Druid, Pinot) hoặc Trino trên Iceberg/Delta phù hợp hơn. Nhờ SQLAlchemy và tầng ngữ nghĩa SQL chuẩn, việc thay tầng phục vụ chủ yếu là chuyển các view, không làm lại dashboard.

8.4. Bài học kỹ thuật và hướng cải tiến

Bài học: (1) tầng trình bày không chỉ "vẽ biểu đồ" — rủi ro lớn nhất nằm ở ngữ nghĩa số liệu (grain, độ đo không cộng dồn, phạm vi thời gian), nên data contract và đối chiếu tự động là bắt buộc; (2) với dịch vụ serverless, độ trễ người dùng cảm nhận chủ yếu đến từ mạng và cold start, cache ở tầng BI có tác dụng lớn hơn tối ưu SQL ở quy mô này; (3) PgBouncer chế độ transaction hợp với truy vấn ngắn của BI nhưng

không hợp với DDL/COPY — phải phân biệt endpoint pooled và direct; (4) dashboard-as-code tốn công ban đầu nhưng trả lại bằng khả năng tái lập và quản lý phiên bản.

Hướng cải tiến:

- Đề xuất Analytics bổ sung cột tử số/mẫu số (đơn đã giao, đơn trễ, đơn có review) và bảng tổng hợp tháng × bang × danh mục × PTTT, để bỏ lớp gold chi tiết mà vẫn lọc chéo được.
- Nạp tăng dần theo `run_id`/phân vùng tháng, lưu lịch sử nhiều lần chạy để so sánh phiên bản phân tích.
- Điều phối toàn pipeline bằng Airflow/Dagster (Analytics xong → nạp → kiểm tra → làm mới view → làm nóng cache); quản lý tầng ngữ nghĩa bằng dbt.
- Bật Alerts & Reports của Superset (Celery + Redis) để gửi cảnh báo khi KPI vượt ngưỡng — cầu nối tự nhiên sang Decision/Action; thêm row-level security, role chỉ đọc, SSO.

9. Kết luận

Đồ án đã thực hiện giai đoạn **Analytics → Presentation** của pipeline dữ liệu lớn trên bộ dữ liệu Olist. Đối chiếu với mục tiêu ở Chương 1:

- **Nạp đúng và đủ:** 23 bảng (16 Analytics + 7 gold, 505.592 dòng) được nạp vào Neon bằng loader tự sinh DDL và `COPY`, có `load_log`; 23/23 bảng khớp số dòng, 19/19 kiểm tra số neo/nhất quán đạt.
- **Tầng ngữ nghĩa:** 22 view và bảng `dim_state` định nghĩa KPI thống nhất, xử lý đúng độ đo không cộng dồn qua lớp cấp đơn, bổ sung cột trình bày và mã hoá 7 luật gợi ý quyết định.
- **Dashboard:** Superset nhiều tab cho năm nhóm người dùng, có thẻ KPI, xu hướng, Pareto, bản đồ Brazil, bảng chi tiết, bộ lọc native và khu vực gợi ý quyết định.
- **Đo lường:** nạp toàn bộ trong 351,34 giây (16 bảng Analytics trong 72,66 giây), nút thắt là băng thông của client; 21 truy vấn dashboard có trung vị 173,4 ms phía client nhưng chỉ 1,25 ms thực thi trên server — mạng chiếm phần lớn độ trễ, còn bảng tổng hợp sẵn nhanh hơn hơn 250 lần so với tính lại từ fact.
- **Tái lập:** Docker Compose, mã nguồn tại https://github.com/viettuong287/BigDataProject, dashboard dựng bằng REST API, báo cáo sinh bằng script.

Dashboard truyền tải năm thông điệp chính: (1) doanh thu đỉnh không đồng nghĩa vận hành khoẻ (11/2017, 03/2018 trễ cao, review thấp); (2) giao trễ làm mất ~1,7 sao, sau 4 ngày trễ phần lớn khách chấm 1 sao; (3) địa lý quyết định trải nghiệm — SP gần người bán nên nhanh và hài lòng; (4) 97% khách chỉ mua một lần, CRM nên tập trung vào 3% khách mua lại và khách mới; (5) ~18% người bán tạo 80% doanh thu, 153 người bán lớn nhưng vận hành kém là rủi ro lớn nhất phía cung.

Khả năng tích hợp: với giai đoạn trước, nhóm giao tiếp qua data contract trên Parquet — Analytics chạy lại thì chỉ cần chạy lại loader. Với giai đoạn sau, `v_decision_hints` và `v_seller_watchlist` là giao diện máy đọc được để kích hoạt cảnh báo, thông báo hoặc điều chỉnh nghiệp vụ mà không phụ thuộc dashboard. Cùng các nhóm khác, đồ án góp phần hình thành pipeline hoàn chỉnh từ dữ liệu thô đến hành động.

Tài liệu tham khảo

[1] Olist, "Brazilian E-Commerce Public Dataset by Olist," Kaggle, 2018. [Trực tuyến]. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

[2] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.

[3] M. Zaharia et al., "Apache Spark: A Unified Engine for Big Data Processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[4] M. Armbrust et al., "Spark SQL: Relational Data Processing in Spark," in *Proc. ACM SIGMOD*, 2015, pp. 1383–1394.

[5] The Apache Software Foundation, "Apache Parquet – File Format Documentation." [Trực tuyến]. https://parquet.apache.org/docs/

[6] S. Melnik et al., "Dremel: Interactive Analysis of Web-Scale Datasets," *Proc. VLDB Endowment*, vol. 3, no. 1–2, pp. 330–339, 2010.

[7] D. J. Abadi, S. R. Madden and N. Hachem, "Column-Stores vs. Row-Stores: How Different Are They Really?," in *Proc. ACM SIGMOD*, 2008, pp. 967–980.

[8] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications, 2015.

[9] M. Armbrust, A. Ghodsi, R. Xin and M. Zaharia, "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," in *Proc. CIDR*, 2021.

[10] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Wiley, 2013.

[11] S. Chaudhuri and U. Dayal, "An Overview of Data Warehousing and OLAP Technology," *ACM SIGMOD Record*, vol. 26, no. 1, pp. 65–74, 1997.

[12] Neon, "Neon architecture overview," Neon Documentation. [Trực tuyến]. https://neon.com/docs/introduction/architecture-overview

[13] Neon, "Autoscaling", "Scale to Zero" và "Connection pooling," Neon Documentation. [Trực tuyến]. https://neon.com/docs/introduction/autoscaling ; https://neon.com/docs/introduction/scale-to-zero ; https://neon.com/docs/connect/connection-pooling

[14] The PostgreSQL Global Development Group, "PostgreSQL 18 Documentation – COPY." [Trực tuyến]. https://www.postgresql.org/docs/18/sql-copy.html

[15] The Apache Software Foundation, "Apache Superset Documentation." [Trực tuyến]. https://superset.apache.org/docs/intro

[16] B. Shneiderman, "The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations," in *Proc. IEEE Symposium on Visual Languages*, 1996, pp. 336–343.

[17] S. Few, *Information Dashboard Design: Displaying Data for At-a-Glance Monitoring*, 2nd ed. Analytics Press, 2013.

[18] P. S. Fader, B. G. S. Hardie and K. L. Lee, "RFM and CLV: Using Iso-Value Curves for Customer Base Analysis," *Journal of Marketing Research*, vol. 42, no. 4, pp. 415–430, 2005.

[19] The Apache Software Foundation, "Apache Arrow – Python bindings (pyarrow)." [Trực tuyến]. https://arrow.apache.org/docs/python/

[20] D. Varrazzo, "Psycopg 3 – Using COPY TO and COPY FROM." [Trực tuyến]. https://www.psycopg.org/psycopg3/docs/basic/copy.html

[21] Docker Inc., "Docker Compose Documentation." [Trực tuyến]. https://docs.docker.com/compose/

[22] Nhóm Storage → Processing, "big-data: pipeline làm sạch và tầng gold Olist," GitHub. [Trực tuyến]. https://github.com/NguyenAlpha/big-data

[23] Nhóm Processing → Analytics, "Analytics – Olist E-commerce: 16 bảng Parquet, báo cáo và tài liệu bàn giao," **[[CẬP NHẬT: URL repo Analytics]]**

[24] A. Verbitski et al., "Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases," in *Proc. ACM SIGMOD*, 2017, pp. 1041–1052.

[25] Bộ môn, "Yêu cầu đồ án học phần Nhập môn Dữ liệu lớn," tài liệu học phần, 2026.

Phụ lục A. Hướng dẫn chạy

A.1. Yêu cầu hệ thống

- Docker Desktop (Windows/macOS) hoặc Docker Engine + Compose v2 (Linux); khoảng 4 GB RAM trống cho Superset.
- Tài khoản Neon (gói miễn phí) và chuỗi kết nối tới database `olist_analytics`.
- Mã nguồn: `git clone https://github.com/viettuong287/BigDataProject`. Dữ liệu đầu vào đã nằm trong `data/analytics/` (sao chép từ repo Analytics); nếu thiếu, sao chép thư mục `data/analytics/` của repo Analytics vào cùng vị trí.
- Không cần cài Python trên máy host (mọi script chạy trong container `tools`).

A.2. Các bước chạy

```
# 1. Cấu hình
Copy-Item .env.example .env
#    điền NEON_DATABASE_URL (endpoint pooled, sslmode=require), SUPERSET_SECRET_KEY,
mật khẩu admin

# 2. Dựng image, khởi động Superset
docker compose build
docker compose up -d superset
docker compose ps                      # superset ở trạng thái healthy

# 3. loader/load_to_neon.py – Nạp toàn bộ output Analytics (Parquet) vào Neon Postgres.
docker compose run --rm tools python loader/load_to_neon.py

# 4. loader/apply_sql.py – Áp dụng lại các file sql/[0-8]*.sql (view tầng trình bày)
mà không nạp lại dữ liệu.
docker compose run --rm tools python loader/apply_sql.py

# 5. loader/validate.py – Kiểm chứng dữ liệu trong Neon:
docker compose run --rm tools python loader/validate.py

# 6. superset/build_dashboard.py – Dựng dashboard Superset từ code (idempotent):
database Neon -> dataset (view presentation.*)
docker compose run --rm tools python superset/build_dashboard.py

# 7. superset/import_dashboard.py – Phương án dự phòng: import dashboard từ
superset/export/dashboard_olist_presentation.zip.
docker compose run --rm tools python superset/import_dashboard.py

# 8. scripts/measure_latency.py – Đo độ trễ các truy vấn mà dashboard gửi xuống Neon
(qua endpoint pooled giống Superset).
docker compose run --rm tools python scripts/measure_latency.py

# 9. scripts/take_screenshots.py – Chụp ảnh dashboard Superset + đo thời gian tải
(cold vs cache) + đo độ trễ từng chart qua API.
docker compose --profile screenshots run --rm screenshots

# 10. scripts/neon_inventory.py – Kiểm kê đối tượng trong Neon (schema, bảng/view, số
dòng, dung lượng) từ catalog Postgres.
docker compose run --rm tools python scripts/neon_inventory.py

# 11. scripts/render_diagrams.py – Xuất các sơ đồ Mermaid trong docs/architecture.md
```

```
thành PNG (Chromium headless + mermaid.js từ CDN).
docker compose run --rm tools python scripts/render_diagrams.py

# 12. scripts/scale_experiment.py – Thí nghiệm mở rộng: nhân bản gold.fact_order_items
×N ngay trong Neon (không tốn băng thông upload),
docker compose run --rm tools python scripts/scale_experiment.py
```

Mở `http://localhost:8088`, đăng nhập bằng `SUPERSET_ADMIN_USER` / `SUPERSET_ADMIN_PASSWORD`. Dừng hệ thống: `docker compose down` (giữ volume metadata) — chỉ dùng `docker compose down -v` khi muốn xoá cả metadata Superset.

A.3. Sinh lại báo cáo này

```
# Cách 1: Python trên máy (đã cài Python 3.12)
python -m venv report\.venv
report\.venv\Scripts\python -m pip install -r report\requirements.txt
report\.venv\Scripts\python report\build_report.py

# Cách 2: không cài Python, dùng Docker
docker run --rm -v "${PWD}:/work" -v "D:\Analytics-main:/upstream:ro" -e
ANALYTICS_REPO=/upstream `
  -w /work python:3.12-slim sh -c "pip install -q -r report/requirements.txt && python
report/build_report.py"
```

Mở tệp `report/BaoCao_DoAn_BigData.docx` bằng Word, nhấn **Ctrl+A rồi F9** để cập nhật mục lục, danh mục hình và danh mục bảng (Word cũng hỏi cập nhật trường khi mở tệp). Ảnh đặt vào `evidence/` hoặc `docs/` sẽ được tự động chèn vào đúng chương theo từ khoá trong tên tệp (`dashboard`, `filter`, `neon`, `load`, `latency`, `sql`, `watchlist`, `architecture`…); chú thích có thể ghi đè trong `report/content/captions.json`.

A.4. Xử lý sự cố thường gặp

Bảng A.1. Sự cố thường gặp khi chạy lại hệ thống

Hiện tượng	Cách xử lý
`docker` không được nhận trong PowerShell	Thêm `%LOCALAPPDATA%\Programs\DockerDesktop\resources\bin` vào `PATH` hoặc mở lại terminal sau khi khởi động Docker Desktop
Loader báo "Thiếu NEON_DATABASE_URL"	Kiểm tra tệp `.env` ở thư mục gốc và tên biến
Kết nối Neon timeout lần đầu	Compute đang ngủ (scale-to-zero); chạy lại hoặc tăng `connect_timeout`
Superset không lên (unhealthy)	`docker compose logs superset`; kiểm tra `SUPERSET_SECRET_KEY` đã đặt
Dashboard trống sau khi nạp lại	Chạy lại script tạo view (view bị xoá theo `DROP ... CASCADE`)

Phụ lục B. Cấu hình

Các tệp cấu hình dưới đây được trích nguyên văn từ repo tại thời điểm sinh báo cáo. Tệp `.env` chứa bí mật nên **không** được đưa vào báo cáo; chỉ trích `.env.example`.

Tệp `.env.example`:

```
# Sao chép thành .env rồi điền giá trị thật. KHÔNG commit .env.

# Chuỗi kết nối Neon (endpoint pooled, có "-pooler" trong hostname).
# Loader tự bỏ "-pooler" để dùng endpoint direct cho COPY/DDL.
NEON_DATABASE_URL=postgresql://<user>:<password>@<endpoint>-
pooler.<region>.aws.neon.tech/olist_analytics?sslmode=require
NEON_PROJECT_ID=<neon-project-id>

# Superset
SUPERSET_SECRET_KEY=<chuoi-ngau-nhien-dai>
SUPERSET_ADMIN_USER=admin
SUPERSET_ADMIN_PASSWORD=admin
```

Tệp `docker-compose.yml`:

```
name: olist-presentation

services:
  # Postgres cục bộ chỉ lưu METADATA của Superset (user, chart, dashboard, log).
  # Dữ liệu nghiệp vụ nằm trên Neon, không nằm ở đây.
  superset-db:
    image: postgres:16-alpine
    container_name: olist-superset-db
    environment:
      POSTGRES_DB: superset
      POSTGRES_USER: superset
      POSTGRES_PASSWORD: superset
    volumes:
      - superset_db:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U superset -d superset"]
      interval: 5s
      timeout: 5s
      retries: 20
    restart: unless-stopped

  # Apache Superset: tầng trình bày (dashboard).
  superset:
    build: ./superset
    image: olist-superset:4.1.2
    container_name: olist-superset
    env_file: .env
    environment:
      SUPERSET_METADATA_URI: postgresql+psycopg2://superset:superset@superset-
db:5432/superset
      SERVER_WORKER_AMOUNT: 4
      SERVER_THREADS_AMOUNT: 8
      GUNICORN_TIMEOUT: 120
    depends_on:
      superset-db:
        condition: service_healthy
    ports:
      - "8088:8088"
```

```
    volumes:
      - ./superset/superset_config.py:/app/pythonpath/superset_config.py:ro
      - superset_home:/app/superset_home
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://localhost:8088/health"]
      interval: 15s
      timeout: 5s
      retries: 20
    restart: unless-stopped

  # Container công cụ Python: loader, validate, build dashboard, đo latency.
  # Chạy theo yêu cầu: docker compose run --rm tools python loader/load_to_neon.py
  tools:
    build: ./loader
    image: olist-tools:latest
    profiles: ["tools"]
    env_file: .env
    environment:
      SUPERSET_URL: http://superset:8088
      TZ: Asia/Ho_Chi_Minh
    volumes:
      - .:/work
    working_dir: /work

  # Chụp ảnh dashboard tự động (Playwright + Chromium headless).
  screenshots:
    build:
      context: ./scripts
      dockerfile: Dockerfile.screenshots
    image: olist-screenshots:latest
    profiles: ["screenshots"]
    env_file: .env
    environment:
      SUPERSET_URL: http://superset:8088
    volumes:
      - .:/work
    working_dir: /work

volumes:
  superset_home:
  superset_db:
```

Tệp superset/superset_config.py:

```
"""Cấu hình Superset cho môi trường demo cục bộ (docker compose)."""
import os

SECRET_KEY = os.environ.get("SUPERSET_SECRET_KEY", "change-me-in-.env")

# Metadata (dashboard, chart, user, log) nằm trong container superset-db (Postgres cục
bộ).
# SQLite chỉ là phương án dự phòng: nó khoá ghi toàn cục nên các chart tải song song
bị xếp hàng.
# Dữ liệu nghiệp vụ nằm ở Neon Postgres, đăng ký qua REST API
(superset/build_dashboard.py).
SQLALCHEMY_DATABASE_URI = os.environ.get(
    "SUPERSET_METADATA_URI",
"sqlite:////app/superset_home/superset.db?check_same_thread=false"
)

FEATURE_FLAGS = {
    "DASHBOARD_NATIVE_FILTERS": True,
    "DASHBOARD_CROSS_FILTERS": True,
    "ENABLE_TEMPLATE_PROCESSING": True,
```

```
}

# Cache kết quả truy vấn chart 10 phút: lần mở dashboard thứ 2 không phải hỏi lại Neon.
DATA_CACHE_CONFIG = {
    "CACHE_TYPE": "FileSystemCache",
    "CACHE_DIR": "/app/superset_home/cache",
    "CACHE_DEFAULT_TIMEOUT": 600,
}

TALISMAN_ENABLED = False
WTF_CSRF_TIME_LIMIT = None
ROW_LIMIT = 50000
BABEL_DEFAULT_LOCALE = "en"
```

Tệp `superset/Dockerfile`:

```
FROM apache/superset:4.1.2

USER root
# psycopg2: driver để Superset kết nối Neon Postgres.
RUN pip install --no-cache-dir psycopg2-binary==2.9.10
COPY docker-init.sh /app/docker-init.sh
RUN sed -i 's/\r$//' /app/docker-init.sh && chmod +x /app/docker-init.sh
USER superset

CMD ["/app/docker-init.sh"]
```

Tệp `loader/Dockerfile`:

```
# Image công cụ Python (loader, validate, build dashboard, đo latency).
# Máy Windows không cần cài Python: mọi script chạy trong container này.
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /work
COPY requirements.txt /tmp/requirements.txt
RUN pip install -r /tmp/requirements.txt

CMD ["python", "--version"]
```

Tệp `loader/requirements.txt`:

```
pyarrow==17.0.0
pandas==2.2.3
SQLAlchemy==2.0.36
psycopg[binary]==3.2.3
requests==2.32.3
tabulate==0.9.0
```

Tệp `loader/load_to_neon.py`:

```
"""Nạp toàn bộ output Analytics (Parquet) vào Neon Postgres.

Luồng: Parquet (data/analytics, data/gold) -> pyarrow -> CSV trong bộ nhớ -> COPY FROM
STDIN
-> khoá chính / index -> ANALYZE -> ghi analytics.load_log + evidence/load_log.csv
-> áp dụng các view tầng trình bày (sql/*.sql).
```

```
Chạy:  docker compose run --rm tools python loader/load_to_neon.py [--only
kpi_monthly,...] [--skip-views]
"""
from __future__ import annotations

import argparse
import csv
import io
import time
import uuid
from datetime import datetime
from pathlib import Path

import pyarrow as pa
import pyarrow.compute as pc
import pyarrow.csv as pacsv
import pyarrow.dataset as ds
from psycopg import sql

from common import DATA_DIR, EVIDENCE_DIR, SQL_DIR, Tee, connect, mask, neon_url

# Khoá chính + index phụ cho từng bảng. Grain lấy từ ANALYTICS_HANDOVER.md /
HANDOVER.md.
TABLES: dict[str, dict] = {
    # ---- Output Analytics (16 bảng) ----
    "analytics.kpi_monthly": {"pk": ["purchase_year", "purchase_month"], "grain": "1
tháng (2017-01→2018-08)"},
    "analytics.category_pareto": {"pk": ["category"], "grain": "1 danh mục"},
    "analytics.category_seasonality": {"pk": ["category", "purchase_month"], "grain":
"1 danh mục × 1 tháng năm 2017"},
    "analytics.state_logistics": {"pk": ["customer_state"], "grain": "1 bang khách
hàng"},
    "analytics.insight_distance": {"pk": ["distance_band"], "grain": "1 band khoảng
cách"},
    "analytics.rfm_scores": {"pk": ["customer_unique_id"], "idx": [["rfm_segment"],
["customer_state"]], "grain": "1 khách thật"},
    "analytics.repeat_profile": {"pk": ["cohort"], "grain": "repeat vs one_shot"},
    "analytics.repeat_categories": {"pk": ["category"], "grain": "1 danh mục của khách
lặp"},
    "analytics.repeat_states": {"pk": ["customer_state"], "grain": "1 bang của khách
lặp"},
    "analytics.customer_segments": {"pk": ["customer_unique_id"], "idx": [["cluster"],
["rfm_segment"], ["customer_state"]], "grain": "1 khách thật + cụm K-Means"},
    "analytics.cluster_profile": {"pk": ["cluster"], "grain": "1 cụm K-Means"},
    "analytics.seller_pareto": {"pk": ["seller_id"], "idx": [["seller_state"]],
"grain": "1 seller"},
    "analytics.seller_watchlist": {"pk": ["seller_id"], "grain": "seller top 10% doanh
thu có vận hành kém"},
    "analytics.insight_late_vs_review": {"pk": ["delivery"], "grain": "late vs
on_time"},
    "analytics.delay_vs_review": {"pk": ["delay_band"], "grain": "1 band ngày trễ"},
    "analytics.payment_mix": {"pk": ["main_payment_type", "installment_band"], "grain":
"PTTT × band trả góp"},
    # ---- Gold (đầu vào của Analytics, dùng cho bộ lọc chi tiết trên dashboard) ----
    "gold.fact_orders": {"pk": ["order_id"], "idx": [["purchase_date"],
["customer_state"], ["main_payment_type"]], "grain": "1 đơn hàng"},
    "gold.fact_order_items": {"pk": ["order_id", "order_item_id"], "idx":
[["purchase_date"], ["category"], ["customer_state"], ["seller_id"]], "grain": "1 sản
phẩm trong 1 đơn"},
    "gold.agg_sales_monthly": {"pk": ["purchase_year", "purchase_month"], "grain": "1
tháng (gồm 2016, 2018-09)"},
    "gold.agg_category_monthly": {"pk": ["purchase_year", "purchase_month",
"category"], "grain": "1 danh mục × 1 tháng"},
    "gold.agg_customer_state": {"pk": ["customer_state"], "grain": "1 bang khách
hàng"},
```

```
    "gold.agg_seller_performance": {"pk": ["seller_id"], "grain": "1 seller"},
    "gold.agg_customer_rfm": {"pk": ["customer_unique_id"], "grain": "1 khách thật"},
}

LOAD_LOG_DDL = """
CREATE TABLE IF NOT EXISTS analytics.load_log (
    id              bigserial PRIMARY KEY,
    run_id          text        NOT NULL,
    schema_name     text        NOT NULL,
    table_name      text        NOT NULL,
    rows            bigint      NOT NULL,
    parquet_rows    bigint      NOT NULL,
    row_count_ok    boolean     NOT NULL,
    parquet_files   int         NOT NULL,
    parquet_bytes   bigint      NOT NULL,
    csv_bytes       bigint      NOT NULL,
    read_seconds    numeric(10,3),
    copy_seconds    numeric(10,3),
    index_seconds   numeric(10,3),
    seconds         numeric(10,3) NOT NULL,
    rows_per_sec    numeric(14,1),
    source_path     text,
    loaded_at       timestamptz NOT NULL DEFAULT now()
);
"""


def pg_type(t: pa.DataType) -> str:
    if pa.types.is_string(t) or pa.types.is_large_string(t) or
pa.types.is_dictionary(t):
        return "text"
    if pa.types.is_boolean(t):
        return "boolean"
    if pa.types.is_int8(t) or pa.types.is_int16(t):
        return "smallint"
    if pa.types.is_int32(t):
        return "integer"
    if pa.types.is_int64(t):
        return "bigint"
    if pa.types.is_float32(t):
        return "real"
    if pa.types.is_float64(t):
        return "double precision"
    if pa.types.is_decimal(t):
        return f"numeric({t.precision},{t.scale})"
    if pa.types.is_date(t):
        return "date"
    if pa.types.is_timestamp(t):
        return "timestamptz" if t.tz else "timestamp"
    raise TypeError(f"Chưa ánh xạ kiểu Arrow {t}")


def read_table(path: Path) -> tuple[pa.Table, int, int]:
    """Đọc 1 thư mục bảng Spark (có thể phân vùng hive year/month). Trả về (table, số
file, bytes)."""
    dataset = ds.dataset(path, format="parquet", partitioning="hive")
    files = dataset.files
    nbytes = sum(Path(f).stat().st_size for f in files)
    table = dataset.to_table()
    # Postgres chỉ giữ tới micro giây; đổi timestamp[ns] -> timestamp[us] để CSV hợp
lệ.
    cols = []
    for field, col in zip(table.schema, table.columns):
        if pa.types.is_timestamp(field.type) and field.type.unit == "ns":
            col = pc.cast(col, pa.timestamp("us", field.type.tz))
```

```
        if pa.types.is_dictionary(field.type):
            col = pc.cast(col, pa.string())
        cols.append(col)
    table = pa.table(cols, names=table.schema.names)
    return table, len(files), nbytes


... (cắt, tổng 262 dòng – xem loader/load_to_neon.py)
```

Phụ lục C. Schema dữ liệu

C.1. Ánh xạ kiểu và ví dụ bản ghi

Bảng C.1. Ánh xạ kiểu dữ liệu Parquet → PostgreSQL do loader áp dụng

Kiểu Arrow/Parquet	Kiểu PostgreSQL	Lý do	Số bảng dùng
int32	INTEGER	Số nguyên 4 byte (tháng, điểm, rank)	8
int64	BIGINT	Đếm số lượng lớn (đơn, khách, món)	16
decimal128(p, s)	NUMERIC(p,s)	Tiền BRL: giữ chính xác 2 chữ số thập phân, không sai số dấu phẩy động	16
double	DOUBLE PRECISION	Giá trị trung bình/tỷ lệ đã làm tròn	10
bool	BOOLEAN	Cờ (vd. tháng Black Friday)	1
string	TEXT	Chuỗi định danh/nhãn, không giới hạn độ dài	14
date32[day]	DATE	Ngày mua đầu/cuối	2

Ví dụ bản ghi thực tế đọc từ Parquet (một dòng của ba bảng tiêu biểu):

```
-- kpi_monthly (1 dòng mẫu)
{
  "purchase_year": 2017,
  "purchase_month": 11,
  "orders": 7421,
  "customers": 7313,
  "items": 8626,
  "revenue": "1003862.14",
  "freight": "168329.54",
  "gmv": "1172191.68",
  "avg_order_value": "157.96",
  "avg_review_score": 3.95,
  "avg_delivery_days": 15.1,
  "late_delivery_pct": 14.31,
  "revenue_mom_pct": "52.06",
  "orders_mom_pct": "63.21",
  "revenue_yoy_pct": "None",
  "revenue_rank": 1,
  "is_black_friday_month": "True"
}

-- customer_segments (1 dòng mẫu)
{
  "customer_unique_id": "317cfc692e3f86c45c95697c61c853a6",
  "customer_state": "SP",
  "first_purchase_date": "2018-08-25",
  "last_purchase_date": "2018-08-25",
  "recency_days": 53,
  "frequency": 1,
  "monetary": "9.59",
  "avg_order_value": "9.59",
  "r_score": 5,
  "f_score": 5,
  "m_score": 1,
  "rfm_segment": "One-shot Recent",
  "cluster": 0
}

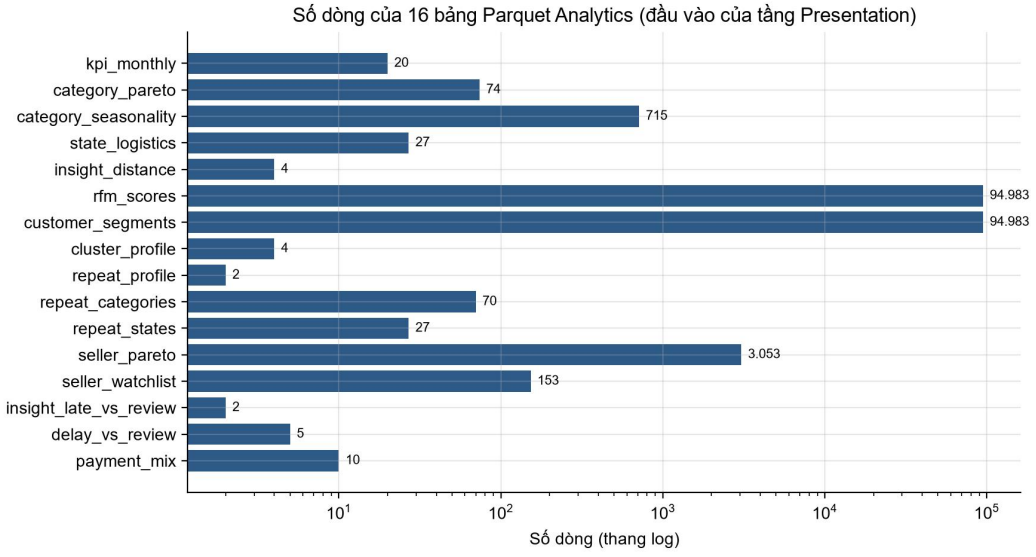
-- seller_watchlist (1 dòng mẫu)
{
```

```
  "seller_id": "4869f7a5dfa277a7dca6462dcf3b52b2",
  "seller_state": "SP",
  "orders": 1131,
  "revenue": "229237.63",
  "avg_review_score": 4.13,
  "avg_delivery_days": 14.9,
  "late_delivery_pct": 11.59,
  "avg_distance_km": 610.0,
  "active_months": 17,
  "revenue_pctile": 1.0
}
```


C.2. Quy mô các bảng đầu vào

Bảng C.2. Quy mô thực tế của các bảng Parquet Analytics (đọc trực tiếp bằng pyarrow khi sinh báo cáo)

Bảng	Số dòng	Số file Parquet	Dung lượng (KB, snappy)
kpi_monthly	20	1	6,5
category_pareto	74	1	7,0
category_seasonality	715	1	12,4
state_logistics	27	1	5,7
insight_distance	4	1	2,2
rfm_scores	94.983	1	3.894,5
customer_segments	94.983	1	3.915,6
cluster_profile	4	1	2,6
repeat_profile	2	1	2,1
repeat_categories	70	1	3,3
repeat_states	27	1	2,3
seller_pareto	3.053	1	159,9
seller_watchlist	153	1	13,5
insight_late_vs_review	2	1	1,5
delay_vs_review	5	1	1,9
payment_mix	10	1	1,9
Tổng	**194.132**	16	**8.033,0**



Hình C.1. *Số dòng của từng bảng Analytics (thang log): hai bảng cấp khách hàng chiếm gần như toàn bộ khối lượng*

Bảng C.3. Các bảng gold được nạp bổ sung vào schema gold (đầu vào chi tiết cho bộ lọc chéo)

Bảng	1 dòng =	Số dòng	Số cột	Số file	MB
gold.agg_category_monthly	1 danh mục x 1 tháng	1.282	9	1	0,03
gold.agg_customer_rfm	1 khách hàng thật	94.983	9	1	4,32
gold.agg_customer_state	1 bang của khách hàng	27	11	1	0,01
gold.agg_sales_monthly	1 tháng	24	12	1	0,01
gold.agg_seller_performance	1 người bán	3.053	15	1	0,20
gold.fact_order_items	1 sản phẩm trong 1 đơn hàng	112.650	27	24	16,93
gold.fact_orders	1 đơn hàng	99.441	29	25	13,17
Tổng		**311.460**		54	34,65

C.3. Schema 16 bảng Analytics

Schema chi tiết của 16 bảng Parquet đầu vào và kiểu PostgreSQL tương ứng trong schema `analytics` (sinh tự động bằng pyarrow khi tạo báo cáo).

Bảng C.4. Schema bảng `kpi_monthly` (20 dòng; 1 dòng = 1 tháng (2017-01 → 2018-08))

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
purchase_year	int32	INTEGER
purchase_month	int32	INTEGER
orders	int64	BIGINT
customers	int64	BIGINT
items	int64	BIGINT
revenue	decimal128(32, 2)	NUMERIC(32,2)
freight	decimal128(32, 2)	NUMERIC(32,2)
gmv	decimal128(33, 2)	NUMERIC(33,2)
avg_order_value	decimal128(24, 2)	NUMERIC(24,2)
avg_review_score	double	DOUBLE PRECISION
avg_delivery_days	double	DOUBLE PRECISION
late_delivery_pct	double	DOUBLE PRECISION
revenue_mom_pct	decimal128(35, 2)	NUMERIC(35,2)
orders_mom_pct	decimal128(27, 2)	NUMERIC(27,2)
revenue_yoy_pct	decimal128(35, 2)	NUMERIC(35,2)
revenue_rank	int32	INTEGER
is_black_friday_month	bool	BOOLEAN

Bảng C.5. Schema bảng `category_pareto` (74 dòng; 1 dòng = 1 danh mục)

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
category	string	TEXT
orders	int64	BIGINT
items	int64	BIGINT
revenue	decimal128(32, 2)	NUMERIC(32,2)
freight	decimal128(32, 2)	NUMERIC(32,2)
avg_price	decimal128(33, 2)	NUMERIC(33,2)
freight_to_revenue_pct	decimal128(35, 2)	NUMERIC(35,2)
revenue_share_pct	decimal128(35, 2)	NUMERIC(35,2)
cumulative_share_pct	decimal128(35, 2)	NUMERIC(35,2)

Bảng C.6. Schema bảng `category_seasonality` (715 dòng; 1 dòng = 1 danh mục × 1 tháng (2017))

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
`category`	string	TEXT
`purchase_month`	int32	INTEGER
`revenue`	decimal128(32, 2)	NUMERIC(32,2)
`orders`	int64	BIGINT
`pct_of_category_year`	decimal128(35, 2)	NUMERIC(35,2)

Bảng C.7. Schema bảng `state_logistics` (27 dòng; 1 dòng = 1 bang của khách)

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
`customer_state`	string	TEXT
`orders`	int64	BIGINT
`customers`	int64	BIGINT
`revenue`	decimal128(32, 2)	NUMERIC(32,2)
`gmv`	decimal128(33, 2)	NUMERIC(33,2)
`avg_order_value`	decimal128(24, 2)	NUMERIC(24,2)
`avg_freight`	decimal128(23, 2)	NUMERIC(23,2)
`freight_pct_of_revenue`	decimal128(35, 2)	NUMERIC(35,2)
`avg_delivery_days`	double	DOUBLE PRECISION
`late_delivery_pct`	double	DOUBLE PRECISION
`avg_review_score`	double	DOUBLE PRECISION
`revenue_share_pct`	decimal128(35, 2)	NUMERIC(35,2)
`revenue_rank`	int32	INTEGER

Bảng C.8. Schema bảng `insight_distance` (4 dòng; 1 dòng = 1 khoảng cách (band km))

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
`distance_band`	string	TEXT
`band_order`	int32	INTEGER
`items`	int64	BIGINT
`avg_freight`	decimal128(13, 2)	NUMERIC(13,2)
`avg_delivery_days`	double	DOUBLE PRECISION
`avg_delay_days`	double	DOUBLE PRECISION
`late_pct`	double	DOUBLE PRECISION

Bảng C.9. Schema bảng `rfm_scores` (94.983 dòng; 1 dòng = 1 khách thật)

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
`customer_unique_id`	string	TEXT
`customer_state`	string	TEXT
`first_purchase_date`	date32[day]	DATE
`last_purchase_date`	date32[day]	DATE
`recency_days`	int32	INTEGER
`frequency`	int64	BIGINT
`monetary`	decimal128(33, 2)	NUMERIC(33,2)
`avg_order_value`	decimal128(24, 2)	NUMERIC(24,2)
`r_score`	int32	INTEGER
`f_score`	int32	INTEGER
`m_score`	int32	INTEGER
`rfm_segment`	string	TEXT

Bảng C.10. Schema bảng `customer_segments` (94.983 dòng; 1 dòng = 1 khách thật (+ cụm K-Means))

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
customer_unique_id	string	TEXT
customer_state	string	TEXT
first_purchase_date	date32[day]	DATE
last_purchase_date	date32[day]	DATE
recency_days	int32	INTEGER
frequency	int64	BIGINT
monetary	decimal128(33, 2)	NUMERIC(33,2)
avg_order_value	decimal128(24, 2)	NUMERIC(24,2)
r_score	int32	INTEGER
f_score	int32	INTEGER
m_score	int32	INTEGER
rfm_segment	string	TEXT
cluster	int32	INTEGER

Bảng C.11. Schema bảng `cluster_profile` (4 dòng; 1 dòng = 1 cụm K-Means)

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
cluster	int32	INTEGER
customers	int64	BIGINT
avg_recency	double	DOUBLE PRECISION
avg_frequency	double	DOUBLE PRECISION
avg_monetary	decimal128(34, 2)	NUMERIC(34,2)
champions	int64	BIGINT
loyal	int64	BIGINT
at_risk	int64	BIGINT

Bảng C.12. Schema bảng `repeat_profile` (2 dòng; 1 dòng = 1 cohort (repeat / one_shot))

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
cohort	string	TEXT
customers	int64	BIGINT
pct	decimal128(27, 2)	NUMERIC(27,2)
avg_recency	double	DOUBLE PRECISION
avg_frequency	double	DOUBLE PRECISION
avg_monetary	decimal128(34, 2)	NUMERIC(34,2)
avg_aov	decimal128(25, 2)	NUMERIC(25,2)

Bảng C.13. Schema bảng `repeat_categories` (70 dòng; 1 dòng = 1 danh mục của khách mua lại)

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
category	string	TEXT
repeat_customers	int64	BIGINT
items	int64	BIGINT
revenue	decimal128(23, 2)	NUMERIC(23,2)

Bảng C.14. Schema bảng `repeat_states` (27 dòng; 1 dòng = 1 bang của khách mua lại)

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
customer_state	string	TEXT
repeat_customers	int64	BIGINT
avg_frequency	double	DOUBLE PRECISION
avg_monetary	decimal128(34, 2)	NUMERIC(34,2)

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
avg_aov	decimal128(25, 2)	NUMERIC(25,2)

Bảng C.15. Schema bảng `seller_pareto` (3.053 dòng; 1 dòng = 1 người bán)

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
seller_id	string	TEXT
seller_state	string	TEXT
orders	int64	BIGINT
revenue	decimal128(22, 2)	NUMERIC(22,2)
avg_review_score	double	DOUBLE PRECISION
avg_delivery_days	double	DOUBLE PRECISION
late_delivery_pct	double	DOUBLE PRECISION
avg_distance_km	double	DOUBLE PRECISION
active_months	int64	BIGINT
revenue_share_pct	decimal128(29, 2)	NUMERIC(29,2)
cumulative_share_pct	decimal128(35, 2)	NUMERIC(35,2)

Bảng C.16. Schema bảng `seller_watchlist` (153 dòng; 1 dòng = 1 người bán cần theo dõi)

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
seller_id	string	TEXT
seller_state	string	TEXT
orders	int64	BIGINT
revenue	decimal128(22, 2)	NUMERIC(22,2)
avg_review_score	double	DOUBLE PRECISION
avg_delivery_days	double	DOUBLE PRECISION
late_delivery_pct	double	DOUBLE PRECISION
avg_distance_km	double	DOUBLE PRECISION
active_months	int64	BIGINT
revenue_pctile	double	DOUBLE PRECISION

Bảng C.17. Schema bảng `insight_late_vs_review` (2 dòng; 1 dòng = 1 nhóm (late / on_time))

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
delivery	string	TEXT
orders	int64	BIGINT
avg_review	double	DOUBLE PRECISION
pct_score_1_2	decimal128(8, 2)	NUMERIC(8,2)
pct_has_comment	decimal128(8, 2)	NUMERIC(8,2)

Bảng C.18. Schema bảng `delay_vs_review` (5 dòng; 1 dòng = 1 khoảng ngày trễ)

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
delay_band	string	TEXT
band_order	int32	INTEGER
orders	int64	BIGINT
avg_review	double	DOUBLE PRECISION
pct_1_star	decimal128(8, 2)	NUMERIC(8,2)
pct_has_comment	decimal128(8, 2)	NUMERIC(8,2)

Bảng C.19. Schema bảng `payment_mix` (10 dòng; 1 dòng = 1 PTTT × 1 nhóm kỳ trả góp)

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
main_payment_type	string	TEXT
installment_band	string	TEXT
orders	int64	BIGINT

Cột	Kiểu Parquet (Arrow)	Kiểu PostgreSQL
gmv	decimal128(34, 2)	NUMERIC(34,2)
avg_order_value	decimal128(24, 2)	NUMERIC(24,2)

Phụ lục D. Tầng ngữ nghĩa: danh mục view và mã SQL

Bảng D.1. Các đối tượng của tầng ngữ nghĩa: 22 view và 1 bảng dimension (quét tự động từ `sql/10_presentation_views.sql`)

Đối tượng (schema presentation)	Loại	Nguồn	Mục đích
`dim_state`	bảng	–	Dimension 27 bang: tên, vùng, mã ISO `BR-xx` cho Country Map
`v_kpi_monthly`	view	`analytics.kpi_monthly`	Chuỗi KPI tháng; thêm `month_date` (DATE) và `month_label` cho bộ lọc thời gian
`v_orders`	view	`gold.fact_orders`, `presentation.dim_state`	Đơn hợp lệ cấp đơn 2017-01→2018-08: thẻ KPI lọc được theo tháng/bang/PTTT; `late_flag` cho % trễ chính xác
`v_kpi_overview`	view	`analytics.insight_late_vs_review`, `analytics.kpi_monthly`, `analytics.repeat_profile`, `analytics.rfm_scores`, `analytics.seller_pareto`, `analytics.seller_watchlist`, `gold.agg_sales_monthly`, `presentation.v_kpi_monthly`	1 dòng số neo toàn kỳ (doanh thu gold, tháng đỉnh, review trễ/đúng hạn, % khách lặp, watchlist)
`v_category_pareto`	view	`analytics.category_pareto`	Pareto danh mục + hạng + nhóm A/B/C
`v_category_seasonality`	view	`analytics.category_seasonality`, `presentation.v_category_pareto`	Heatmap danh mục × tháng năm 2017
`v_category_monthly`	view	`gold.agg_category_monthly`	Doanh thu danh mục theo tháng (từ gold) cho xu hướng có lọc danh mục
`v_sales_items`	view	`gold.fact_order_items`, `presentation.dim_state`	Dòng sản phẩm hợp lệ: doanh thu/ship theo danh mục × bang × tháng (không chứa review)
`v_state_logistics`	view	`analytics.state_logistics`, `presentation.dim_state`	Logistics theo bang, kèm tên bang, vùng, mã ISO
`v_insight_distance`	view	`analytics.insight_distance`	Phí ship, ngày giao, % trễ theo band khoảng cách
`v_rfm_customers`	view	`analytics.customer_segments`, `presentation.dim_state`	1 khách: điểm RFM, phân khúc, cohort, cụm K-Means có tên nghiệp vụ
`v_rfm_segments`	view	`presentation.v_rfm_customers`	Tổng hợp theo phân khúc RFM + hành động CRM gợi ý
`v_cluster_profile`	view	`analytics.cluster_profile`	Hồ sơ 4 cụm K-Means với tên nghiệp vụ
`v_repeat_profile`	view	`analytics.repeat_profile`	Khách mua lại vs mua một lần
`v_repeat_categories`	view	`analytics.repeat_categories`	Danh mục ưa thích của khách mua lại
`v_repeat_states`	view	`analytics.repeat_states`, `presentation.dim_state`	Bang của khách mua lại
`v_seller_pareto`	view	`analytics.seller_pareto`, `analytics.seller_watchlist`	Pareto người bán, nhóm A/B/C, cờ thuộc watchlist
`v_seller_watchlist`	view	`analytics.seller_watchlist`	Người bán cần theo dõi: số vi phạm, lý do, mức cảnh báo ĐỎ/VÀNG, hành động khuyến nghị
`v_late_vs_review`	view	`analytics.insight_late_vs_review`	Review đơn trễ vs đúng hạn (nhãn tiếng Việt)
`v_delay_vs_review`	view	`analytics.delay_vs_review`	Review và % 1 sao theo số ngày trễ
`v_payment_mix`	view	`analytics.payment_mix`	Cơ cấu PTTT × kỳ trả góp, % đơn
`v_decision_hints`	view	`analytics.category_pareto`, `analytics.delay_vs_review`, `analytics.insight_late_vs_review`, `analytics.seller_pareto`, `analytics.state_logistics`, `presentation.v_category_pareto`, `presentation.v_kpi_monthly`, `presentation.v_rfm_segments`, `presentation.v_seller_pareto`, `presentation.v_seller_watchlist`, `presentation.v_state_logistics`	7 luật gợi ý quyết định có ngưỡng, giá trị, hành động, nguồn – giao diện cho Decision/Action
`v_load_log`	view	`analytics.load_log`	Lần nạp gần nhất của mỗi bảng (lineage, độ tươi dữ liệu trên dashboard)

Tệp sql/10_presentation_views.sql:

```
-- ====================================================================
-- Tầng ngữ nghĩa (semantic layer) cho dashboard Superset.
-- Đầu vào : schema analytics (16 bảng output Analytics) + gold (fact/agg).
-- Đầu ra  : schema presentation, mỗi view phục vụ 1 nhóm biểu đồ.
-- Idempotent: chạy lại bao nhiêu lần cũng được (loader tự chạy sau khi nạp).
-- Quy tắc bàn giao được giữ nguyên:
--   * chuỗi thời gian 2017-01 → 2018-08 (kpi_monthly đã cắt 2016 và 2018-09)
--   * đếm khách bằng customer_unique_id
--   * review lấy ở cấp đơn (fact_orders / insight_* / kpi_monthly), KHÔNG từ
fact_order_items
-- ====================================================================

DROP SCHEMA IF EXISTS presentation CASCADE;
CREATE SCHEMA presentation;

-- ---------------------------------------------------------------------
-- Dimension bang Brazil: tên, vùng, mã ISO 3166-2 (khớp geojson Country Map của
Superset)
-- ---------------------------------------------------------------------
CREATE TABLE presentation.dim_state (
    customer_state char(2) PRIMARY KEY,
    state_name     text NOT NULL,
    region         text NOT NULL,
    iso_code       text NOT NULL
);
INSERT INTO presentation.dim_state (customer_state, state_name, region, iso_code)
SELECT s, n, r, 'BR-' || s
FROM (VALUES
    ('AC','Acre','Norte'), ('AL','Alagoas','Nordeste'), ('AP','Amapá','Norte'),
    ('AM','Amazonas','Norte'), ('BA','Bahia','Nordeste'), ('CE','Ceará','Nordeste'),
    ('DF','Distrito Federal','Centro-Oeste'), ('ES','Espírito Santo','Sudeste'),
    ('GO','Goiás','Centro-Oeste'), ('MA','Maranhão','Nordeste'), ('MT','Mato
Grosso','Centro-Oeste'),
    ('MS','Mato Grosso do Sul','Centro-Oeste'), ('MG','Minas Gerais','Sudeste'),
('PA','Pará','Norte'),
    ('PB','Paraíba','Nordeste'), ('PR','Paraná','Sul'), ('PE','Pernambuco','Nordeste'),
    ('PI','Piauí','Nordeste'), ('RJ','Rio de Janeiro','Sudeste'), ('RN','Rio Grande do
Norte','Nordeste'),
    ('RS','Rio Grande do Sul','Sul'), ('RO','Rondônia','Norte'),
('RR','Roraima','Norte'),
    ('SC','Santa Catarina','Sul'), ('SP','São Paulo','Sudeste'),
('SE','Sergipe','Nordeste'),
    ('TO','Tocantins','Norte')
) AS t(s, n, r);

-- ====================================================================
-- Chủ đề 1. Tăng trưởng & vận hành
-- ====================================================================
CREATE VIEW presentation.v_kpi_monthly AS
SELECT make_date(k.purchase_year, k.purchase_month, 1)                    AS
month_date,
       to_char(make_date(k.purchase_year, k.purchase_month, 1), 'YYYY-MM') AS
month_label,
       k.purchase_year, k.purchase_month,
       k.orders, k.customers, k.items, k.revenue, k.freight, k.gmv,
       k.avg_order_value, k.avg_review_score, k.avg_delivery_days, k.late_delivery_pct,
       k.revenue_mom_pct, k.orders_mom_pct, k.revenue_yoy_pct, k.revenue_rank,
       k.is_black_friday_month
FROM analytics.kpi_monthly k;

-- Đơn hợp lệ (cấp đơn) trong khung KPI: nguồn cho thẻ KPI có bộ lọc tháng/bang/PTTT.
```

```
CREATE VIEW presentation.v_orders AS
SELECT o.order_id,
       o.customer_unique_id,
       o.purchase_date,
       date_trunc('month', o.purchase_date)::date             AS month_date,
       o.customer_state,
       s.region,
       o.order_status,
       COALESCE(o.main_payment_type, 'unknown')               AS main_payment_type,
       o.max_installments,
       o.items,
       o.items_value                                          AS revenue,
       o.freight_value                                        AS freight,
       o.order_value,
       o.delivery_days,
       o.delivery_delay_days,
       CASE WHEN o.is_late THEN 1 WHEN NOT o.is_late THEN 0 END AS late_flag,
       o.review_score
FROM gold.fact_orders o
LEFT JOIN presentation.dim_state s USING (customer_state)
WHERE o.is_valid_sale
  AND o.purchase_date >= DATE '2017-01-01'
  AND o.purchase_date <  DATE '2018-09-01';

-- Thẻ KPI "toàn kỳ" và số neo (1 dòng).
CREATE VIEW presentation.v_kpi_overview AS
SELECT (SELECT SUM(revenue) FROM gold.agg_sales_monthly)                        AS
gold_revenue_total,
       (SELECT SUM(orders)  FROM gold.agg_sales_monthly)                        AS
gold_valid_orders,
       (SELECT SUM(revenue) FROM analytics.kpi_monthly)                         AS
kpi_window_revenue,
       (SELECT COUNT(*)     FROM analytics.kpi_monthly)                         AS
kpi_months,
       (SELECT COUNT(*)     FROM analytics.rfm_scores)                          AS
customers,
       (SELECT customers FROM analytics.repeat_profile WHERE cohort = 'repeat') AS
repeat_customers,
       (SELECT pct       FROM analytics.repeat_profile WHERE cohort = 'repeat') AS
repeat_pct,
       (SELECT month_label FROM presentation.v_kpi_monthly ORDER BY revenue DESC LIMIT
1)     AS peak_month,
       (SELECT revenue_mom_pct FROM presentation.v_kpi_monthly ORDER BY revenue DESC
LIMIT 1) AS peak_mom_pct,
       (SELECT avg_review FROM analytics.insight_late_vs_review WHERE delivery =
'late')    AS late_review,
       (SELECT avg_review FROM analytics.insight_late_vs_review WHERE delivery =
'on_time') AS on_time_review,
       (SELECT COUNT(*) FROM analytics.seller_pareto)                           AS
sellers,
       (SELECT COUNT(*) FROM analytics.seller_pareto WHERE cumulative_share_pct <= 80)
AS sellers_80pct_revenue,
       (SELECT COUNT(*) FROM analytics.seller_watchlist)                        AS
watchlist_sellers;

-- ====================================================================
-- Chủ đề 2. Danh mục & mùa vụ
-- ====================================================================
-- category_label có tiền tố hạng ("01 health_beauty") để trục X của chart Pareto sắp
đúng thứ tự doanh thu.
CREATE VIEW presentation.v_category_pareto AS
SELECT c.*,
       ROW_NUMBER() OVER (ORDER BY c.revenue DESC) AS revenue_rank,
       lpad((ROW_NUMBER() OVER (ORDER BY c.revenue DESC))::text, 2, '0') || ' ' ||
c.category AS category_label,
```

```
       CASE WHEN c.cumulative_share_pct <= 80 THEN 'A (≤80% doanh thu)'
            WHEN c.cumulative_share_pct <= 95 THEN 'B (80–95%)'
            ELSE 'C (đuôi dài)' END                AS pareto_class
FROM analytics.category_pareto c;

-- revenue_rank lấy từ Pareto để chart lọc top N mà không cần sub-query (Superset chặn
sub-query trong filter).
CREATE VIEW presentation.v_category_seasonality AS
SELECT make_date(2017, s.purchase_month, 1) AS month_date,
       s.category, s.purchase_month, s.revenue, s.orders, s.pct_of_category_year,
       p.revenue_rank
FROM analytics.category_seasonality s
LEFT JOIN presentation.v_category_pareto p USING (category);

-- Doanh thu danh mục theo tháng (gold agg, 1.282 dòng) cho biểu đồ xu hướng có bộ lọc
danh mục.
CREATE VIEW presentation.v_category_monthly AS
SELECT make_date(a.purchase_year, a.purchase_month, 1) AS month_date,
       a.category, a.orders, a.items, a.revenue, a.freight, a.avg_price
FROM gold.agg_category_monthly a
WHERE make_date(a.purchase_year, a.purchase_month, 1) BETWEEN DATE '2017-01-01' AND
DATE '2018-08-01';

-- Dòng sản phẩm hợp lệ (doanh thu/ship theo danh mục × bang × tháng). Không chứa
review (quy tắc 3).
CREATE VIEW presentation.v_sales_items AS
SELECT i.order_id,
       i.order_item_id,
       date_trunc('month', i.purchase_date)::date AS month_date,
       i.category,
       i.customer_state,
       s.region,
       i.seller_id,
       i.seller_state,
       i.price         AS revenue,
       i.freight_value AS freight,
       i.distance_km
FROM gold.fact_order_items i
LEFT JOIN presentation.dim_state s USING (customer_state)
WHERE i.is_valid_sale
  AND i.purchase_date >= DATE '2017-01-01'
  AND i.purchase_date <  DATE '2018-09-01';

-- ====================================================================
-- Chủ đề 3. Địa lý & logistics
-- ====================================================================
CREATE VIEW presentation.v_state_logistics AS
SELECT l.customer_state, d.state_name, d.region, d.iso_code,
       l.orders, l.customers, l.revenue, l.gmv, l.avg_order_value, l.avg_freight,
       l.freight_pct_of_revenue, l.avg_delivery_days, l.late_delivery_pct,
l.avg_review_score,
       l.revenue_share_pct, l.revenue_rank
FROM analytics.state_logistics l
JOIN presentation.dim_state d USING (customer_state);
-- ... (cắt, tổng 374 dòng)
```

Tệp `sql/90_anchor_checks.sql`:

```
-- Kiểm tra số neo của ANALYTICS_HANDOVER.md trên dữ liệu ĐÃ NẠP vào Neon.
-- Định dạng: mỗi khối bắt đầu bằng "-- check: <tên> | expected: <giá trị|info> | tol:
<sai số>"
-- Chạy bằng: docker compose run --rm tools python loader/validate.py

-- check: Gold SUM(revenue) – agg_sales_monthly | expected: 13494400.74 | tol: 0.005
```

```
SELECT SUM(revenue) FROM gold.agg_sales_monthly;

-- check: Gold SUM(price) – fact_order_items (is_valid_sale) | expected: 13494400.74 |
tol: 0.005
SELECT SUM(price) FROM gold.fact_order_items WHERE is_valid_sale;

-- check: kpi_monthly có 20 tháng | expected: 20 | tol: 0
SELECT COUNT(*) FROM analytics.kpi_monthly;

-- check: kpi_monthly không chứa 2016 và 2018-09 | expected: 0 | tol: 0
SELECT COUNT(*) FROM analytics.kpi_monthly
WHERE purchase_year = 2016 OR (purchase_year = 2018 AND purchase_month >= 9);

-- check: Tổng revenue kpi_monthly (nhỏ hơn gold vì cắt tháng rỗng) | expected: info |
tol: 0
SELECT SUM(revenue) FROM analytics.kpi_monthly;

-- check: Tháng đỉnh doanh thu | expected: 2017-11 | tol: 0
SELECT month_label FROM presentation.v_kpi_monthly ORDER BY revenue DESC LIMIT 1;

-- check: MoM tháng đỉnh (%) | expected: 52.06 | tol: 0.005
SELECT revenue_mom_pct FROM presentation.v_kpi_monthly ORDER BY revenue DESC LIMIT 1;

-- check: Review đơn trễ | expected: 2.56 | tol: 0.005
SELECT avg_review FROM analytics.insight_late_vs_review WHERE delivery = 'late';

-- check: Review đơn đúng hạn | expected: 4.29 | tol: 0.005
SELECT avg_review FROM analytics.insight_late_vs_review WHERE delivery = 'on_time';

-- check: Số khách mua lại (frequency ≥ 2) | expected: 2887 | tol: 0
SELECT COUNT(*) FROM analytics.rfm_scores WHERE frequency >= 2;

-- check: Tổng khách thật (customer_unique_id) | expected: 94983 | tol: 0
SELECT COUNT(DISTINCT customer_unique_id) FROM analytics.rfm_scores;

-- check: Tỷ lệ khách mua lại (%) | expected: 3.04 | tol: 0.005
SELECT ROUND(100.0 * COUNT(*) FILTER (WHERE frequency >= 2) / COUNT(*), 2) FROM
analytics.rfm_scores;

-- check: Tỷ lệ frequency = 1 (%) | expected: 96.96 | tol: 0.005
SELECT ROUND(100.0 * COUNT(*) FILTER (WHERE frequency = 1) / COUNT(*), 2) FROM
analytics.rfm_scores;

-- check: Seller watchlist | expected: 153 | tol: 0
SELECT COUNT(*) FROM analytics.seller_watchlist;

-- check: Số seller tạo ≤80% doanh thu | expected: 539 | tol: 0
SELECT COUNT(*) FROM analytics.seller_pareto WHERE cumulative_share_pct <= 80;

-- check: Nhất quán: SUM(revenue) v_orders = SUM(revenue) kpi_monthly | expected: 0 |
tol: 0.005
SELECT (SELECT SUM(revenue) FROM presentation.v_orders) - (SELECT SUM(revenue) FROM
analytics.kpi_monthly);

-- check: Nhất quán: COUNT đơn v_orders = SUM(orders) kpi_monthly | expected: 0 | tol:
0
SELECT (SELECT COUNT(*) FROM presentation.v_orders) - (SELECT SUM(orders) FROM
analytics.kpi_monthly);

-- check: Nhất quán: SUM(revenue) v_sales_items = SUM(revenue) kpi_monthly | expected:
0 | tol: 0.005
SELECT (SELECT SUM(revenue) FROM presentation.v_sales_items) - (SELECT SUM(revenue)
FROM analytics.kpi_monthly);

-- check: Nhất quán: tổng khách theo segment = tổng khách RFM | expected: 94983 | tol:
```

```
0
SELECT SUM(customers) FROM presentation.v_rfm_segments;

-- check: Số dòng gợi ý quyết định | expected: 7 | tol: 0
SELECT COUNT(*) FROM presentation.v_decision_hints;
```

Phụ lục E. Minh chứng thực nghiệm chi tiết

E.1. Kết quả và log nạp dữ liệu

Bảng E.1. Kết quả nạp từng bảng, lần chạy `20261112-145508-40efaf` (tệp `evidence/load_log.csv`)

Bảng	Số dòng	Parquet (MB)	CSV truyền (MB)	COPY (s)	PK/index + ANALYZE (s)	Tổng (s)	MB/s COPY	Khớp
analytics.kpi_monthly	20	0,01	0,00	0,79	0,68	1,49	0,00	✓
analytics.category_pareto	74	0,01	0,01	1,08	0,60	1,70	0,01	✓
analytics.category_seasonality	715	0,01	0,03	1,04	0,71	1,76	0,03	✓
analytics.state_logistics	27	0,01	0,00	0,88	0,53	1,42	0,00	✓
analytics.insight_distance	4	0,00	0,00	0,96	0,56	1,54	0,00	✓
analytics.rfm_scores	94.983	3,99	11,49	18,39	1,14	19,63	0,62	✓
analytics.repeat_profile	2	0,00	0,00	0,93	0,58	1,53	0,00	✓
analytics.repeat_categories	70	0,00	0,00	0,97	0,59	1,58	0,00	✓
analytics.repeat_states	27	0,00	0,00	0,84	0,70	1,55	0,00	✓
analytics.customer_segments	94.983	4,01	11,87	26,73	1,36	28,19	0,44	✓
analytics.cluster_profile	4	0,00	0,00	0,81	0,60	1,42	0,00	✓
analytics.seller_pareto	3.053	0,16	0,30	1,28	0,83	2,13	0,23	✓
analytics.seller_watchlist	153	0,01	0,02	0,84	0,70	1,56	0,02	✓
analytics.insight_late_vs_review	2	0,00	0,00	1,03	0,58	1,63	0,00	✓
analytics.delay_vs_review	5	0,00	0,00	3,38	0,59	3,99	0,00	✓
analytics.payment_mix	10	0,00	0,00	0,91	0,63	1,55	0,00	✓
gold.fact_orders	99.441	13,17	31,71	101,08	1,46	102,97	0,31	✓
gold.fact_order_items	112.650	16,93	42,19	154,98	2,83	158,29	0,27	✓
gold.agg_sales_monthly	24	0,01	0,00	0,93	0,58	1,53	0,00	✓
gold.agg_category_monthly	1.282	0,03	0,09	0,97	0,52	1,50	0,10	✓
gold.agg_customer_state	27	0,01	0,00	1,09	0,54	1,65	0,00	✓
gold.agg_seller_performance	3.053	0,20	0,43	1,20	0,55	1,77	0,35	✓
gold.agg_customer_rfm	94.983	4,32	9,21	9,70	1,18	10,96	0,95	✓
Tổng	**505.592**	42,88	107,36	330,80	19,06	**351,34**	0,32	

Tệp `evidence/load_run.log`:

```
[2026-11-12 14:55:08] Run 20261112-145508-40efaf ->
postgresql://olist_analytics_owner:****@ep-flat-king-aoupo1l3.c-2.ap-southeast-
1.aws.neon.tech/olist_analytics?sslmode=require
[2026-11-12 14:55:09] PostgreSQL 18.6 (ac70818) on aarch64-unknown-linux-gnu, compiled
by gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, 64-bit
[2026-11-12 14:55:11] OK  analytics.kpi_monthly              rows=     20  parquet=
0.01MB/ 1f  read= 0.02s copy= 0.79s idx= 0.68s         25 rows/s
[2026-11-12 14:55:13] OK  analytics.category_pareto          rows=     74  parquet=
0.01MB/ 1f  read= 0.02s copy= 1.08s idx= 0.60s         69 rows/s
[2026-11-12 14:55:15] OK  analytics.category_seasonality     rows=    715  parquet=
0.01MB/ 1f  read= 0.02s copy= 1.04s idx= 0.71s        689 rows/s
[2026-11-12 14:55:16] OK  analytics.state_logistics          rows=     27  parquet=
0.01MB/ 1f  read= 0.01s copy= 0.88s idx= 0.53s         31 rows/s
[2026-11-12 14:55:18] OK  analytics.insight_distance         rows=      4  parquet=
0.00MB/ 1f  read= 0.02s copy= 0.96s idx= 0.56s          4 rows/s
[2026-11-12 14:55:37] OK  analytics.rfm_scores               rows= 94,983  parquet=
3.99MB/ 1f  read= 0.05s copy=18.39s idx= 1.14s      5,164 rows/s
[2026-11-12 14:55:39] OK  analytics.repeat_profile           rows=      2  parquet=
0.00MB/ 1f  read= 0.01s copy= 0.93s idx= 0.58s          2 rows/s
[2026-11-12 14:55:41] OK  analytics.repeat_categories        rows=     70  parquet=
0.00MB/ 1f  read= 0.02s copy= 0.97s idx= 0.59s         72 rows/s
[2026-11-12 14:55:43] OK  analytics.repeat_states            rows=     27  parquet=
0.00MB/ 1f  read= 0.01s copy= 0.84s idx= 0.70s         32 rows/s
```

```
[2026-11-12 14:56:11] OK  analytics.customer_segments        rows= 94,983  parquet=
4.01MB/ 1f  read= 0.05s copy=26.73s idx= 1.36s      3,553 rows/s
[2026-11-12 14:56:12] OK  analytics.cluster_profile          rows=      4  parquet=
0.00MB/ 1f  read= 0.01s copy= 0.81s idx= 0.60s          5 rows/s
[2026-11-12 14:56:15] OK  analytics.seller_pareto            rows=  3,053  parquet=
0.16MB/ 1f  read= 0.02s copy= 1.28s idx= 0.83s      2,386 rows/s
[2026-11-12 14:56:16] OK  analytics.seller_watchlist         rows=    153  parquet=
0.01MB/ 1f  read= 0.02s copy= 0.84s idx= 0.70s        183 rows/s
[2026-11-12 14:56:18] OK  analytics.insight_late_vs_review   rows=      2  parquet=
0.00MB/ 1f  read= 0.02s copy= 1.03s idx= 0.58s          2 rows/s
[2026-11-12 14:56:22] OK  analytics.delay_vs_review          rows=      5  parquet=
0.00MB/ 1f  read= 0.02s copy= 3.38s idx= 0.59s          2 rows/s
[2026-11-12 14:56:24] OK  analytics.payment_mix              rows=     10  parquet=
0.00MB/ 1f  read= 0.01s copy= 0.91s idx= 0.63s         11 rows/s
[2026-11-12 14:58:07] OK  gold.fact_orders                   rows= 99,441  parquet=
13.17MB/25f  read= 0.33s copy=101.08s idx= 1.46s        984 rows/s
[2026-11-12 15:00:45] OK  gold.fact_order_items              rows=112,650  parquet=
16.93MB/24f  read= 0.35s copy=154.98s idx= 2.83s        727 rows/s
[2026-11-12 15:00:47] OK  gold.agg_sales_monthly             rows=     24  parquet=
0.01MB/ 1f  read= 0.02s copy= 0.93s idx= 0.58s         26 rows/s
[2026-11-12 15:00:49] OK  gold.agg_category_monthly          rows=  1,282  parquet=
0.03MB/ 1f  read= 0.01s copy= 0.97s idx= 0.52s      1,319 rows/s
[2026-11-12 15:00:51] OK  gold.agg_customer_state            rows=     27  parquet=
0.01MB/ 1f  read= 0.02s copy= 1.09s idx= 0.54s         25 rows/s
[2026-11-12 15:00:52] OK  gold.agg_seller_performance        rows=  3,053  parquet=
0.20MB/ 1f  read= 0.02s copy= 1.20s idx= 0.55s      2,543 rows/s
[2026-11-12 15:01:04] OK  gold.agg_customer_rfm              rows= 94,983  parquet=
4.32MB/ 1f  read= 0.06s copy= 9.70s idx= 1.18s      9,794 rows/s
[2026-11-12 15:01:04] ------------------------------------------------------------
------------------------------------
[2026-11-12 15:01:04] Tổng: 23 bảng, 505,592 dòng, Parquet 42.88 MB
[2026-11-12 15:01:04] Thời gian nạp (đọc+COPY+index): 355.83s; riêng COPY 330.80s ->
1,528 rows/s; end-to-end 1,421 rows/s
[2026-11-12 15:01:04] Đối chiếu số dòng Parquet vs Postgres: 23/23 khớp
[2026-11-12 15:01:04] Tổng thời gian chạy: 355.84s
```

E.2. Log áp dụng tầng ngữ nghĩa

Tệp `evidence/apply_sql.log`:

```
[2026-11-12 15:59:40] SQL 10_presentation_views.sql áp dụng trong 0.49s
[2026-11-12 15:59:40]   presentation.dim_state                  BASE TABLE
[2026-11-12 15:59:40]   presentation.v_category_monthly         VIEW
[2026-11-12 15:59:40]   presentation.v_category_pareto          VIEW
[2026-11-12 15:59:40]   presentation.v_category_seasonality     VIEW
[2026-11-12 15:59:40]   presentation.v_cluster_profile          VIEW
[2026-11-12 15:59:40]   presentation.v_decision_hints           VIEW
[2026-11-12 15:59:40]   presentation.v_delay_vs_review          VIEW
[2026-11-12 15:59:40]   presentation.v_insight_distance         VIEW
[2026-11-12 15:59:40]   presentation.v_kpi_monthly              VIEW
[2026-11-12 15:59:40]   presentation.v_kpi_overview             VIEW
[2026-11-12 15:59:40]   presentation.v_late_vs_review           VIEW
[2026-11-12 15:59:40]   presentation.v_load_log                 VIEW
[2026-11-12 15:59:40]   presentation.v_orders                   VIEW
[2026-11-12 15:59:40]   presentation.v_payment_mix              VIEW
[2026-11-12 15:59:40]   presentation.v_repeat_categories        VIEW
[2026-11-12 15:59:40]   presentation.v_repeat_profile           VIEW
[2026-11-12 15:59:40]   presentation.v_repeat_states            VIEW
[2026-11-12 15:59:40]   presentation.v_rfm_customers            VIEW
[2026-11-12 15:59:40]   presentation.v_rfm_segments             VIEW
[2026-11-12 15:59:40]   presentation.v_sales_items              VIEW
[2026-11-12 15:59:40]   presentation.v_seller_pareto            VIEW
[2026-11-12 15:59:40]   presentation.v_seller_watchlist         VIEW
```

```
[2026-11-12 15:59:40]   presentation.v_state_logistics          VIEW
[2026-11-12 15:59:40] 23 đối tượng trong schema presentation
```

E.3. Đối chiếu số dòng và giá trị kỳ vọng

Bảng E.2. Đối chiếu số dòng Parquet ↔ Neon: 23/23 bảng khớp (tệp `evidence/validation.json`)

Bảng trên Neon	Dòng Parquet	Dòng Postgres	Kết quả
analytics.kpi_monthly	20	20	ĐẠT
analytics.category_pareto	74	74	ĐẠT
analytics.category_seasonality	715	715	ĐẠT
analytics.state_logistics	27	27	ĐẠT
analytics.insight_distance	4	4	ĐẠT
analytics.rfm_scores	94.983	94.983	ĐẠT
analytics.repeat_profile	2	2	ĐẠT
analytics.repeat_categories	70	70	ĐẠT
analytics.repeat_states	27	27	ĐẠT
analytics.customer_segments	94.983	94.983	ĐẠT
analytics.cluster_profile	4	4	ĐẠT
analytics.seller_pareto	3.053	3.053	ĐẠT
analytics.seller_watchlist	153	153	ĐẠT
analytics.insight_late_vs_review	2	2	ĐẠT
analytics.delay_vs_review	5	5	ĐẠT
analytics.payment_mix	10	10	ĐẠT
gold.fact_orders	99.441	99.441	ĐẠT
gold.fact_order_items	112.650	112.650	ĐẠT
gold.agg_sales_monthly	24	24	ĐẠT
gold.agg_category_monthly	1.282	1.282	ĐẠT
gold.agg_customer_state	27	27	ĐẠT
gold.agg_seller_performance	3.053	3.053	ĐẠT
gold.agg_customer_rfm	94.983	94.983	ĐẠT

Bảng E.3. Bộ kiểm tra đối chiếu Parquet ↔ Neon và giá trị kỳ vọng

Kiểm tra	Giá trị kỳ vọng (từ Parquet)	Ghi chú
SUM(revenue) `kpi_monthly` (20 tháng)	13.449.529,68	Nhỏ hơn 13.494.400,74 vì đã cắt tháng 2016 và 09/2018
SUM(revenue) `state_logistics`	13.494.400,74	Số neo gold toàn kỳ
SUM(revenue) `seller_pareto`	13.494.400,74	Tổng theo người bán
SUM(revenue) `category_pareto`	13.449.529,68	Tổng theo danh mục
SUM(customers) `repeat_profile`	94.983	= số khách thật (customer_unique_id)
COUNT(*) `analytics.kpi_monthly`	20	Số dòng Parquet = số dòng bảng Neon
COUNT(*) `analytics.category_pareto`	74	Số dòng Parquet = số dòng bảng Neon
COUNT(*) `analytics.category_seasonality`	715	Số dòng Parquet = số dòng bảng Neon
COUNT(*) `analytics.state_logistics`	27	Số dòng Parquet = số dòng bảng Neon
COUNT(*) `analytics.insight_distance`	4	Số dòng Parquet = số dòng bảng Neon
COUNT(*) `analytics.rfm_scores`	94.983	Số dòng Parquet = số dòng bảng Neon
COUNT(*) `analytics.customer_segments`	94.983	Số dòng Parquet = số dòng bảng Neon
COUNT(*) `analytics.cluster_profile`	4	Số dòng Parquet = số dòng bảng Neon

Kiểm tra	Giá trị kỳ vọng (từ Parquet)	Ghi chú
COUNT(*) analytics.repeat_profile	2	Số dòng Parquet = số dòng bảng Neon
COUNT(*) analytics.repeat_categories	70	Số dòng Parquet = số dòng bảng Neon
COUNT(*) analytics.repeat_states	27	Số dòng Parquet = số dòng bảng Neon
COUNT(*) analytics.seller_pareto	3.053	Số dòng Parquet = số dòng bảng Neon
COUNT(*) analytics.seller_watchlist	153	Số dòng Parquet = số dòng bảng Neon
COUNT(*) analytics.insight_late_vs_review	2	Số dòng Parquet = số dòng bảng Neon
COUNT(*) analytics.delay_vs_review	5	Số dòng Parquet = số dòng bảng Neon
COUNT(*) analytics.payment_mix	10	Số dòng Parquet = số dòng bảng Neon

E.4. Danh sách tệp minh chứng

Bảng E.4. Danh sách tệp minh chứng trong evidence/

Tệp	Loại	KB
evidence/apply_sql.log	LOG	1,7
evidence/architecture_pipeline.png	Ảnh	55,7
evidence/dashboard_load_latency.md	MD	2,5
evidence/dashboard_load_summary.json	JSON	0,3
evidence/dashboard_load_time.csv	CSV	0,6
evidence/dashboard_tab1_tong_quan_xu_huong.png	Ảnh	240,0
evidence/dashboard_tab2_danh_muc_mua_vu.png	Ảnh	342,1
evidence/dashboard_tab3_dia_ly_logistics.png	Ảnh	238,5
evidence/dashboard_tab4_khach_hang_rfm.png	Ảnh	222,0
evidence/dashboard_tab5_seller.png	Ảnh	52,8
evidence/dashboard_tab6_giao_tre_review_thanh_toan.png	Ảnh	204,0
evidence/dashboard_tab7_goi_y_quyet_dinh_nguon_du_lieu.png	Ảnh	262,2
evidence/filter_tab1_thoi_gian_2018_bang_SP.png	Ảnh	205,2
evidence/filter_tab2_danh_muc_bed_bath_health_watches.png	Ảnh	236,9
evidence/filter_tab3_vung_nordeste.png	Ảnh	234,6
evidence/filter_tab4_rfm_khach_mua_lai_bang_RJ.png	Ảnh	242,0
evidence/filter_tab5_bang_seller_SP.png	Ảnh	229,5
evidence/filter_tab6_pttt_boleto.png	Ảnh	241,5
evidence/load_log.csv	CSV	3,2
evidence/load_run.log	LOG	4,1
evidence/neon_inventory.csv	CSV	3,8
evidence/neon_inventory.html	HTML	11,3
evidence/neon_project.json	JSON	0,5
evidence/neon_tables_inventory.png	Ảnh	301,8
evidence/parquet_schema_inspect.txt	TXT	23,0
evidence/query_latency.csv	CSV	1,7
evidence/query_latency.md	MD	2,8
evidence/scale_experiment.csv	CSV	0,3
evidence/scale_experiment.md	MD	1,2
evidence/superset_chart_latency.csv	CSV	2,0
evidence/validation.json	JSON	8,3
evidence/validation.md	MD	3,1
evidence/watchlist_goi_y_quyet_dinh.png	Ảnh	62,0
evidence/watchlist_seller_canh_bao.png	Ảnh	110,0

Phụ lục F. Mã nguồn

- Kho mã nguồn của nhóm: https://github.com/viettuong287/BigDataProject
- Repo giai đoạn trước (Analytics): **[[CẬP NHẬT: URL repo Analytics]]**; repo Processing: https://github.com/NguyenAlpha/big-data
- Thư mục dữ liệu đầu vào được dùng khi sinh báo cáo: `D:/BigDataProject/data/analytics`

```
BigDataProject/
├── data/            (Parquet nhận từ Analytics: analytics/, gold/)
├── docs/
│   ├── architecture.md
│   ├── architecture.png
│   └── architecture_pipeline_position.png
├── evidence/
│   ├── apply_sql.log
│   ├── architecture_pipeline.png
│   ├── dashboard_load_latency.md
│   ├── dashboard_load_summary.json
│   ├── dashboard_load_time.csv
│   ├── dashboard_tab1_tong_quan_xu_huong.png
│   ├── dashboard_tab2_danh_muc_mua_vu.png
│   ├── dashboard_tab3_dia_ly_logistics.png
│   ├── dashboard_tab4_khach_hang_rfm.png
│   ├── dashboard_tab5_seller.png
│   ├── dashboard_tab6_giao_tre_review_thanh_toan.png
│   ├── dashboard_tab7_goi_y_quyet_dinh_nguon_du_lieu.png
│   ├── filter_tab1_thoi_gian_2018_bang_SP.png
│   ├── filter_tab2_danh_muc_bed_bath_health_watches.png
│   ├── filter_tab3_vung_nordeste.png
│   ├── filter_tab4_rfm_khach_mua_lai_bang_RJ.png
│   ├── filter_tab5_bang_seller_SP.png
│   ├── filter_tab6_pttt_boleto.png
│   ├── load_log.csv
│   ├── load_run.log
│   ├── neon_inventory.csv
│   ├── neon_inventory.html
│   ├── neon_project.json
│   ├── neon_tables_inventory.png
│   ├── parquet_schema_inspect.txt
│   ├── query_latency.csv
│   ├── query_latency.md
│   ├── scale_experiment.csv
│   ├── scale_experiment.md
│   ├── superset_chart_latency.csv
│   ├── validation.json
│   ├── validation.md
│   ├── watchlist_goi_y_quyet_dinh.png
│   └── watchlist_seller_canh_bao.png
├── loader/
│   ├── apply_sql.py
│   ├── common.py
│   ├── Dockerfile
│   ├── load_to_neon.py
│   ├── requirements.txt
│   └── validate.py
├── report/
│   ├── content/
```

```
│   ├── tools/
│   ├── .gitignore
│   ├── BaoCao_DoAn_BigData.docx
│   ├── BaoCao_DoAn_BigData.pdf
│   ├── build_report.py
│   ├── charts.py
│   ├── inputs.py
│   ├── placeholders.txt
│   └── requirements.txt
├── scripts/
│   ├── Dockerfile.screenshots
│   ├── inspect_parquet.py
│   ├── measure_latency.py
│   ├── neon_inventory.py
│   ├── render_diagrams.py
│   ├── scale_experiment.py
│   └── take_screenshots.py
├── sql/
│   ├── 10_presentation_views.sql
│   └── 90_anchor_checks.sql
├── superset/
│   ├── export/
│   ├── build_dashboard.py
│   ├── docker-init.sh
│   ├── Dockerfile
│   ├── import_dashboard.py
│   ├── superset_client.py
│   └── superset_config.py
├── .env.example
├── .gitignore
├── docker-compose.yml
└── README.md
```

Bảng F.1. Vị trí các thành phần trong repo

Thành phần	Đường dẫn
Loader Parquet → Neon	`loader/`
Mã SQL tầng ngữ nghĩa	`sql/`
Cấu hình và script Superset	`superset/`
Script tiện ích (đo latency, chụp ảnh, kiểm tra)	`scripts/`
Minh chứng (log, số đo, ảnh)	`evidence/`
Tài liệu, sơ đồ, tài liệu bàn giao upstream	`docs/`
Báo cáo (script sinh, nội dung, hình)	`report/`

Phụ lục G. Bảng phân công và đóng góp

Bảng G.1. Bảng phân công nhiệm vụ và mức đóng góp của các thành viên

STT	Họ và tên	MSSV	Vai trò	Nhiệm vụ thực tế	Đóng góp
1	**[[CẬP NHẬT: Họ tên TV1]]**	**[[CẬP NHẬT: MSSV]]**	Nạp dữ liệu & Neon (trưởng nhóm)	Khởi tạo project Neon, database, schema; Viết loader Parquet → Neon (COPY); Bảng load_log, kiểm tra số dòng	**[[CẬP NHẬT: %]]**
2	**[[CẬP NHẬT: Họ tên TV2]]**	**[[CẬP NHẬT: MSSV]]**	Semantic layer & SQL	Thiết kế schema presentation; Viết các view v_* và định nghĩa KPI; Đối chiếu số liệu với Analytics	**[[CẬP NHẬT: %]]**
3	**[[CẬP NHẬT: Họ tên TV3]]**	**[[CẬP NHẬT: MSSV]]**	Dashboard Superset	Triển khai Superset bằng Docker; Tạo datasets, charts, dashboard; Cấu hình native filters, gợi ý quyết định	**[[CẬP NHẬT: %]]**
4	**[[CẬP NHẬT: Họ tên TV4]]**	**[[CẬP NHẬT: MSSV]]**	Minh chứng & kiểm thử	Đo thời gian nạp, độ trễ truy vấn; Kiểm thử bộ lọc, chụp ảnh minh chứng; Kịch bản demo	**[[CẬP NHẬT: %]]**
5	**[[CẬP NHẬT: Họ tên TV5]]**	**[[CẬP NHẬT: MSSV]]**	Báo cáo & tài liệu	Viết báo cáo, sơ đồ kiến trúc; README, hướng dẫn chạy; Tổng hợp tài liệu tham khảo	**[[CẬP NHẬT: %]]**

Bảng G.2. Ma trận đóng góp theo sản phẩm bàn giao (C = chính, H = hỗ trợ)

Sản phẩm	TV1	TV2	TV3	TV4	TV5
Project Neon, loader, load_log	C	H		H	
View ngữ nghĩa, định nghĩa KPI	H	C	H		
Superset Docker, dashboard, bộ lọc		H	C	H	
Đo hiệu năng, kiểm thử, ảnh minh chứng	H		H	C	
Báo cáo, sơ đồ, README		H		H	C

[[CẬP NHẬT: điều chỉnh ma trận theo phân công thực tế và tỷ lệ % đóng góp đã thống nhất trong nhóm]]

Phụ lục H. Ghi chú sử dụng công cụ AI

Theo Mục 8 của yêu cầu đồ án, nhóm ghi nhận việc sử dụng AI tạo sinh như sau:

Bảng H.1. Ghi nhận sử dụng công cụ AI

Công cụ	Mục đích sử dụng	Phần nhóm đã kiểm tra / chỉnh sửa
Trợ lý lập trình AI trong Cursor	Gợi ý cấu trúc mã loader, script dựng dashboard qua REST API, Dockerfile	**[[CẬP NHẬT: thành viên kiểm tra, nội dung đã sửa]]**
Trợ lý lập trình AI trong Cursor	Sinh script `report/build_report.py`, biểu đồ matplotlib và bản nháp nội dung báo cáo từ tài liệu bàn giao	**[[CẬP NHẬT: thành viên kiểm tra số liệu, đã đối chiếu với analytics_report.md]]**
[[CẬP NHẬT: công cụ khác nếu có]]	**[[CẬP NHẬT]]**	**[[CẬP NHẬT]]**

Mọi số liệu trong báo cáo được lấy trực tiếp từ tệp Parquet, log và tệp đo của hệ thống (không do AI tự tạo); các quyết định kỹ thuật (chọn Neon, Superset, thiết kế schema, định nghĩa KPI, tiêu chí đánh giá) đã được nhóm thảo luận và có lập luận ở Chương 2, 4 và 6. Nhóm chịu trách nhiệm hoàn toàn về nội dung nộp và có thể giải thích từng thành phần của hệ thống.